

Multi-Key Homomorphic Secret Sharing

From Theory To Practice

Multi-Key Homomorphic Secret Sharing

Geoffroy Couteau, **Lali Devadas**, Aditya Hegde, Abhishek Jain, Sacha Servan-Schreiber



Roadmap

1. Summary of our contributions
 - a. Motivating example: two-party succinct secure computation
 - b. Define multi-key homomorphic secret sharing (MKHSS)
 - c. Application: non-interactive conditional key exchange
2. Background on HSS from DCR
3. Constructing MKHSS from the DCR assumption

next: Kevin will talk about optimized implementations of MKHSS/key exchange

Roadmap

1. Summary of our contributions

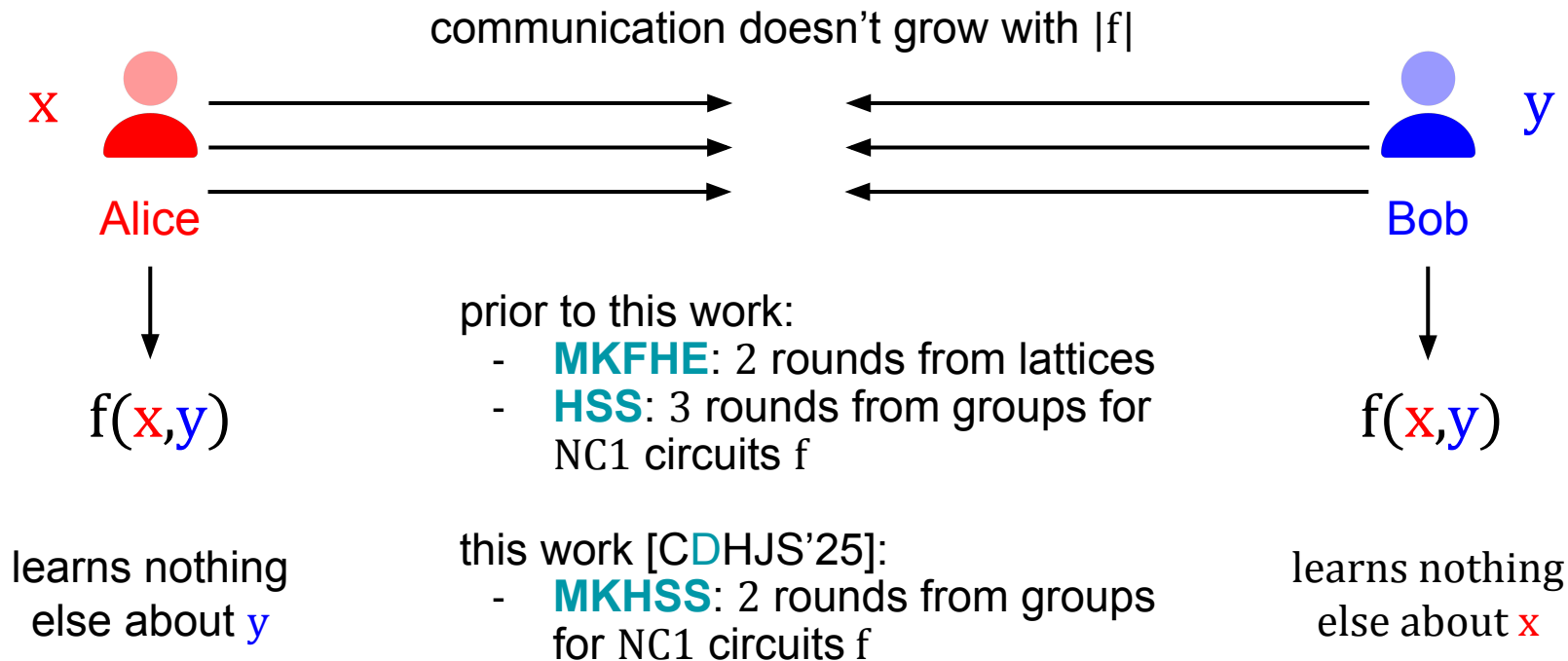
- a. Motivating example: two-party succinct secure computation
- b. Define multi-key homomorphic secret sharing (MKHSS)
- c. Application: non-interactive conditional key exchange

2. Background on HSS from DCR

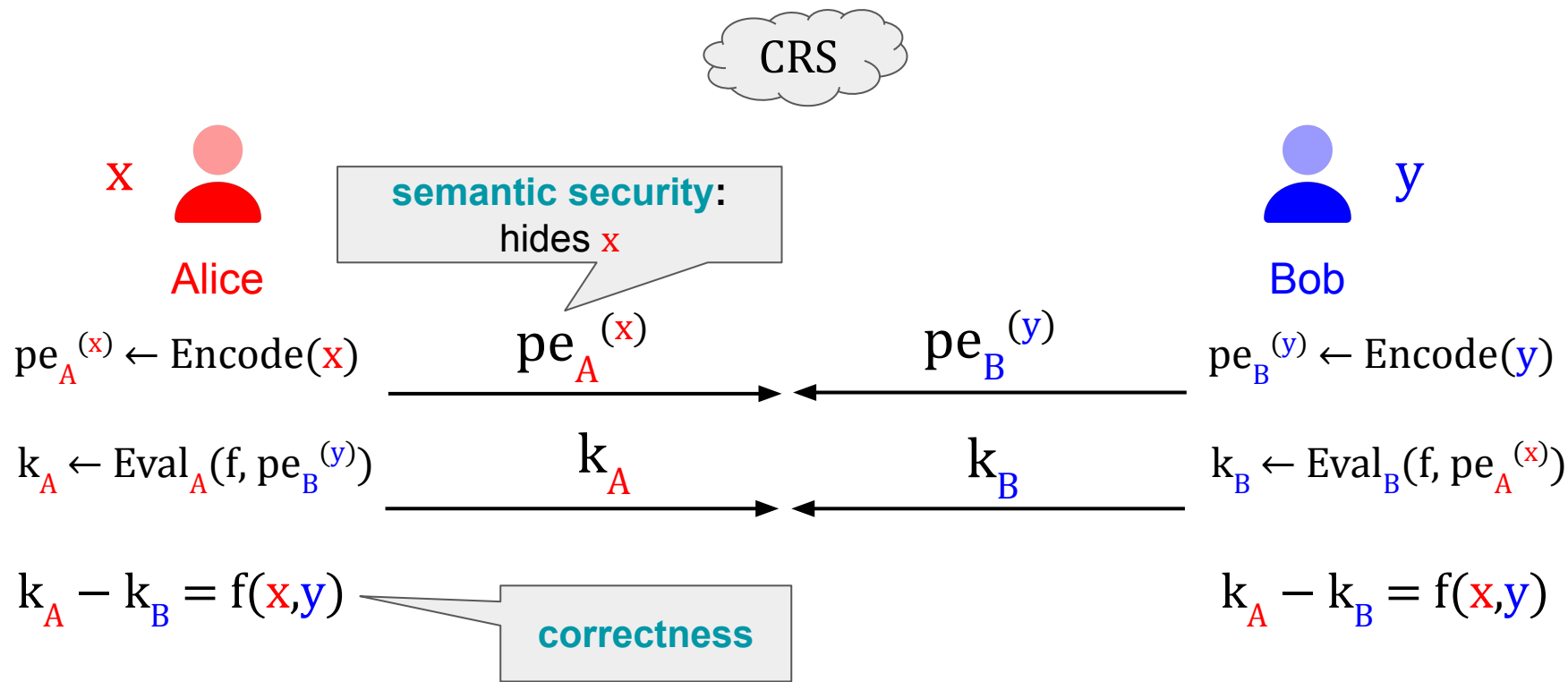
3. Constructing MKHSS from the DCR assumption

next: Kevin will talk about optimized implementations of MKHSS/key exchange

Motivation: Two-Party Succinct Secure Computation



Multi-Key HSS [CDHJS'25]



Our results [CDHJS'25]

we construct multi-key HSS for NC1 circuits from any of the following:

- Decisional Diffie-Hellman (DDH)
- DDH-like assumptions over class groups
- Decisional Composite Residuosity (DCR)



this talk

this the first two-round succinct secure computation protocol from group-based assumptions for NC1 circuits.

Applications [CDHJS'25]

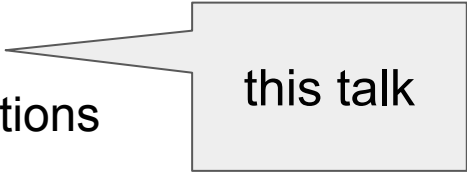
MKHSS achieves our goal of two-round succinct secure computation.

Q: after exchanging simultaneous messages, Alice and Bob have subtractive shares of the output – are there applications where this is sufficient?

A: yes!

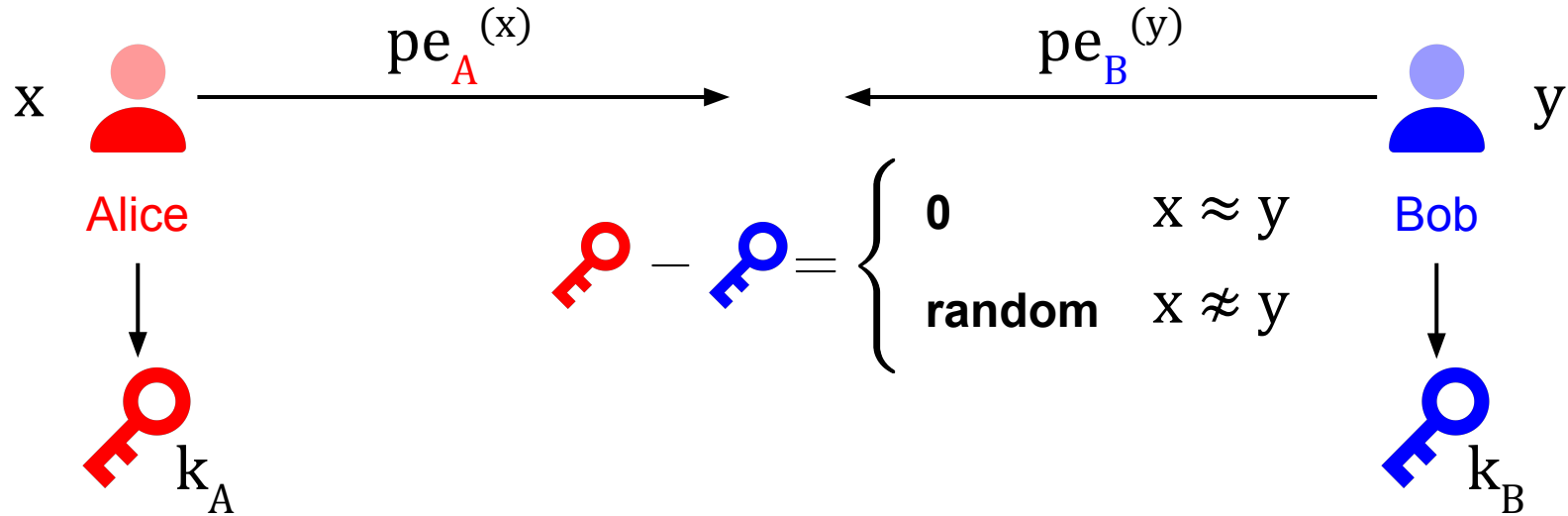
subtractive structure of shares also gives interesting *non-interactive* applications

- non-interactive conditional key exchange
- public-key pseudorandom correlation functions
- silent preprocessing secure computation

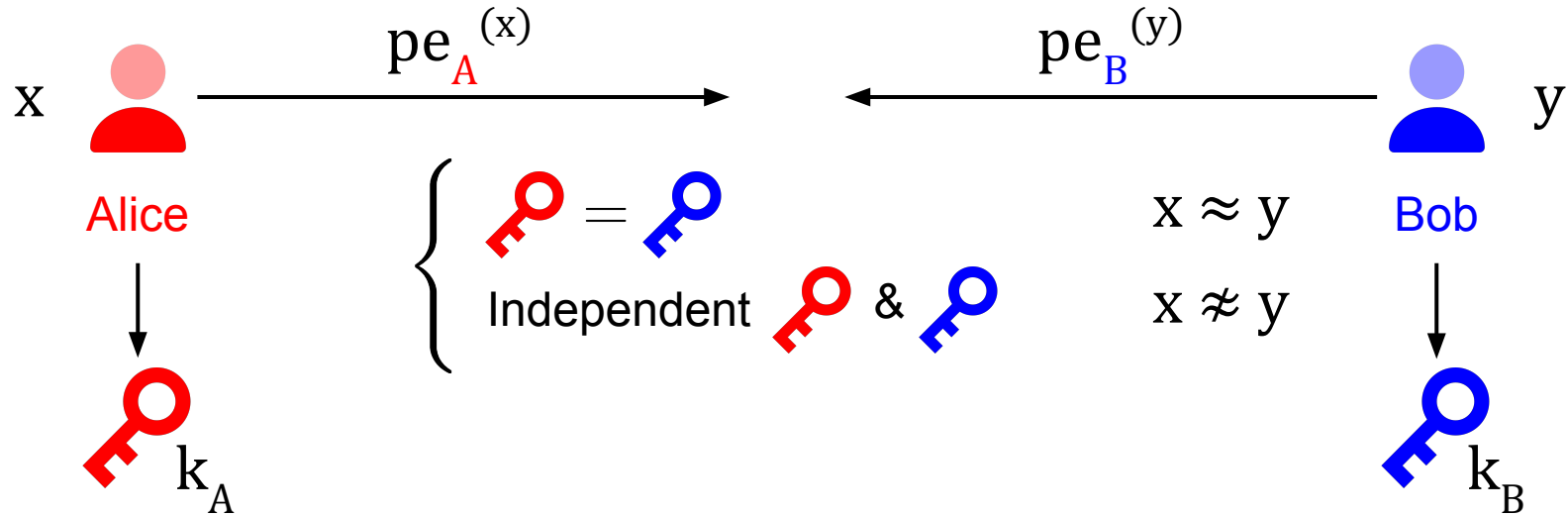


this talk

Application: Non-interactive Conditional Key Exchange

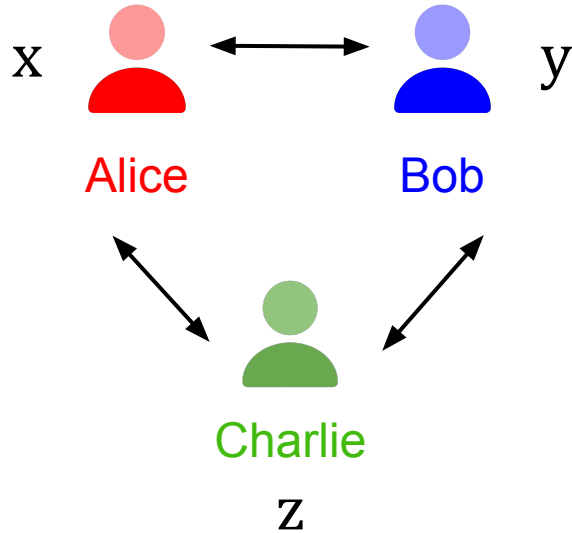


Application: Non-interactive Conditional Key Exchange

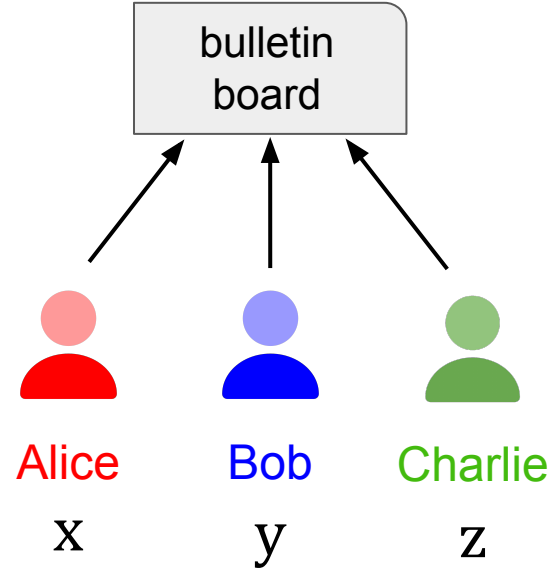


Added Benefit: Reusability

correlated setup



non-interactive setup



Roadmap

1. Summary of our contributions

- a. Motivating example: two-party succinct secure computation
- b. Define multi-key homomorphic secret sharing (MKHSS)
- c. Application: non-interactive conditional key exchange

2. Background on HSS from DCR

3. Constructing MKHSS from the DCR assumption

next: Kevin will talk about optimized implementations of MKHSS/key exchange

HSS template [Boyle-Gilboa-Ishai'16]

we make this setup
non-interactive

0. assume that Alice and Bob have pk and shares of sk
1. they exchange encodings of their inputs x and y under pk
2. they perform local computations to obtain shares of output $f(x,y)$

next we will see how *input encodings* and *local computations* work for HSS from DCR [OSY'21] (with some helpful modifications)

Input encodings: Paillier-ElGamal encryptions

N = product of two
safe primes
 g = generator of the
 $2N^{\text{th}}$ residue subgroup
of $Z_{N^2}^*$

$$\text{Input_Encode}(\text{pk}, x) = (\text{Enc}_{\text{pk}}(x \cdot \text{sk}), \text{Enc}_{\text{pk}}(x))$$

for HSS from DCR, these are Paillier-ElGamal encryptions [BCP'03]

$$\text{sk} \leftarrow \$ [N] \quad \text{pk} = g^{-\text{sk}} \bmod N^2 \quad \text{Enc}_{\text{pk}}(x) = (g^r \bmod N^2, \text{pk}^r (1+N)^x \bmod N^2)$$

we will use a “flipped encryption” for the other component

$$\text{Enc}_{\text{pk}}(x \cdot \text{sk}) = (g^r (1+N)^x \bmod N^2, \text{pk}^r \bmod N^2)$$

(this helps us later because it can be computed without knowing sk)

Input encodings: Paillier-ElGamal encryptions

another helpful note for later:

$$\text{pk} = g^{-\text{sk}} \quad \text{Enc}_{\text{pk}}(x) = (g^r, \text{pk}^r (1+N)^x)$$

what happens if we do $(\text{pk}^r (1+N)^x)^{\text{sk}'}$ for some sk' ?

we end up with a ciphertext of $x \cdot \text{sk}'$ with respect to public key $\text{pk}^{\text{sk}'}$:

$$(g^r, (\text{pk}^{\text{sk}'})^r ((1+N)^x)^{\text{sk}'}) = (g^r, (g^{\text{sk} \cdot \text{sk}'})^r (1+N)^{x \cdot \text{sk}'})$$

also a ciphertext which decrypts to $x \cdot \text{sk}'$ using secret key $\text{sk} \cdot \text{sk}'$.

morally multiplying message and secret key by same value sk'

Local computations: RMS multiplication

our HSS supports evaluating RMS multiplication programs:

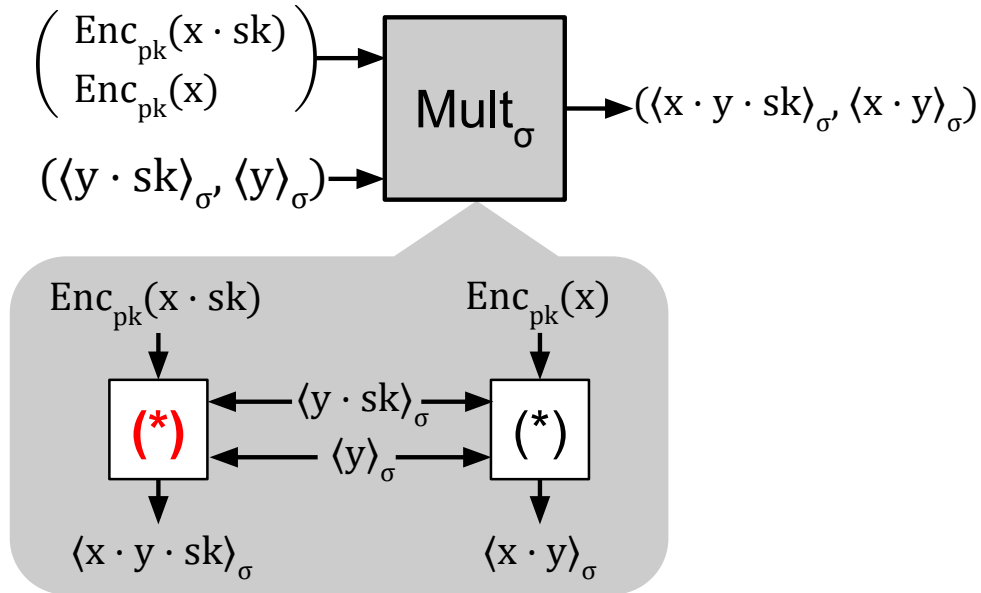
- start with *input encodings*
- intermediate computation values are computed as *memory shares*
- values held in memory shares can only be multiplied by values held in input encodings, not other values held in memory shares

input encoding of x : $\text{Input_Encode}(\text{pk}, x) = (\text{Enc}_{\text{pk}}(x \cdot \text{sk}), \text{Enc}_{\text{pk}}(x))$

memory share of y : *subtractive shares* $(\langle y \cdot \text{sk} \rangle_{\sigma}, \langle y \rangle_{\sigma})$ $y = \langle y \rangle_{\text{A}} - \langle y \rangle_{\text{B}}$

need to be able to compute a memory share of xy given these

RMS multiplication: high level idea [Boyle-Gilboa-Ishai'16]



High level idea of $(*)$:

- 1) Decrypt ciphertext
- 2) Multiply plaintext by y **in secret-shared form.**

OSY'21 shows how to do this for DCR encodings

this computation requires **modular exponentiations**

Roadmap

1. Summary of our contributions
 - a. Motivating example: two-party succinct secure computation
 - b. Define multi-key homomorphic secret sharing (MKHSS)
 - c. Application: non-interactive conditional key exchange
2. Background on HSS from DCR
3. Constructing MKHSS from the DCR assumption

next: Kevin will talk about optimized implementations of MKHSS/key exchange

Removing correlated setup [CDHJS'25]

0. assume that Alice and Bob have pk and shares of sk
1. they exchange encodings of their inputs x and y under pk
2. they perform local computations to obtain shares of output $f(x,y)$

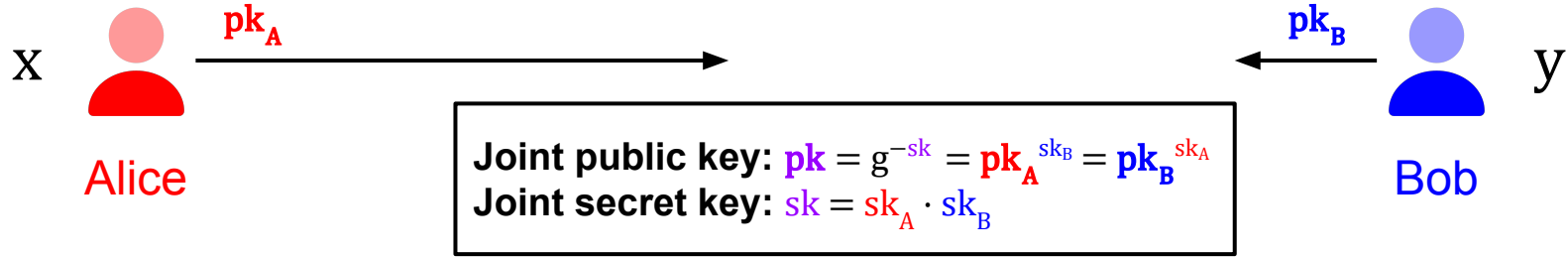
now we will see how to remove the assumption in 1 by having Alice and Bob

- use Diffie-Hellman key exchange to agree on a joint pk
- synchronize their input encodings under the joint pk
- (shares of joint sk are easy to generate with existing tools)

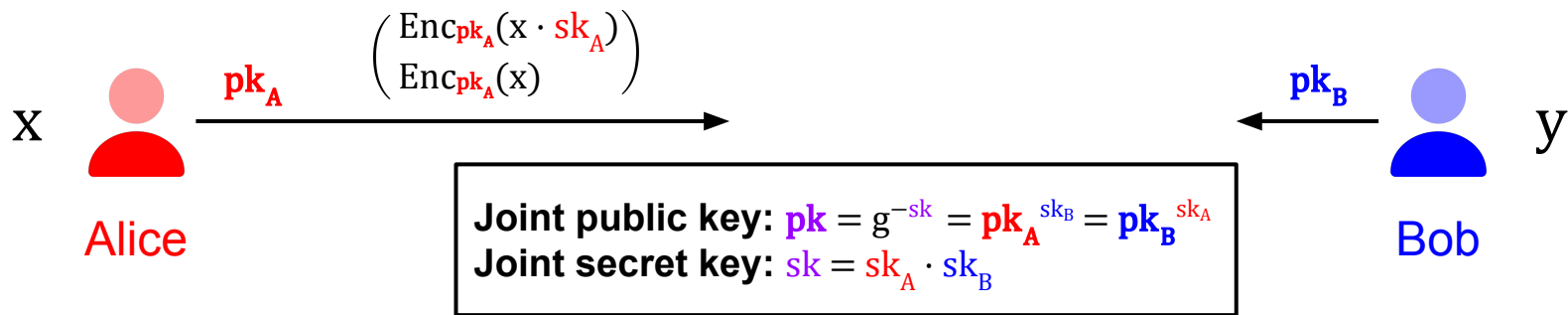


the rest of
my part of
the talk

Alice and Bob agree on joint key



Synchronizing Alice's input encoding

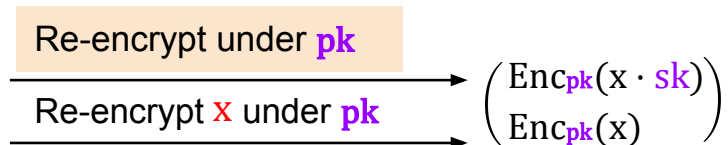
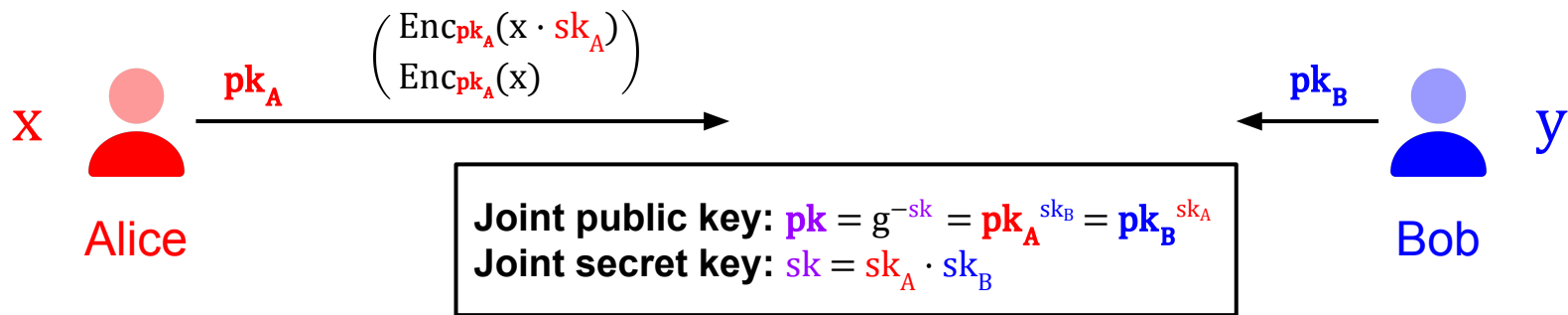


① both Alice and Bob need to compute
 ② Alice's synchronized input encoding:

$$\begin{pmatrix} \text{Enc}_{pk}(x \cdot sk) \\ \text{Enc}_{pk}(x) \end{pmatrix}$$

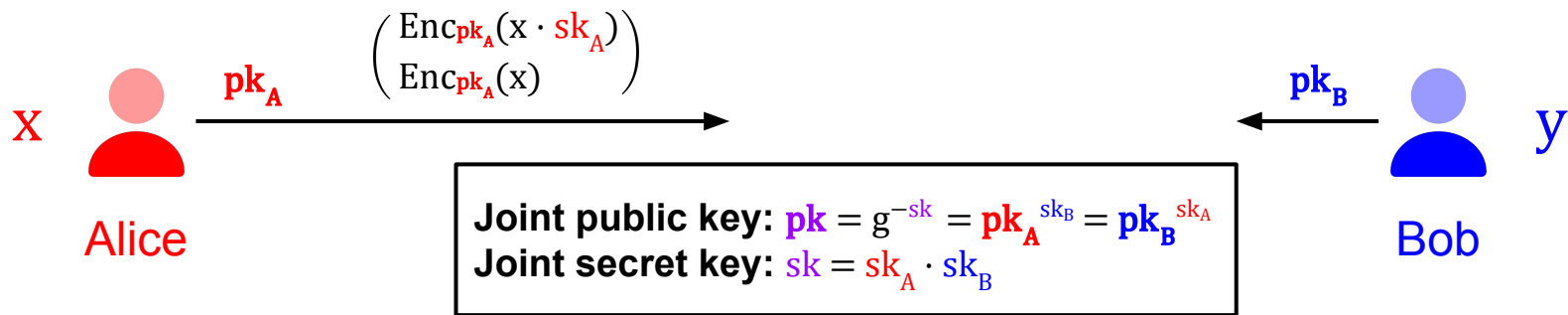
(Bob's encoding is
synchronized symmetrically)

①: Alice syncs her own share



Flipped encryption:
Can compute **without** knowing secret key.

②: Bob syncs Alice's share



Problem: junk term sk_B

$$\begin{array}{c}
 \left(\begin{array}{c} \text{Enc}_{\text{pk}_A}(x \cdot \text{sk}_A) \\ \text{Enc}_{\text{pk}_A}(x) \end{array} \right) \xrightarrow{\text{Multiply message/key by } \text{sk}_B} \left(\begin{array}{c} \text{Enc}_{\text{pk}}(x \cdot \text{sk}) \\ \text{Enc}_{\text{pk}}(x \cdot \text{sk}_B) \end{array} \right) \\
 \xrightarrow{\text{Multiply message/key by } \text{sk}_B}
 \end{array}$$

plaintext space = $[N]$
 solve by $\text{sk}_B = 1 \bmod N!$
 but...

Issue with circular encryptions

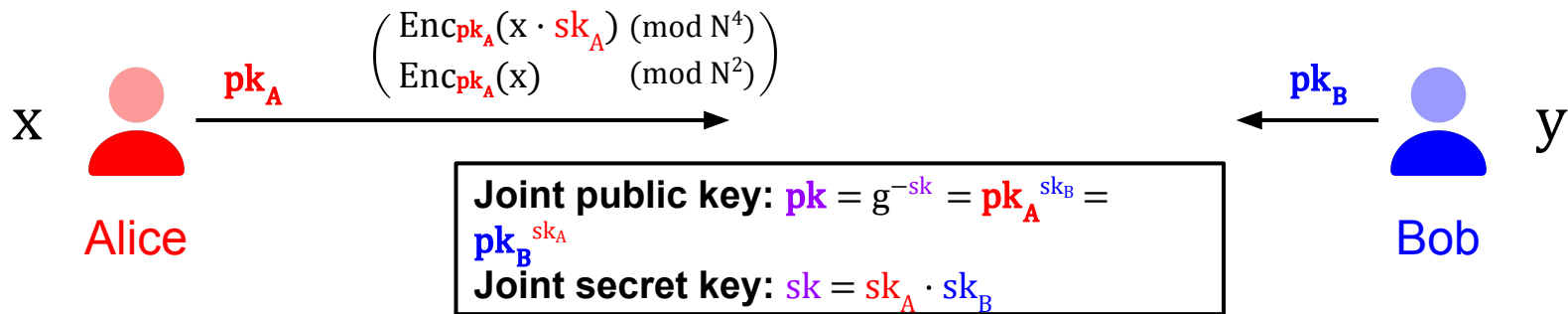
problem: if sk is always $1 \bmod N$, then we can no longer encode $x \cdot sk$ in a plaintext space of $[N]$

- recall that plaintexts are encoded in the exponent of $(1+N)$, which has order N

solution: compute circular encryptions $\bmod N^4$ instead of $\bmod N^2$! (i.e., generalized Damgard-Jurik encryptions)

in $Z_{N^4}^*$, the element $(1+N)$ has order N^3 , so we have a plaintext space large enough to encode $x \cdot sk$

Alice's input encoding



both Alice and Bob need to compute
Alice's synchronized input encoding:

$$\begin{pmatrix} \text{Enc}_{pk}(x \cdot sk) \pmod{N^4} \\ \text{Enc}_{pk}(x) \pmod{N^2} \end{pmatrix}$$

Why is sampling the secret key this way secure?

- instead of sampling $sk \leftarrow \$ [N]$, we now sample $sk' \leftarrow \$ \{0, \dots, N-1\}$ and set $sk = sk' \cdot N + 1$ so that $sk = 1 \pmod N$
- note that g has order $\phi(N)/4$, which is coprime to N
- so the distribution over public keys $pk = g^{-sk}$ remains statistically close to the old distribution over public keys

Aside: Short Exponent Assumption

essentially says: sampling much shorter sk is still secure

- for this construction, no need to make this assumption to prove security
 - construction for class groups does require making this assumption
- but it allows sampling *much smaller keys* in practice
 - no longer have statistical closeness to original distribution of Pailler ElGamal public keys

Roadmap

1. Summary of our contributions
 - a. Motivating example: two-party succinct secure computation
 - b. Define multi-key homomorphic secret sharing (MKHSS)
 - c. Application: non-interactive conditional key exchange
2. Background on HSS from DCR
3. Constructing MKHSS from the DCR assumption

next: Kevin will talk about optimized implementations of MKHSS/key exchange

Concretely-Efficient Multi-Key Homomorphic Secret Sharing and Applications

Kaiwen (Kevin) He, Sacha Servan-Schreiber, Geoffroy Couteau, Sridhar Devadas



To appear at *IEEE S&P* 2026

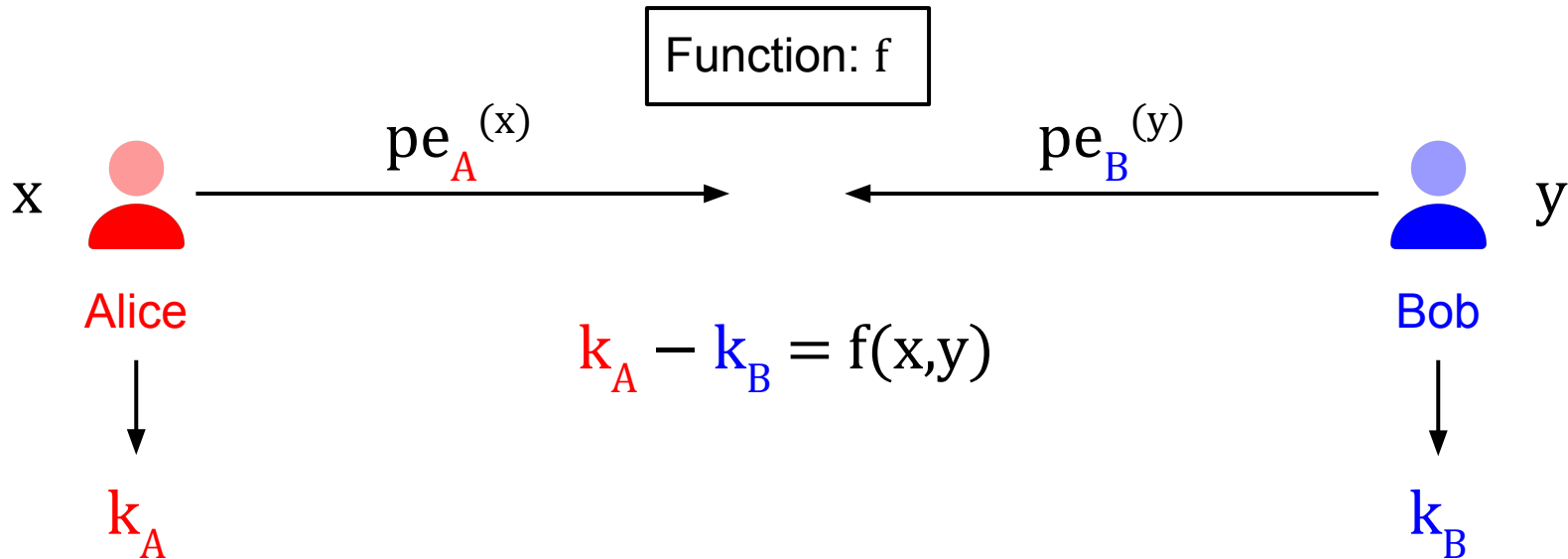
Roadmap

1. Overview of our work
2. MKHSS optimizations
3. Non-interactive conditional key exchange optimizations
4. Useful instantiations of key exchange
 - a. Fuzzy password-authenticated key exchange
 - b. Geolocation-based key exchange
5. Performance evaluation
6. Future works and conclusion

Roadmap

1. Overview of our work
2. MKHSS optimizations
3. Non-interactive conditional key exchange optimizations
4. Useful instantiations of key exchange
 - a. Fuzzy password-authenticated key exchange
 - b. Geolocation-based key exchange
5. Performance evaluation
6. Future works and conclusion

Recall: Multi-Key HSS Syntax [CDHJS'25]



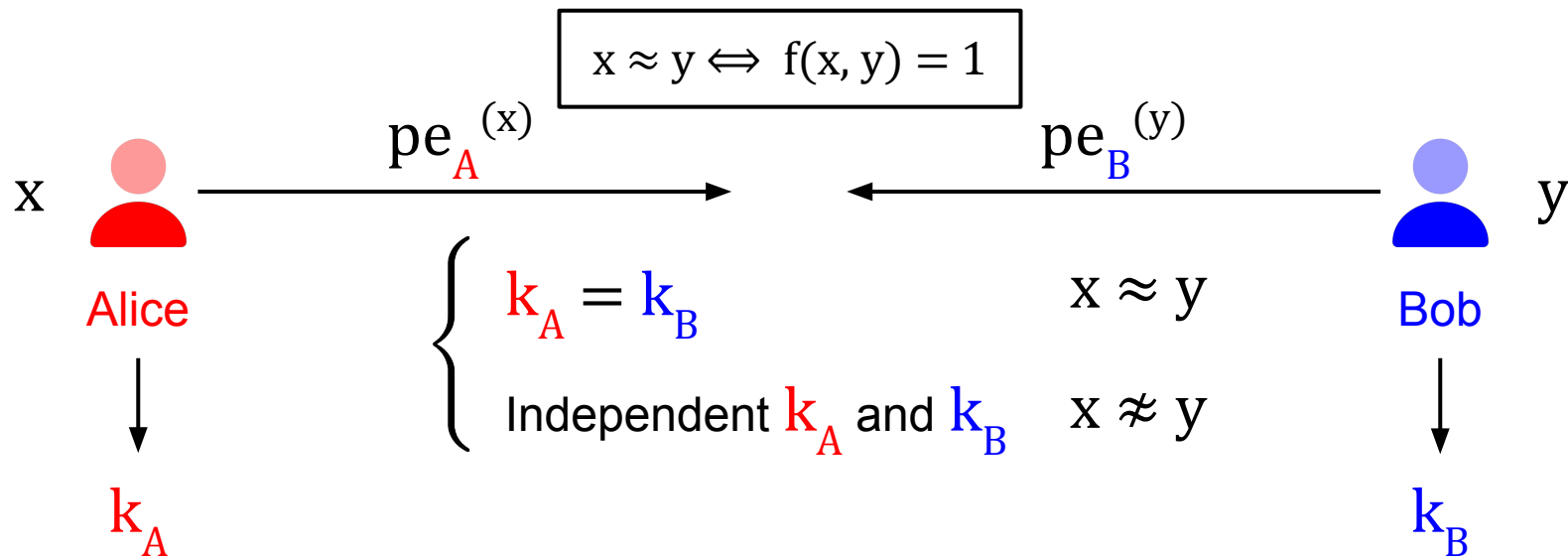
$$pe_A^{(x)}, st_A \leftarrow \text{Encode}(x)$$

$$k_A \leftarrow \text{Eval}_A(f, pe_B^{(y)}, st_A)$$

$$\text{Encode}(y) \rightarrow pe_B^{(y)}, st_B$$

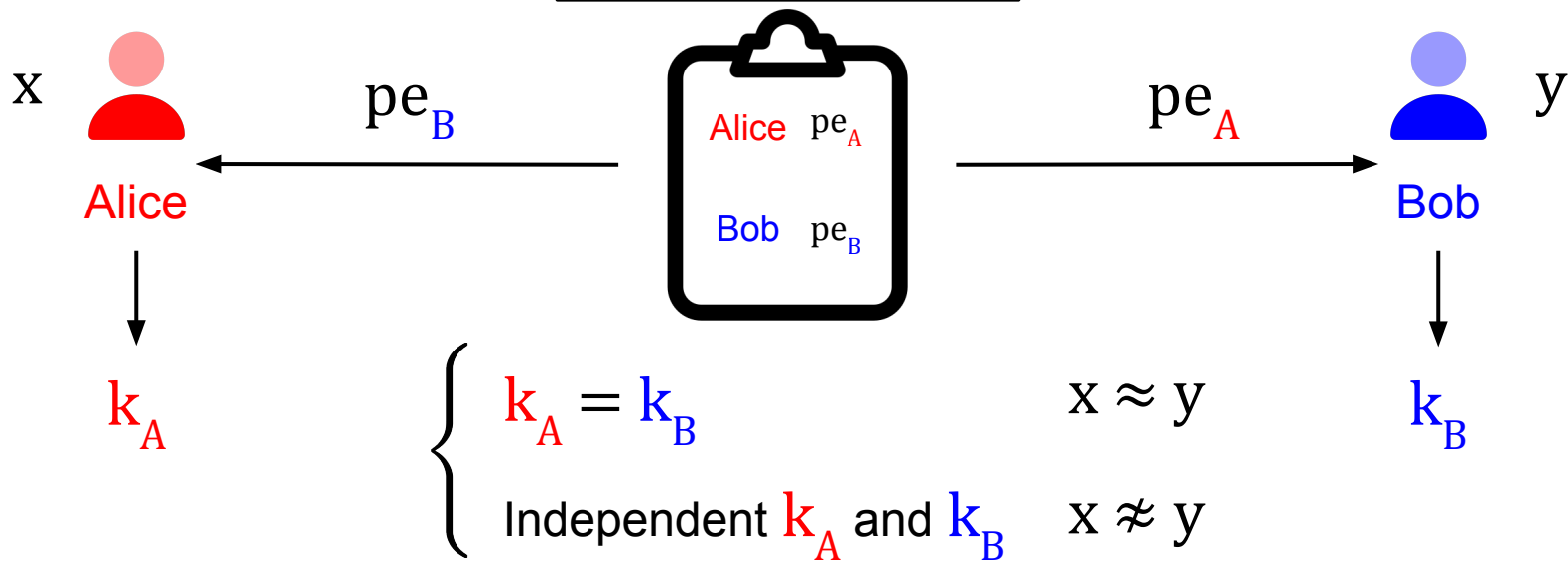
$$\text{Eval}_B(f, pe_A^{(x)}, st_B) \rightarrow k_B$$

Application: Non-Interactive Conditional Key Exchange [CDHJS'25]

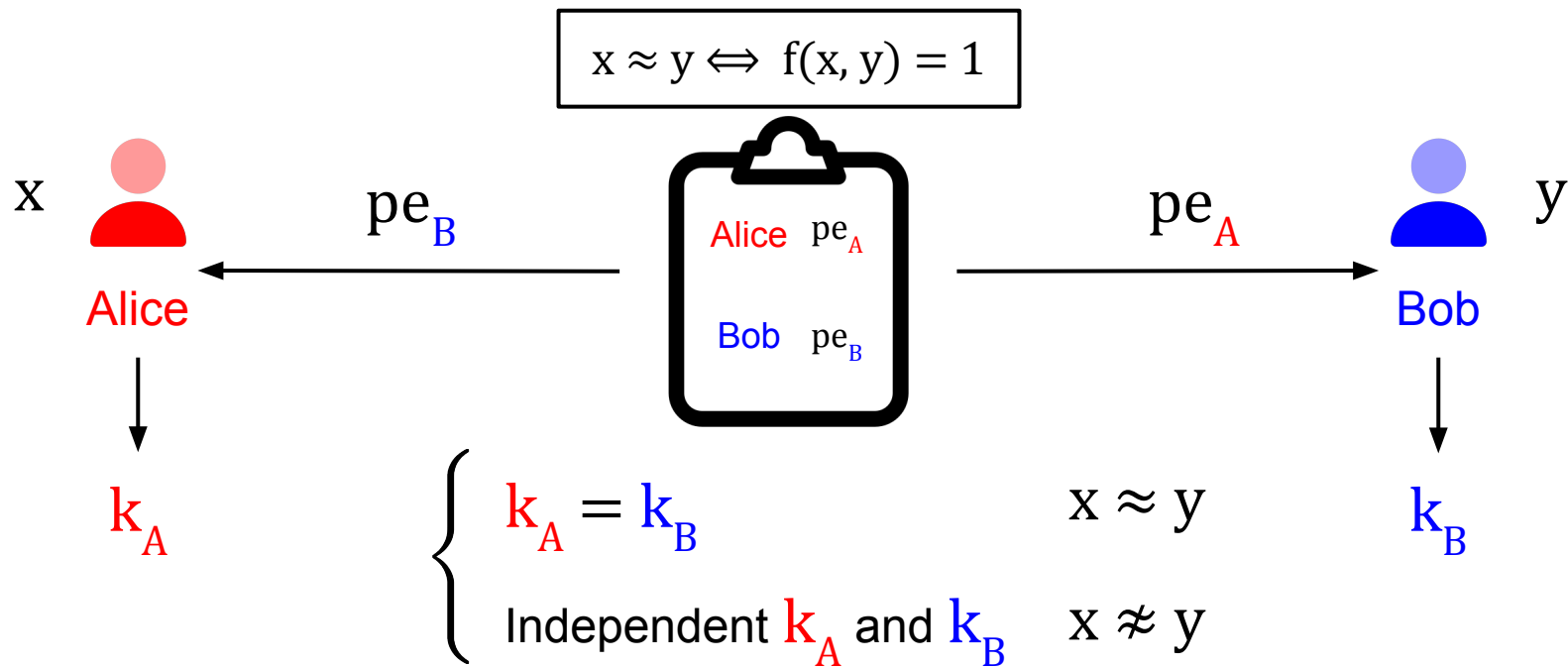


Application: Non-Interactive Conditional Key Exchange [CDHJS'25]

$$x \approx y \Leftrightarrow f(x, y) = 1$$



Application: Non-Interactive Conditional Key Exchange [CDHJS'25]

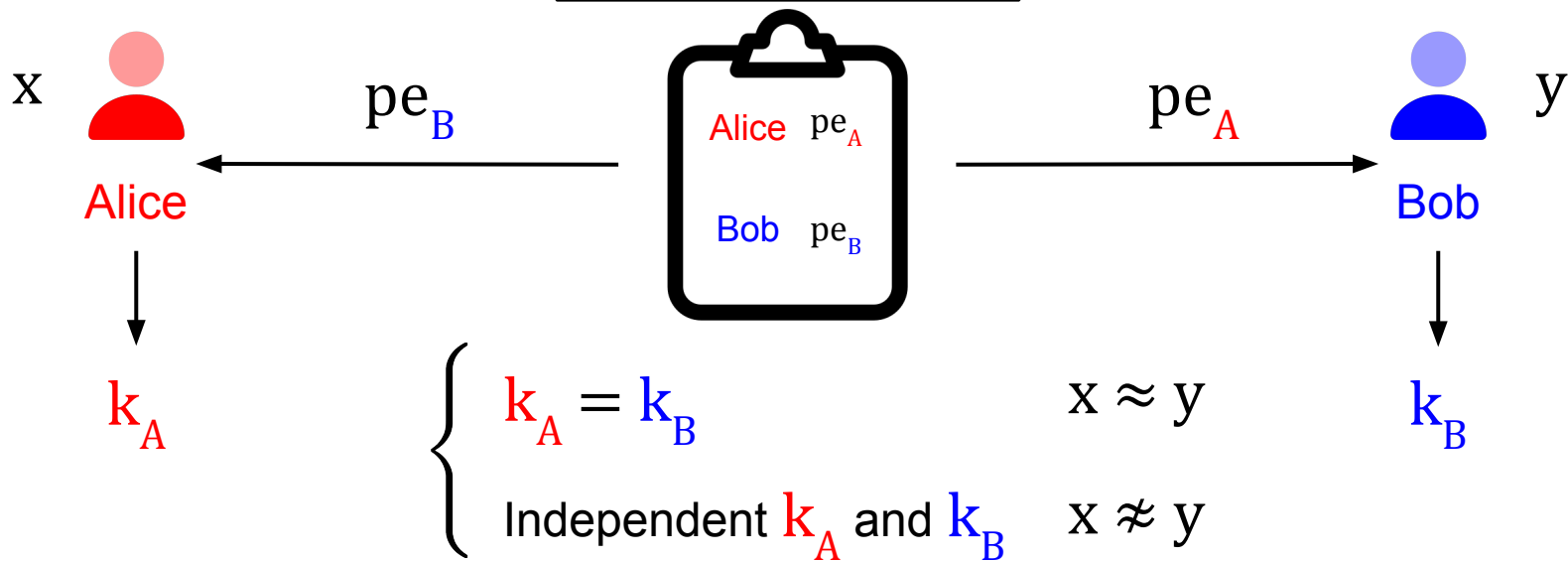


★ A natural generalization of Diffie-Hellman-style key exchange [DH'76, FHKP'13]

Application: Non-Interactive Conditional Key Exchange [CDHJS'25]

$$x \approx y \Leftrightarrow \mathbf{f}(x, y) = 1$$

Concrete instantiations of the predicate $\mathbf{f}$?



★ A natural generalization of Diffie-Hellman-style key exchange [DH'76, FHKP'13]

Concrete Instantiation: **f**PAKE [DHPRY'18]

$$x \approx y \Leftrightarrow \text{EditDistance}(x, y) \leq T$$

correct

horse
battery
staple

= x

fPAKE: **fuzzy** password-authenticated key exchange

y =

corrupt

hose
buttery
stable

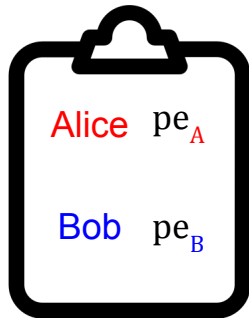
pe_B



Alice



k_A



pe_A



Bob



k_B

$$\left\{ \begin{array}{ll} k_A = k_B & x \approx y \\ \text{Independent } k_A \text{ and } k_B & x \not\approx y \end{array} \right.$$

Prior work requires 5+ rounds of interaction

Concrete Instantiation: **f**PAKE [DHPRY'18]

$$x \approx y \Leftrightarrow \text{EditDistance}(x, y) \leq T$$

correct
horse
battery
staple = x

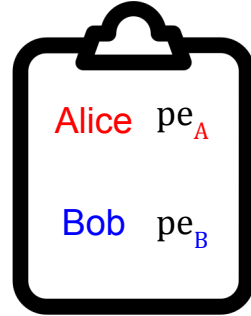
fPAKE: **fuzzy** password-authenticated key exchange

y = corrupt
hose
buttery
stable

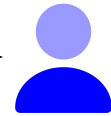


Alice

pe_B



pe_A



Bob



k_A

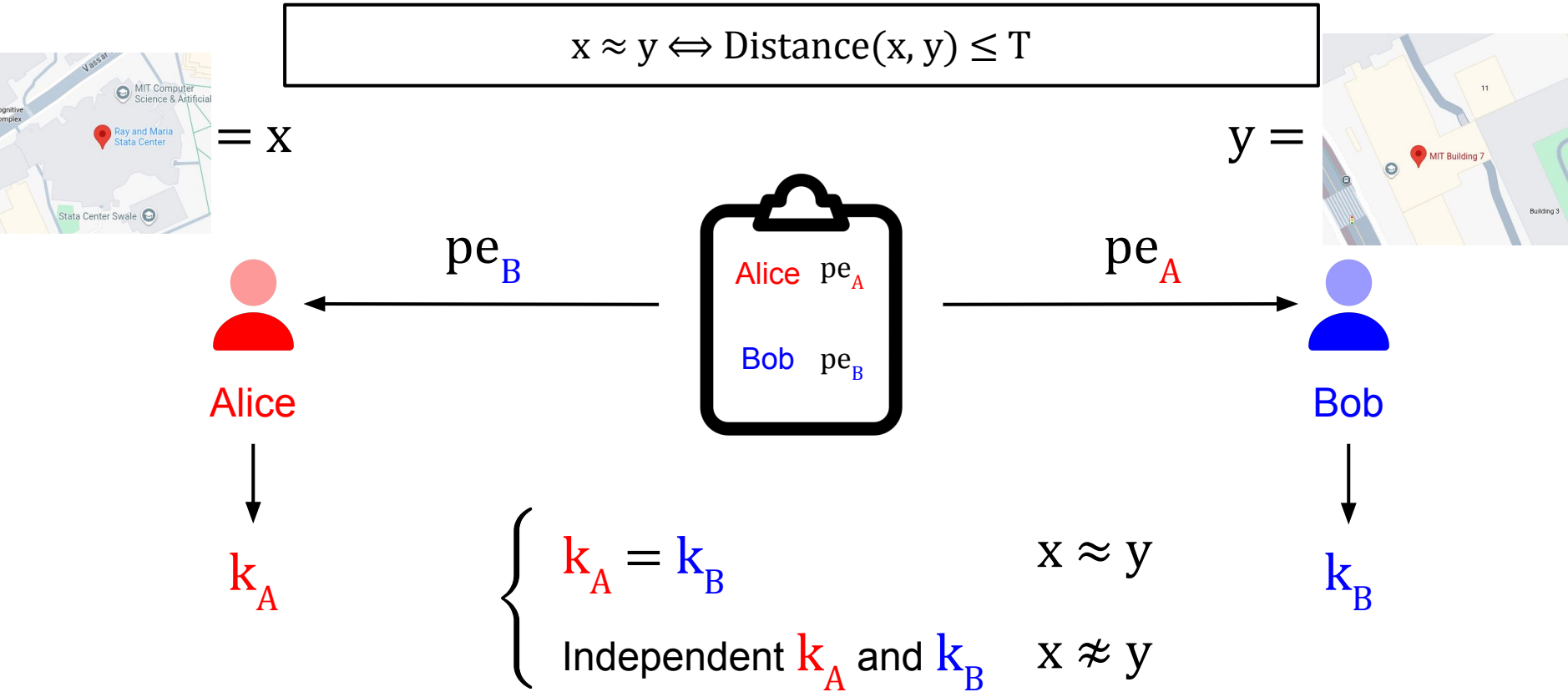


k_B

Multi-Key HSS gives a non-interactive solution

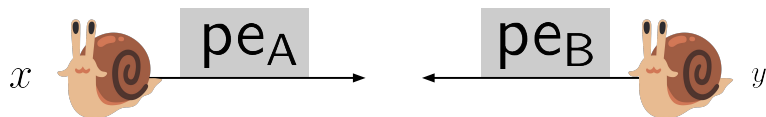
$$\left\{ \begin{array}{ll} k_A = k_B & x \approx y \\ \text{Independent } k_A \text{ and } k_B & x \not\approx y \end{array} \right.$$

Concrete Instantiation: Geolocation-Based Key Exchange



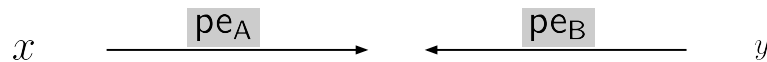
Our Contributions

Prior work [CDHJS'25]



✗ Theoretical feasibility result, no code

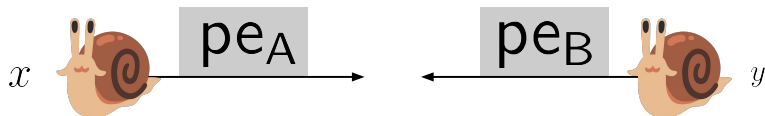
Our work



✓ Open-source implementation

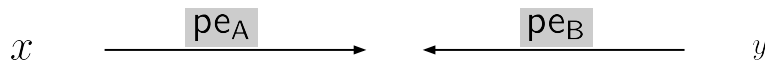
Our Contributions

Prior work [CDHJS'25]



- ✗ Theoretical feasibility result, no code
- ✗ A multiplication takes **224.6 ms** (if implemented)

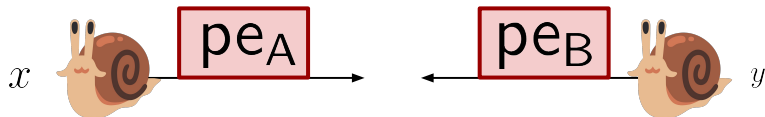
Our work



- ✓ Open-source implementation
- ✓ A multiplication takes **5.0 ms** (**45×** speedup)

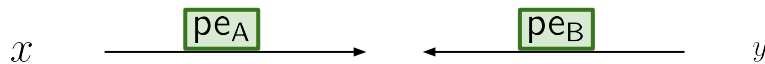
Our Contributions

Prior work [CDHJS'25]



- ✗ Theoretical feasibility result, no code
- ✗ A multiplication takes **224.6 ms** (if implemented)
- ✗ Large communication overhead

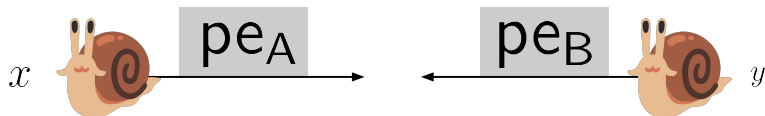
Our work



- ✓ Open-source implementation
- ✓ A multiplication takes **5.0 ms** (**45×** speedup)
- ✓ **3×** reduction in communication for all apps

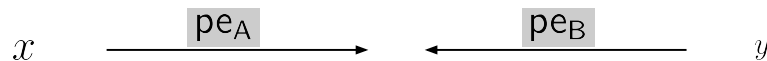
Our Contributions

Prior work [CDHJS'25]



- ✗ Theoretical feasibility result, no code
- ✗ A multiplication takes **224.6 ms** (if implemented)
- ✗ Large communication overhead
- ✗ Did not develop concrete applications
 - Mentioned fPAKE in passing without giving a concrete instantiation

Our work



- ✓ Open-source implementation
- ✓ A multiplication takes **5.0 ms** (**45×** speedup)
- ✓ **3×** reduction in communication for all apps
- ✓ Identifies two useful applications of MKHSS:
 - fPAKE
 - Geolocation-based key exchange

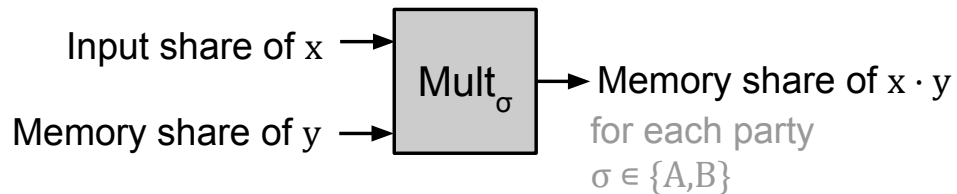
In addition, each app runs in a few seconds.

Roadmap

1. Overview of our work
2. **MKHSS optimizations**
3. Non-interactive conditional key exchange optimizations
4. Useful instantiations of key exchange
 - a. Fuzzy PAKE
 - b. Geolocation-based key exchange
5. Performance evaluation
6. Future works and conclusion

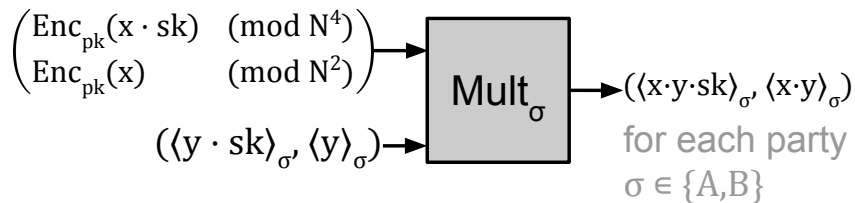
Bottleneck of (MK)HSS: RMS Multiplication

RMS Multiplication



Bottleneck of (MK)HSS: RMS Multiplication

Prior work [CDHJS'25]



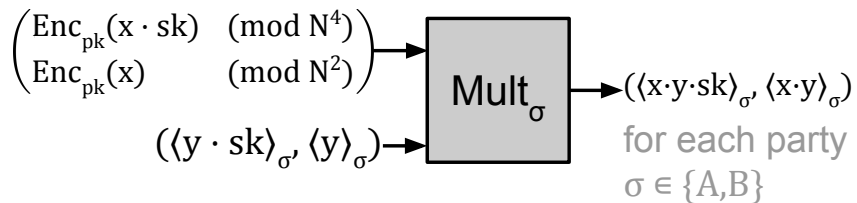
Notation

$$x = \langle x \rangle_A - \langle x \rangle_B$$

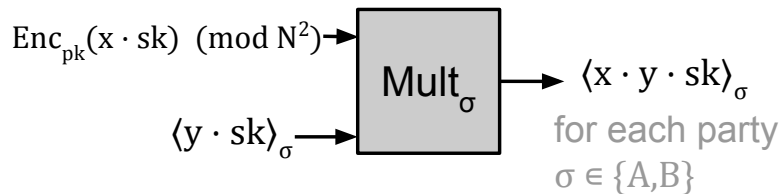
$$\text{Enc}_{\text{pk}}(x) = (g^r, \text{pk}^r \cdot (1+N)^x) \text{ [BCP'03, DJ'03]}$$

Overview Of Our Optimizations

Prior work [CDHJS'25]



Our work



Notation

$$x = \langle x \rangle_A - \langle x \rangle_B$$

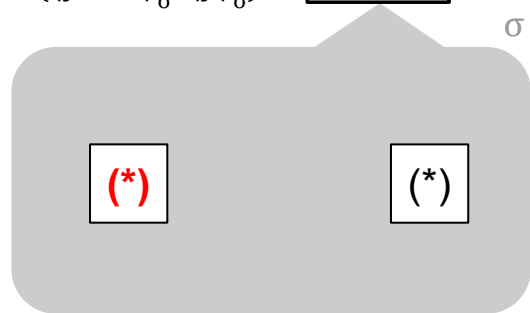
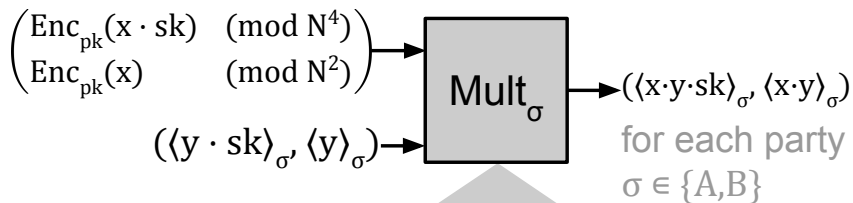
$$\text{Enc}_{\text{pk}}(x) = (g^r, \text{pk}^r \cdot (1+N)^x) \text{ [BCP'03, DJ'03]}$$

Overview Of Our Optimizations

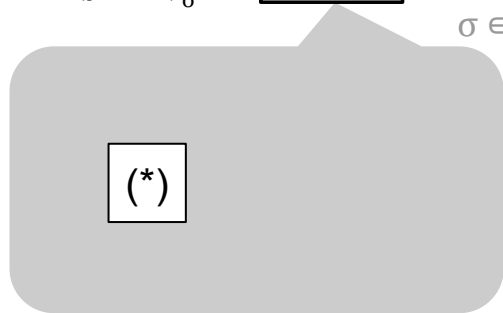
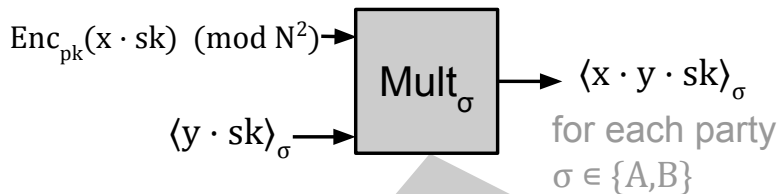
Notation

$$x = \langle x \rangle_A - \langle x \rangle_B$$

Prior work [CDHJS'25]



Our work



$(*)$: two exponentiations mod N^4

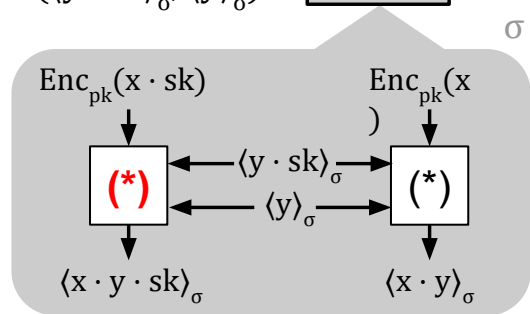
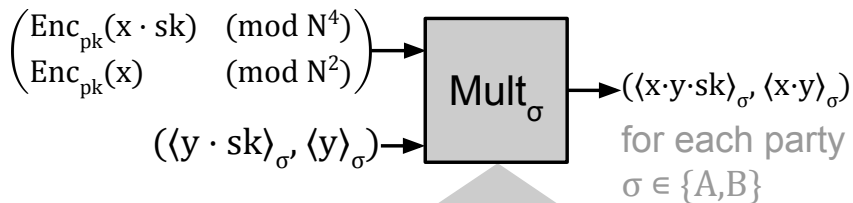
$(*)$: two exponentiations mod N^2

Overview Of Our Optimizations

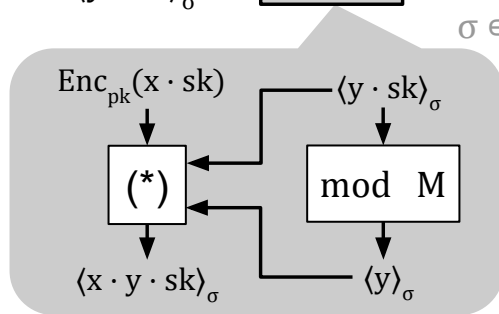
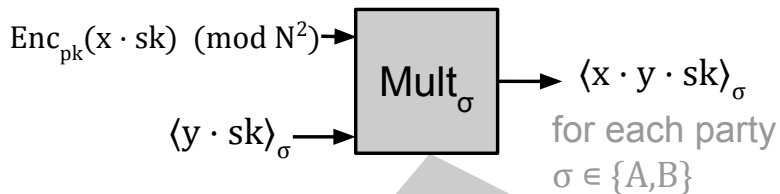
Notation

$$x = \langle x \rangle_A - \langle x \rangle_B$$

Prior work [CDHJS'25]



Our work



A key procedure used by much of the HSS literature [BGI'16]

$(*)$: two exponentiations mod N^4

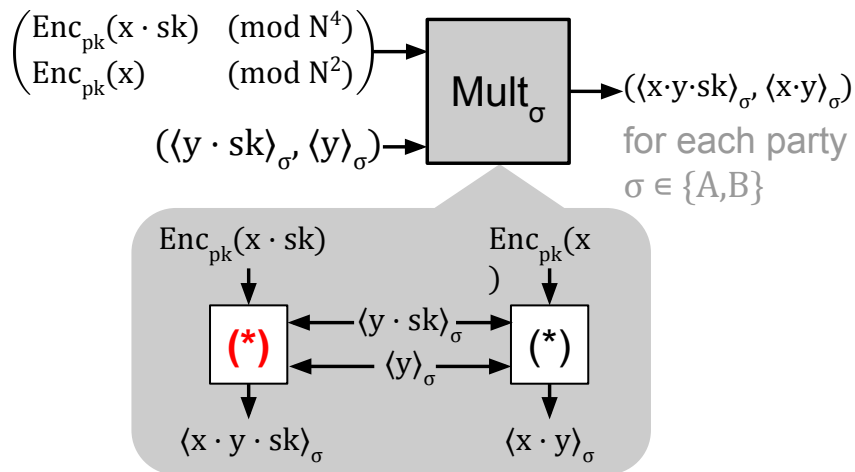
$(*)$: two exponentiations mod N^2

Key Procedure of Our Work

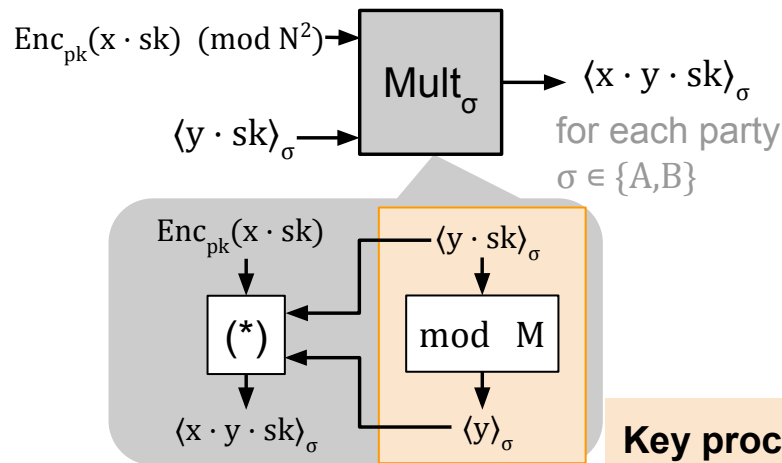
Notation

$$x = \langle x \rangle_A - \langle x \rangle_B$$

Prior work [CDHJS'25]



Our work



Key procedure of our work

$(*)$: two exponentiations mod N^4

$(*)$: two exponentiations mod N^2

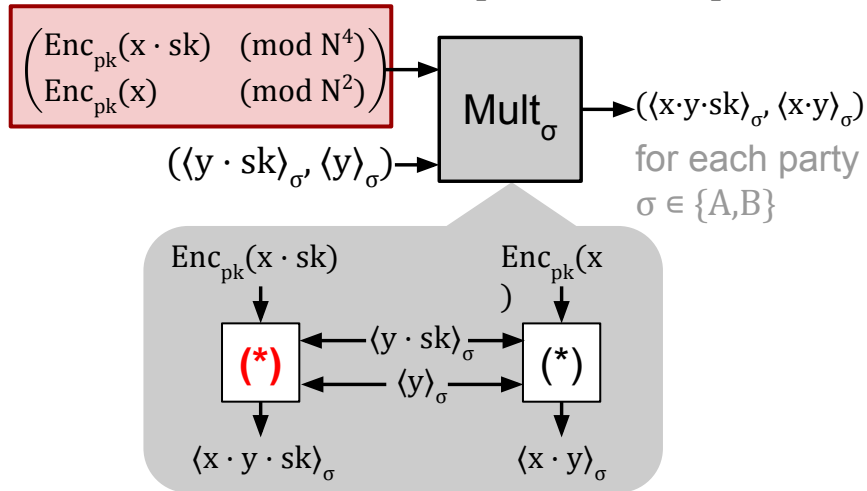
Simplifying Input Shares

Notation

$$x = \langle x \rangle_A - \langle x \rangle_B$$

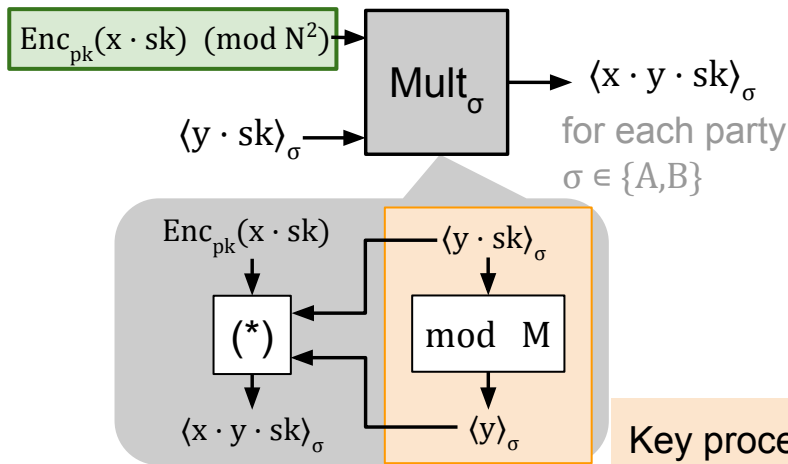
Input Share

Prior work [CDHJS'25]



Input Share

Our work



Key procedure of our work

Crucially simplifies **input share** structure.

$(*)$: two exponentiations mod N^4

$(*)$: two exponentiations mod N^2

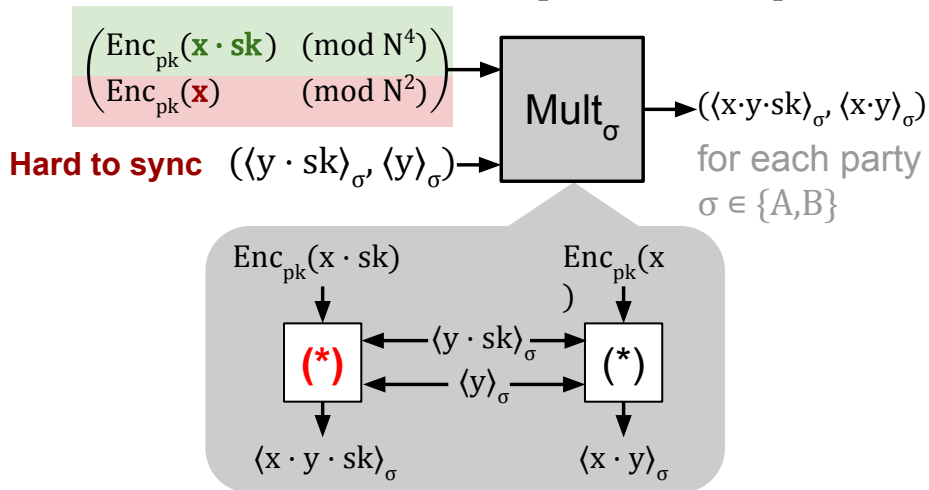
Making Share Synchronization Easier

Notation

$$x = \langle x \rangle_A - \langle x \rangle_B$$

Easy to sync

Prior work [CDHJS'25]

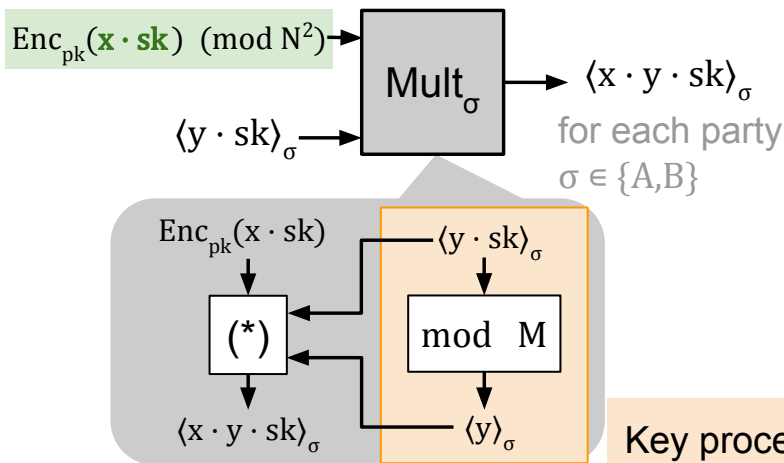


Share synchronization [CDHJS'25] is a key step to realize multi-key HSS.

(*) : two exponentiations mod N^4

Easy to sync

Our work



Key procedure of our work

Crucially simplifies **input share** structure.

(*) : two exponentiations mod N^2

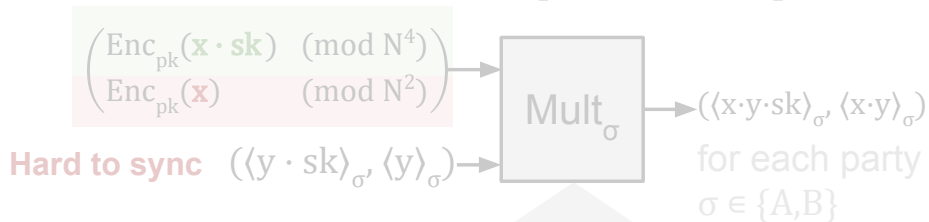
Making Share Synchronization Easier

Notation

$$\mathbf{x} = \langle \mathbf{x} \rangle_A - \langle \mathbf{x} \rangle_B$$

Easy to sync

Prior work [CDHJS'25]

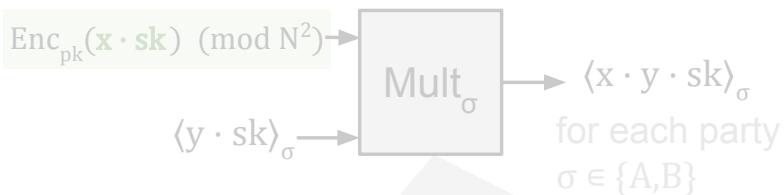


Share synchronization [CDHJS'25] is a key step to realize multi-key HSS.

$(*)$: two exponentiations mod N^4

Easy to sync

Our work



Key procedure of our work

Crucially simplifies input share structure.

$(*)$: two exponentiations mod N^2

Review: Share Synchronization [CDHJS'25]

Input share before: $\begin{pmatrix} \text{Enc}_{\text{pk}}(x \cdot \text{sk}) & (\text{mod } N^4) \\ \text{Enc}_{\text{pk}}(x) & (\text{mod } N^2) \end{pmatrix}$

Cheatsheet

Secret keys of Alice and Bob: sk_A, sk_B

Joint secret key: $\text{sk} = \text{sk}_A \cdot \text{sk}_B$

Public key of party σ : $\text{pk}_\sigma \equiv g^{-\text{sk}_\sigma} \pmod{N^{w+1}}$

Joint public key: $\text{pk} \equiv g^{-\text{sk}} \equiv \text{pk}_A^{\text{sk}_B} \equiv \text{pk}_B^{\text{sk}_A} \pmod{N^{w+1}}$

Review: Share Synchronization [CDHJS'25]

Input share before: $\begin{pmatrix} \text{Enc}_{\text{pk}}(x \cdot \text{sk}) & (\text{mod } N^4) \\ \text{Enc}_{\text{pk}}(x) & (\text{mod } N^2) \end{pmatrix}$

Easy to synchronize $\text{Enc}_{\text{pk}}(x \cdot \text{sk})$:

$$\text{Enc}_{\text{pk}}(x \cdot \text{sk}) = \text{Mul}(\text{Enc}_{\text{pk}_A}(x \cdot \text{sk}_A), \text{sk}_B)$$

Cheatsheet

Secret keys of Alice and Bob: sk_A, sk_B

Joint secret key: $\text{sk} = \text{sk}_A \cdot \text{sk}_B$

Public key of party σ : $\text{pk}_\sigma \equiv g^{-\text{sk}_\sigma} \pmod{N^{w+1}}$

Joint public key: $\text{pk} \equiv g^{-\text{sk}} \equiv \text{pk}_A^{\text{sk}_B} \equiv \text{pk}_B^{\text{sk}_A} \pmod{N^{w+1}}$

$\text{Mul}(\text{ct}, u)$ multiplies both key and message by u .

Review: Share Synchronization [CDHJS'25]

Input share before: $\begin{pmatrix} \text{Enc}_{\text{pk}}(x \cdot \text{sk}) & (\text{mod } N^4) \\ \text{Enc}_{\text{pk}}(x) & (\text{mod } N^2) \end{pmatrix}$

Easy to synchronize $\text{Enc}_{\text{pk}}(x \cdot \text{sk})$:

$$\text{Enc}_{\text{pk}}(x \cdot \text{sk}) = \text{Mul}(\text{Enc}_{\text{pk}_A}(x \cdot \text{sk}_A), \text{sk}_B)$$

Hard to synchronize $\text{Enc}_{\text{pk}}(x)$:

$$\text{Enc}_{\text{pk}}(x) = \text{Enc}_{\text{pk}}(x \cdot \text{sk}_B) = \text{Mul}(\text{Enc}_{\text{pk}_A}(x), \text{sk}_B)$$

Requires $\text{sk}_B = \text{sk}_B' \cdot N + 1$ (Likewise for sk_A)

Cheatsheet

Secret keys of Alice and Bob: sk_A, sk_B

Joint secret key: $\text{sk} = \text{sk}_A \cdot \text{sk}_B$

Public key of party σ : $\text{pk}_\sigma \equiv g^{-\text{sk}_\sigma} \pmod{N^{w+1}}$

Joint public key: $\text{pk} \equiv g^{-\text{sk}} \equiv \text{pk}_A^{\text{sk}_B} \equiv \text{pk}_B^{\text{sk}_A} \pmod{N^{w+1}}$

$\text{Mul}(\text{ct}, u)$ multiplies both key and message by u .

Review: Share Synchronization [CDHJS'25]

Input share before: $\begin{pmatrix} \text{Enc}_{\text{pk}}(x \cdot \text{sk}) & (\text{mod } N^4) \\ \text{Enc}_{\text{pk}}(x) & (\text{mod } N^2) \end{pmatrix}$

Easy to synchronize $\text{Enc}_{\text{pk}}(x \cdot \text{sk})$:

$$\text{Enc}_{\text{pk}}(x \cdot \text{sk}) = \text{Mul}(\text{Enc}_{\text{pk}_A}(x \cdot \text{sk}_A), \text{sk}_B)$$

Hard to synchronize $\text{Enc}_{\text{pk}}(x)$:

$$\text{Enc}_{\text{pk}}(x) = \text{Enc}_{\text{pk}}(x \cdot \text{sk}_B) = \text{Mul}(\text{Enc}_{\text{pk}_A}(x), \text{sk}_B)$$

Requires $\text{sk}_B = \text{sk}_B' \cdot N + 1$ (Likewise for sk_A)

Problem: large joint secret key:

$$|x \cdot \text{sk}| \approx N^2 \cdot |x| \cdot |\text{sk}_A'| \cdot |\text{sk}_B'| \gg N^2$$

Cheatsheet

Secret keys of Alice and Bob: sk_A, sk_B

Joint secret key: $\text{sk} = \text{sk}_A \cdot \text{sk}_B$

Public key of party σ : $\text{pk}_\sigma \equiv g^{-\text{sk}_\sigma} \pmod{N^{w+1}}$

Joint public key: $\text{pk} \equiv g^{-\text{sk}} \equiv \text{pk}_A^{\text{sk}_B} \equiv \text{pk}_B^{\text{sk}_A} \pmod{N^{w+1}}$

$\text{Mul}(\text{ct}, u)$ multiplies both key and message by u .

Review: Share Synchronization [CDHJS'25]

Input share before: $\begin{pmatrix} \text{Enc}_{\text{pk}}(x \cdot \text{sk}) & (\text{mod } N^4) \\ \text{Enc}_{\text{pk}}(x) & (\text{mod } N^2) \end{pmatrix}$

Easy to synchronize $\text{Enc}_{\text{pk}}(x \cdot \text{sk})$:

$$\text{Enc}_{\text{pk}}(x \cdot \text{sk}) = \text{Mul}(\text{Enc}_{\text{pk}_A}(x \cdot \text{sk}_A), \text{sk}_B)$$

Hard to synchronize $\text{Enc}_{\text{pk}}(x)$:

$$\text{Enc}_{\text{pk}}(x) = \text{Enc}_{\text{pk}}(x \cdot \text{sk}_B) = \text{Mul}(\text{Enc}_{\text{pk}_A}(x), \text{sk}_B)$$

Requires $\text{sk}_B = \text{sk}_B' \cdot N + 1$ (Likewise for sk_A)

Problem: large joint secret key:

$$|x \cdot \text{sk}| \approx N^2 \cdot |x| \cdot |\text{sk}_A'| \cdot |\text{sk}_B'| \gg N^2$$

For $\text{Enc}_{\text{pk}}(x \cdot \text{sk})$ to decrypt correctly, need modulus N^4
even when we use short exponents: $|\text{sk}_\sigma'| \approx 2^{2\lambda}$.

Cheatsheet

Secret keys of Alice and Bob: sk_A, sk_B

Joint secret key: $\text{sk} = \text{sk}_A \cdot \text{sk}_B$

Public key of party σ : $\text{pk}_\sigma \equiv g^{-\text{sk}_\sigma} \pmod{N^{w+1}}$

Joint public key: $\text{pk} \equiv g^{-\text{sk}} \equiv \text{pk}_A^{\text{sk}_B} \equiv \text{pk}_B^{\text{sk}_A} \pmod{N^{w+1}}$

$\text{Mul}(\text{ct}, u)$ multiplies both key and message by u .

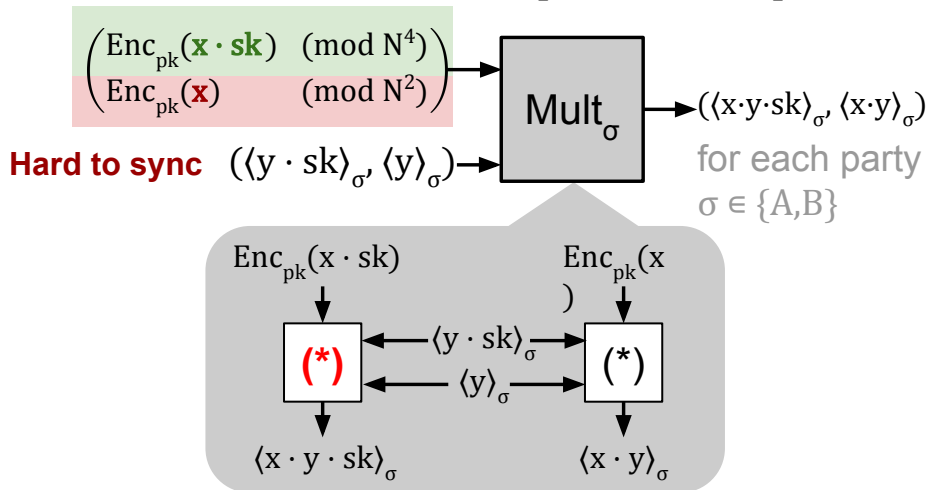
Making Share Synchronization Easier

Notation

$$x = \langle x \rangle_A - \langle x \rangle_B$$

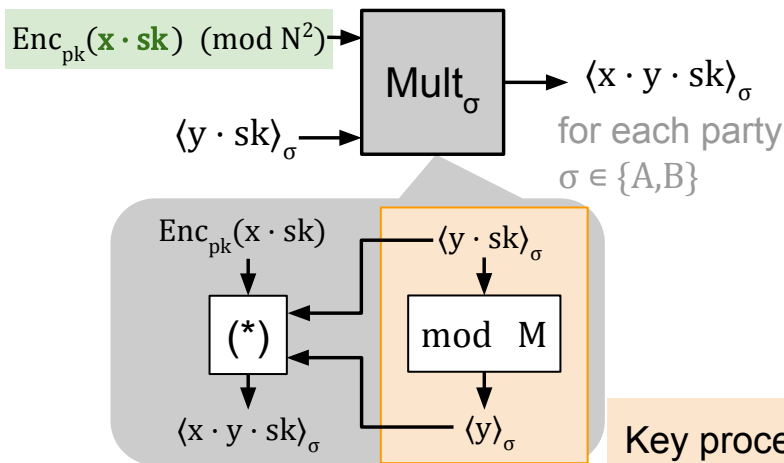
Easy to sync

Prior work [CDHJS'25]



Easy to sync

Our work



Key procedure of our work

✓ We do not need arithmetic mod N^4 .

(*) : two exponentiations mod N^4

(*) : two exponentiations mod N^2

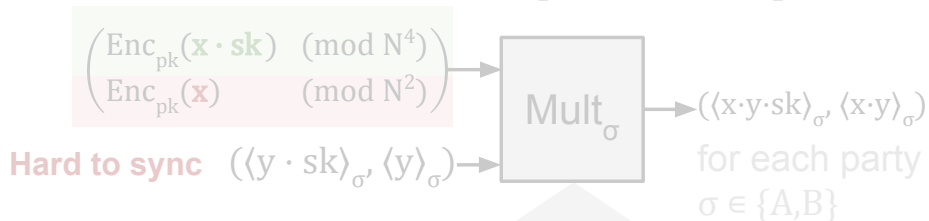
Making Share Synchronization Easier

Notation

$$\mathbf{x} = \langle \mathbf{x} \rangle_A - \langle \mathbf{x} \rangle_B$$

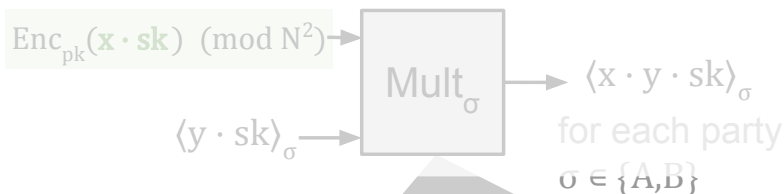
Easy to sync

Prior work [CDHJS'25]



Easy to sync

Our work



Key procedure of our work

✓ We do not need arithmetic mod N^4 .

(*) : two exponentiations mod N^4

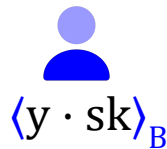
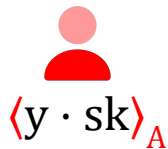
(*) : two exponentiations mod N^2

Simplifying Memory Shares (This Work)

Notation

$$x = \langle x \rangle_A - \langle x \rangle_B$$

Goal: derive $\langle y \rangle_\sigma$ locally from $\langle y \cdot sk \rangle_\sigma$



Simplifying Memory Shares (This Work)

Notation

$$x = \langle x \rangle_A - \langle x \rangle_B$$

Goal: derive $\langle y \rangle_\sigma$ locally from $\langle y \cdot sk \rangle_\sigma$


$$\langle y \cdot sk \rangle_A$$
$$= r + y \cdot sk$$

Precondition: Parties hold random shares

Achievable by applying a public random offset

$$r \leftarrow \$ \{0, \dots, N - 1\}$$


$$\langle y \cdot sk \rangle_B$$
$$= r$$

Simplifying Memory Shares (This Work)

Notation

$$x = \langle x \rangle_A - \langle x \rangle_B$$

Goal: derive $\langle y \rangle_\sigma$ locally from $\langle y \cdot sk \rangle_\sigma$


$$\begin{aligned} & \langle y \cdot sk \rangle_A \\ &= r + y \cdot sk \\ & \quad \downarrow \text{mod } M \\ &= \boxed{r'} + y \cdot sk \pmod{M} \end{aligned}$$

$$r \leftarrow \$ \{0, \dots, N - 1\}$$


$$\begin{aligned} & \langle y \cdot sk \rangle_B \\ &= r \\ & \quad \downarrow \text{mod } M \\ &= \boxed{r'} \end{aligned}$$

Define $r' := r \pmod{M}$

Simplifying Memory Shares (This Work)

Notation

$$x = \langle x \rangle_A - \langle x \rangle_B$$

Goal: derive $\langle y \rangle_\sigma$ locally from $\langle y \cdot sk \rangle_\sigma$


$$\begin{aligned} & \langle y \cdot sk \rangle_A \\ &= r + y \cdot sk \\ & \quad \downarrow \text{mod } M \\ &= \boxed{r'} + y \cdot sk \pmod{M} \end{aligned}$$

Pick $M \leq N \cdot 2^{-\lambda}$

$$r \leftarrow \{0, \dots, N - 1\}$$


$$\begin{aligned} & \langle y \cdot sk \rangle_B \\ &= r \\ & \quad \downarrow \text{mod } M \\ &= \boxed{r'} \end{aligned}$$

Observe: $r' \approx_s \{0, \dots, M - 1\}$

Define $r' := r \bmod M$

Simplifying Memory Shares (This Work)

Notation

$$x = \langle x \rangle_A - \langle x \rangle_B$$

Goal: derive $\langle y \rangle_\sigma$ locally from $\langle y \cdot sk \rangle_\sigma$


$$\begin{aligned} & \langle y \cdot sk \rangle_A \\ &= r + y \cdot sk \\ & \quad \downarrow \text{mod } M \\ &= r' + y \cdot sk \pmod{M} \end{aligned}$$

Pick $M \leq N \cdot 2^{-\lambda}$

$$r \leftarrow \{0, \dots, N - 1\}$$


$$\begin{aligned} & \langle y \cdot sk \rangle_B \\ &= r \\ & \quad \downarrow \text{mod } M \\ &= r' \end{aligned}$$

Observe: $r' \approx_s \{0, \dots, M - 1\}$

Define $r' := r \bmod M$

Simplifying Memory Shares (This Work)

Notation

$$x = \langle x \rangle_A - \langle x \rangle_B$$

Goal: derive $\langle y \rangle_\sigma$ locally from $\langle y \cdot sk \rangle_\sigma$


$$\begin{aligned} & \langle y \cdot sk \rangle_A \\ &= r + y \cdot sk \\ & \quad \downarrow \text{mod } M \\ &= \boxed{r'} + y \cdot \boxed{sk} \pmod{M} \end{aligned}$$

Pick $M \leq N \cdot 2^{-\lambda}$

$$r \leftarrow \$ \{0, \dots, N - 1\}$$

Idea: What if $sk \equiv 1 \pmod{M}$?

Observe: $r' \approx_s \{0, \dots, M - 1\}$

Define $r' := r \bmod M$


$$\begin{aligned} & \langle y \cdot sk \rangle_B \\ &= r \\ & \quad \downarrow \text{mod } M \\ &= \boxed{r'} \end{aligned}$$

Simplifying Memory Shares (This Work)

Notation

$$\mathbf{x} = \langle \mathbf{x} \rangle_A - \langle \mathbf{x} \rangle_B$$

Goal: derive $\langle y \rangle_\sigma$ locally from $\langle y \cdot \text{sk} \rangle_\sigma$


$$\begin{aligned} & \langle y \cdot \text{sk} \rangle_A \\ &= r + y \cdot \text{sk} \\ & \quad \downarrow \text{mod } M \\ &= r' + y \cdot \text{sk} \pmod{M} \end{aligned}$$

Pick $M \leq N \cdot 2^{-\lambda}$

$$r \leftarrow \$ \{0, \dots, N - 1\}$$

Sample sk_A, sk_B like in [CDHJS'25],
except with M instead of N :

$$\text{sk}'_\sigma \leftarrow \$ \{0, \dots, 2^{2\lambda} - 1\}$$

$$\text{sk}_\sigma = \text{sk}'_\sigma \cdot M + 1$$


$$\begin{aligned} & \langle y \cdot \text{sk} \rangle_B \\ &= r \\ & \quad \downarrow \text{mod } M \\ &= r' \end{aligned}$$

$$r' \approx_s \{0, \dots, M - 1\}$$

$$r' := r \bmod M$$

Simplifying Memory Shares (This Work)

Notation

$$\mathbf{x} = \langle \mathbf{x} \rangle_A - \langle \mathbf{x} \rangle_B$$

Goal: derive $\langle y \rangle_\sigma$ locally from $\langle y \cdot \text{sk} \rangle_\sigma$


$$\begin{aligned} & \langle y \cdot \text{sk} \rangle_A \\ &= r + y \cdot \text{sk} \\ & \quad \downarrow \text{mod } M \\ &= r' + y \cdot \cancel{\text{sk}} \pmod{M} \end{aligned}$$

Pick $M \leq N \cdot 2^{-\lambda}$

$$r \leftarrow \$ \{0, \dots, N - 1\}$$

Sample sk_A, sk_B like in [CDHJS'25],
except with M instead of N :

$$\text{sk}'_\sigma \leftarrow \$ \{0, \dots, 2^{2\lambda} - 1\}$$

$$\text{sk}_\sigma = \text{sk}'_\sigma \cdot M + 1$$

$$\Rightarrow \text{sk} := \text{sk}_A \cdot \text{sk}_B \equiv 1 \pmod{M}$$


$$\begin{aligned} & \langle y \cdot \text{sk} \rangle_B \\ &= r \\ & \quad \downarrow \text{mod } M \\ &= r' \end{aligned}$$

$$r' \approx_s \{0, \dots, M - 1\}$$

$$r' := r \bmod M$$

Simplifying Memory Shares (This Work)

Notation

$$x = \langle x \rangle_A - \langle x \rangle_B$$

Goal: derive $\langle y \rangle_\sigma$ locally from $\langle y \cdot sk \rangle_\sigma$


$$\begin{aligned} & \langle y \cdot sk \rangle_A \\ &= r + y \cdot sk \\ & \quad \downarrow \text{mod } M \\ &= r' + y \pmod{M} \end{aligned}$$

Pick $M \leq N \cdot 2^{-\lambda}$

$$r \leftarrow \{0, \dots, N - 1\}$$

Problem: subtractive shares must be over the integers


$$\begin{aligned} & \langle y \cdot sk \rangle_B \\ &= r \\ & \quad \downarrow \text{mod } M \\ &= r' \end{aligned}$$

$$r' \approx_s \{0, \dots, M - 1\}$$

$$r' := r \bmod M$$

Simplifying Memory Shares (This Work)

Notation

$$x = \langle x \rangle_A - \langle x \rangle_B$$

Goal: derive $\langle y \rangle_\sigma$ locally from $\langle y \cdot sk \rangle_\sigma$


$$\begin{aligned} & \langle y \cdot sk \rangle_A \\ &= r + y \cdot sk \\ & \quad \downarrow \text{mod } M \\ &= r' + y \pmod{M} \\ &= r' + y \text{ (over } \mathbb{Z}) = \langle y \rangle_A \end{aligned}$$

holds if $r' < M - y$

Pick $M \leq N \cdot 2^{-\lambda}$

$$r \leftarrow \{0, \dots, N - 1\}$$

$$r' \approx_s \{0, \dots, M - 1\}$$

$$r' := r \bmod M$$


$$\begin{aligned} & \langle y \cdot sk \rangle_B \\ &= r \\ & \quad \downarrow \text{mod } M \\ &= r' = \langle y \rangle_B \end{aligned}$$

Simplifying Memory Shares (This Work)

Notation

$$x = \langle x \rangle_A - \langle x \rangle_B$$

Goal: derive $\langle y \rangle_\sigma$ locally from $\langle y \cdot sk \rangle_\sigma$


$$\begin{aligned} & \langle y \cdot sk \rangle_A \\ &= r + y \cdot sk \\ & \quad \downarrow \text{mod } M \\ &= r' + y \pmod{M} \\ &= r' + y \text{ (over } \mathbb{Z}) = \langle y \rangle_A \end{aligned}$$

$$\text{Need } y \cdot 2^\lambda \leq M \leq N \cdot 2^{-\lambda}$$

$$r \leftarrow \$ \{0, \dots, N - 1\}$$

holds if $r' < M - y$, which is true with probability $\geq 1 - 2^{-\lambda}$

$$r' \approx_s \{0, \dots, M - 1\}$$

$$r' := r \bmod M$$


$$\begin{aligned} & \langle y \cdot sk \rangle_B \\ &= r \\ & \quad \downarrow \text{mod } M \\ &= r' = \langle y \rangle_B \end{aligned}$$

Simplifying Memory Shares (This Work)

Notation

$$x = \langle x \rangle_A - \langle x \rangle_B$$

Goal: derive $\langle y \rangle_\sigma$ locally from $\langle y \cdot sk \rangle_\sigma$


$$\begin{aligned} & \langle y \cdot sk \rangle_A \\ &= r + y \cdot sk \\ & \quad \downarrow \text{mod } M \\ &= r' + y \pmod{M} \\ &= r' + y \pmod{\mathbb{Z}} = \langle y \rangle_A \end{aligned}$$

Pick $M = \max(y) \cdot 2^\lambda$

Need $y \cdot 2^\lambda \leq M \leq N \cdot 2^{-\lambda}$

$r \leftarrow \{0, \dots, N - 1\}$

holds if $r' < M - y$, which is true with probability $\geq 1 - 2^{-\lambda}$

$r' \approx_s \{0, \dots, M - 1\}$

$r' := r \bmod M$


$$\begin{aligned} & \langle y \cdot sk \rangle_B \\ &= r \\ & \quad \downarrow \text{mod } M \\ &= r' = \langle y \rangle_B \end{aligned}$$

Simplifying Memory Shares (This Work)

Notation

$$\mathbf{x} = \langle \mathbf{x} \rangle_A - \langle \mathbf{x} \rangle_B$$

Goal: derive $\langle y \rangle_\sigma$ locally from $\langle y \cdot \text{sk} \rangle_\sigma$


$$\begin{aligned} & \langle y \cdot \text{sk} \rangle_A \\ &= r + y \cdot \text{sk} \\ & \quad \downarrow \text{mod } M \\ &= r' + y \pmod{M} \\ &= r' + y \text{ (over } \mathbb{Z}) = \langle y \rangle_A \end{aligned}$$

Pick $M = \mathbf{B} \cdot 2^\lambda$

Parameter:
 $\mathbf{B}$: magnitude bound on memory shares
Looking ahead, our key exchange
applications only need $\mathbf{B} = 1$


$$\begin{aligned} & \langle y \cdot \text{sk} \rangle_B \\ &= r \\ & \quad \downarrow \text{mod } M \\ &= r' = \langle y \rangle_B \end{aligned}$$

holds if $r' < M - y$, which is true with probability $\geq 1 - 2^{-\lambda}$

$$r' \approx_s \{0, \dots, M - 1\}$$

$$r' := r \bmod M$$

Simplifying Memory Shares (This Work)

Notation

$$\mathbf{x} = \langle \mathbf{x} \rangle_A - \langle \mathbf{x} \rangle_B$$

Goal: derive $\langle y \rangle_\sigma$ locally from $\langle y \cdot \mathbf{sk} \rangle_\sigma$


$$\begin{aligned} & \langle y \cdot \mathbf{sk} \rangle_A \\ &= r + y \cdot \mathbf{sk} \\ & \quad \downarrow \text{mod } M \\ &= r' + y \pmod{M} \\ &= r' + y \text{ (over } \mathbb{Z}) = \langle y \rangle_A \end{aligned}$$

Pick $M = \mathbf{B} \cdot 2^\lambda$

Secret key size analysis:

$$\begin{aligned} \mathbf{sk}'_\sigma &\leftarrow \$ \{0, \dots, 2^{2\lambda} - 1\} \\ \mathbf{sk}_\sigma &= \mathbf{sk}'_\sigma \cdot M + 1 \\ \Rightarrow \mathbf{sk} &:= \mathbf{sk}_A \cdot \mathbf{sk}_B \equiv 1 \pmod{M} \\ \mathbf{sk} &\leq \mathbf{B}^2 \cdot 2^{6\lambda} \\ \mathbf{B} = 1, \lambda = 128, N \approx 2^{3072} &\Rightarrow \mathbf{sk} \leq N \end{aligned}$$

holds if $r' < M - y$, which is true with probability $\geq 1 - 2^{-\lambda}$

$$r' \approx_s \{0, \dots, M - 1\}$$

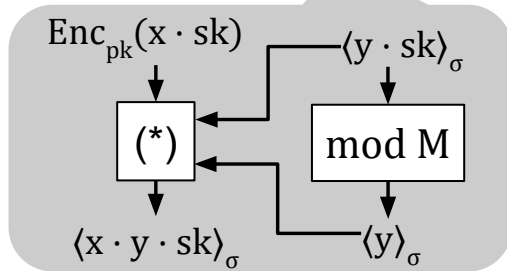
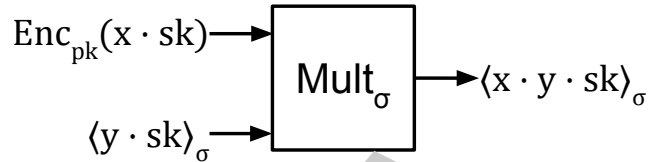
$$r' := r \bmod M$$


$$\begin{aligned} & \langle y \cdot \mathbf{sk} \rangle_B \\ &= r \\ & \quad \downarrow \text{mod } M \\ &= r' = \langle y \rangle_B \end{aligned}$$

Our Optimization: Speed Up (*) By Shortening Exponent

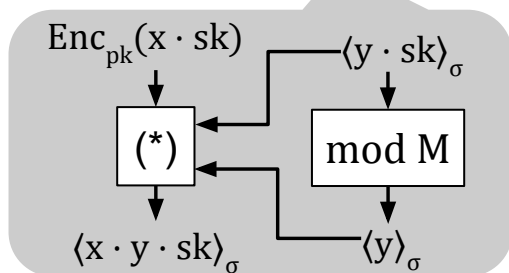
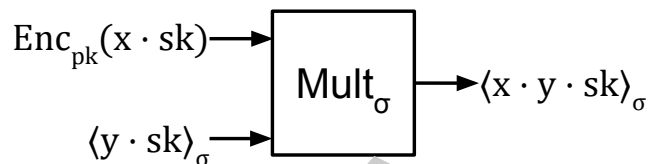
Recall our scheme

Running time of (*) $\approx$ two modular exponentiations



Our Optimization: Speed Up (*) By Shortening Exponent

Recall our scheme



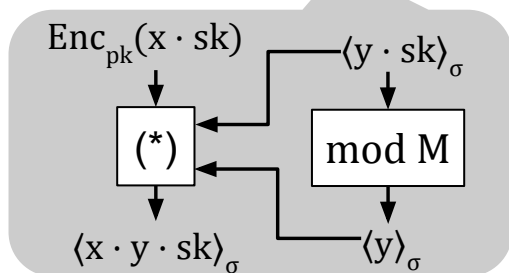
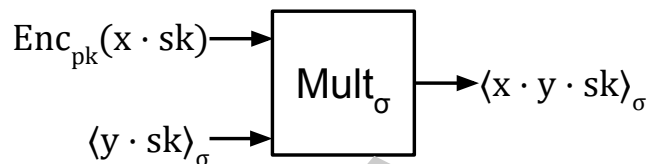
Running time of $(*) \approx$ two modular exponentiations
More precisely, this is the bottleneck:

$$\text{ct}_0^{\langle y \cdot \text{sk} \rangle_\sigma} \cdot \text{ct}_1^{\langle y \rangle_\sigma} \pmod{N^2}$$

where we unpack $\text{Enc}_{\text{pk}}(x \cdot \text{sk}) = (\text{ct}_0, \text{ct}_1)$

Our Optimization: Speed Up (*) By Shortening Exponent

Recall our scheme



Running time of $(*) \approx$ two modular exponentiations
More precisely, this is the bottleneck:

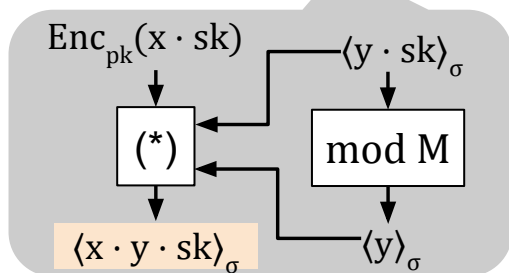
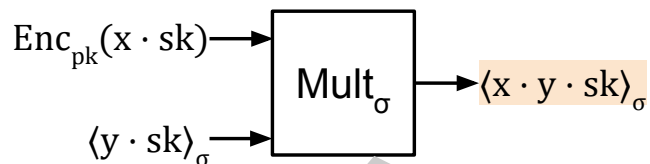
$$ct_0 \langle y \cdot sk \rangle_\sigma \cdot ct_1 \langle y \rangle_\sigma \pmod{N^2}$$

where we unpack $Enc_{pk}(x \cdot sk) = (ct_0, ct_1)$

Time to compute modular exponentiation scales
 $\sim$ **linearly** with the **length of the exponent**

Our Optimization: Speed Up (*) By Shortening Exponent

Recall our scheme



Output of $(*)$ is:

$$\log_2(N) \approx 3072 \text{ bits}$$

Running time of $(*) \approx$ two modular exponentiations
More precisely, this is the bottleneck:

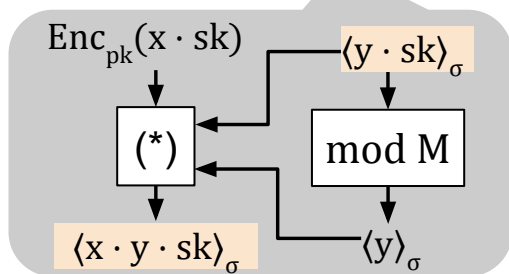
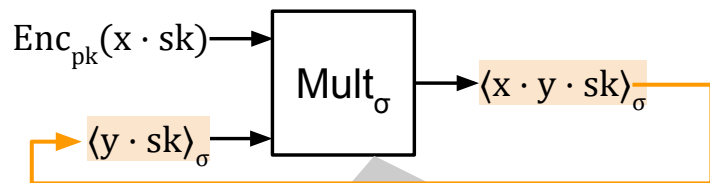
$$ct_0 \langle y \cdot sk \rangle_\sigma \cdot ct_1 \langle y \rangle_\sigma \pmod{N^2}$$

where we unpack $\text{Enc}_{\text{pk}}(x \cdot sk) = (ct_0, ct_1)$

Time to compute modular exponentiation scales
~ **linearly** with the **length of the exponent**

Our Optimization: Speed Up (*) By Shortening Exponent

Recall our scheme



Running time of $(*) \approx$ two modular exponentiations
More precisely, this is the bottleneck:

$$ct_0 \langle y \cdot sk \rangle_\sigma \cdot ct_1 \langle y \rangle_\sigma \pmod{N^2}$$

where we unpack $\text{Enc}_{\text{pk}}(x \cdot sk) = (ct_0, ct_1)$

3072 bits

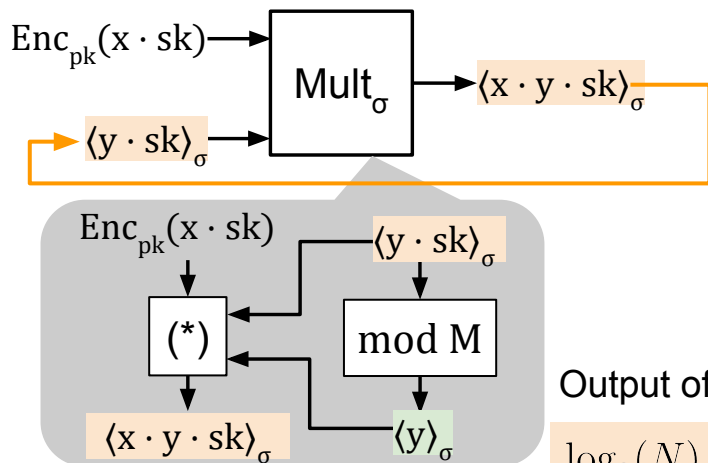
Output of $(*)$ is:

$$\log_2(N) \approx 3072 \text{ bits}$$

Time to compute modular exponentiation scales
 $\sim$ **linearly** with the **length of the exponent**

Our Optimization: Speed Up (*) By Shortening Exponent

Recall our scheme



Running time of $(*) \approx$ two modular exponentiations
More precisely, this is the bottleneck:

$$ct_0 \langle y \cdot sk \rangle_\sigma \cdot ct_1 \langle y \rangle_\sigma \pmod{N^2}$$

where we unpack $\text{Enc}_{\text{pk}}(x \cdot sk) = (ct_0, ct_1)$

3072 bits

$\log_2(B) + \lambda \approx 128$ bits thanks
to our mod M procedure

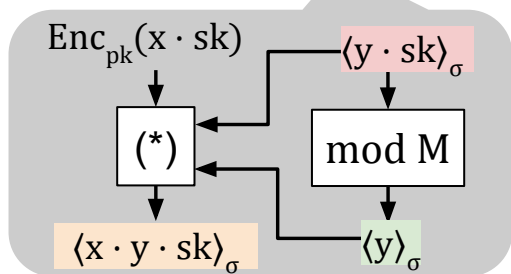
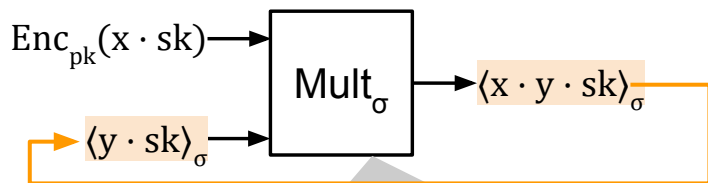
Output of $(*)$ is:

$\log_2(N) \approx 3072$ bits

Time to compute modular exponentiation scales
 $\sim$ **linearly** with the **length of the exponent**

Our Optimization: Speed Up (*) By Shortening Exponent

Recall our scheme



Running time of $(*) \approx$ two modular exponentiations
More precisely, this is the bottleneck:

$$ct_0 \langle y \cdot sk \rangle_\sigma \cdot ct_1 \langle y \rangle_\sigma \pmod{N^2}$$

where we unpack $Enc_{pk}(x \cdot sk) = (ct_0, ct_1)$

3072 bits

$\log_2(B) + \lambda \approx 128$ bits thanks
to our mod M procedure

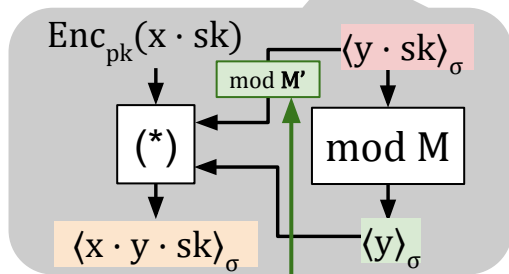
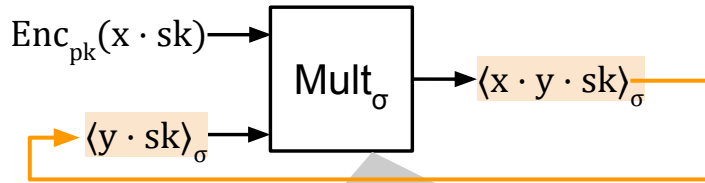
Output of $(*)$ is:

$\log_2(N) \approx 3072$ bits

Time to compute modular exponentiation scales
~ **linearly** with the **length of the exponent**

Our Optimization: Speed Up (*) By Shortening Exponent

Recall our scheme



Running time of $(*) \approx$ two modular exponentiations
More precisely, this is the bottleneck:

$$ct_0 \langle y \cdot sk \rangle_\sigma \cdot ct_1 \langle y \rangle_\sigma \pmod{N^2}$$

where we unpack $\text{Enc}_{\text{pk}}(x \cdot sk) = (ct_0, ct_1)$

3072 bits

$\log_2(B) + \lambda \approx 128$ bits thanks
to our mod M procedure

Output of $(*)$ is:

$\log_2(N) \approx 3072$ bits

Time to compute modular exponentiation scales
 $\sim$ **linearly** with the **length of the exponent**

Idea: Use the same “mod M ” trick, but use a larger modulus M'

Recall: Simplifying memory shares (**this work**)

Notation

$$x = \langle x \rangle_A - \langle x \rangle_B$$

Goal: derive $\langle y \rangle_\sigma$ locally from $\langle y \cdot sk \rangle_\sigma$



$$\langle y \cdot sk \rangle_A$$

$$= r + y \cdot sk$$

$$\downarrow \text{mod } M$$

$$= r' + y \cdot sk \pmod{M}$$

$$sk \equiv 1 \pmod{M}$$

$$\text{Pick } M \leq N \cdot 2^{-\lambda}$$

$$r \leftarrow \{0, \dots, N-1\}$$



$$\langle y \cdot sk \rangle_B$$

$$= r$$

$$\downarrow \text{mod } M$$

$$= r'$$

$$r' \approx_s \{0, \dots, M-1\}$$

$$r' := r \bmod M$$

Shortening memory shares (this work)

Goal: derive a shorter share $\langle y \cdot \text{sk} \rangle_\sigma$, of length $\log_2(\mathbf{M}')$ bits.


$$\begin{aligned} & \langle y \cdot \text{sk} \rangle_A \\ &= r + y \cdot \text{sk} \\ & \quad \downarrow \text{mod } \mathbf{M}' \\ &= r' + y \cdot \text{sk} \pmod{\mathbf{M}'} \end{aligned}$$

$$\text{sk} \not\equiv 1 \pmod{\mathbf{M}'}$$

$$\text{sk} \equiv 1 \pmod{M}$$

Pick $\mathbf{M}' \leq N \cdot 2^{-\lambda}$

$$r \leftarrow \$ \{0, \dots, N - 1\}$$

$$r' \approx_s \{0, \dots, \mathbf{M}' - 1\}$$

$$r' := r \bmod \mathbf{M}'$$

Notation

$$x = \langle x \rangle_A - \langle x \rangle_B$$


$$\begin{aligned} & \langle y \cdot \text{sk} \rangle_B \\ &= r \\ & \quad \downarrow \text{mod } \mathbf{M}' \\ &= r' \end{aligned}$$

Shortening memory shares (this work)

Notation

$$x = \langle x \rangle_A - \langle x \rangle_B$$

Goal: derive a shorter share $\langle y \cdot sk \rangle_\sigma$, of length $\log_2(\mathbf{M}')$ bits.



$$\langle y \cdot sk \rangle_A$$

$$= r + y \cdot sk$$

$$\downarrow \text{mod } \mathbf{M}'$$

$$= r' + y \cdot sk \pmod{\mathbf{M}'}$$

$$= r' + y \cdot sk \pmod{\mathbb{Z}} = \langle y \cdot sk \rangle_A$$

holds if $r' < \mathbf{M}' - y \cdot sk$, which is true with probability $\geq 1 - 2^{-\lambda}$

$$r' \approx_s \{0, \dots, \mathbf{M}' - 1\}$$

$$r' := r \text{ mod } \mathbf{M}'$$

$$\text{Pick } \mathbf{M}' = \max(y) \cdot \max(sk) \cdot 2^\lambda$$

$$\text{Need } y \cdot sk \cdot 2^\lambda \leq \mathbf{M}' \leq N \cdot 2^{-\lambda}$$

$$r \leftarrow \{0, \dots, N - 1\}$$



$$\langle y \cdot sk \rangle_B$$

$$= r$$

$$\downarrow \text{mod } \mathbf{M}'$$

$$= r' = \langle y \cdot sk \rangle_B$$

Shortening memory shares (this work)

Notation

$$x = \langle x \rangle_A - \langle x \rangle_B$$

Goal: derive a shorter share $\langle y \cdot sk \rangle_\sigma$, of length $\log_2(\mathbf{M}')$ bits.



$$\langle y \cdot sk \rangle_A$$

$$= r + y \cdot sk$$

$$\downarrow \text{mod } \mathbf{M}'$$

$$= r' + y \cdot sk \pmod{\mathbf{M}'}$$

$$= r' + y \cdot sk \pmod{\mathbb{Z}} = \langle y \cdot sk \rangle_A$$

$$\text{Pick } \mathbf{M}' = \max(y) \cdot \max(sk) \cdot 2^\lambda$$

$$|sk| \leq B^2 \cdot 2^{6\lambda}, \quad B := \max(y)$$

$$\text{Can set } \mathbf{M}' = B^3 \cdot 2^{7\lambda}$$

$$B = 1, \lambda = 128 \Rightarrow \mathbf{M}' \text{ is 896 bits}$$

$$\text{(cf. } N \text{ is 3072 bits)}$$



$$\langle y \cdot sk \rangle_B$$

$$= r$$

$$\downarrow \text{mod } \mathbf{M}'$$

$$= r' = \langle y \cdot sk \rangle_B$$

holds if $r' < \mathbf{M}' - y \cdot sk$, which is true with probability $\geq 1 - 2^{-\lambda}$

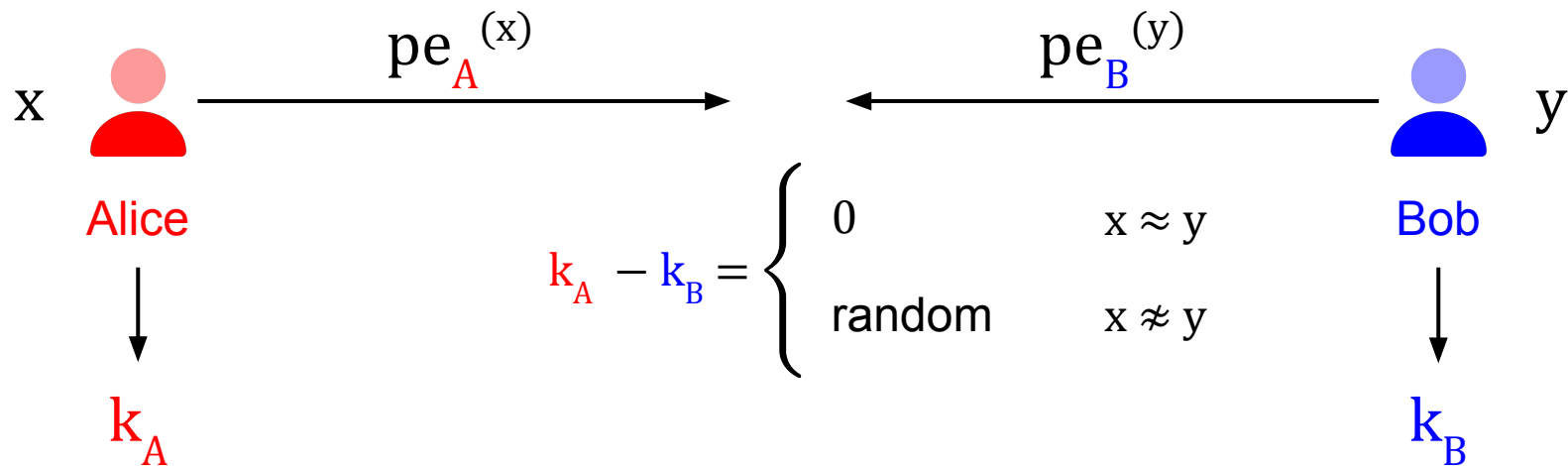
$$r' \approx_s \{0, \dots, \mathbf{M}' - 1\}$$

$$r' := r \bmod \mathbf{M}'$$

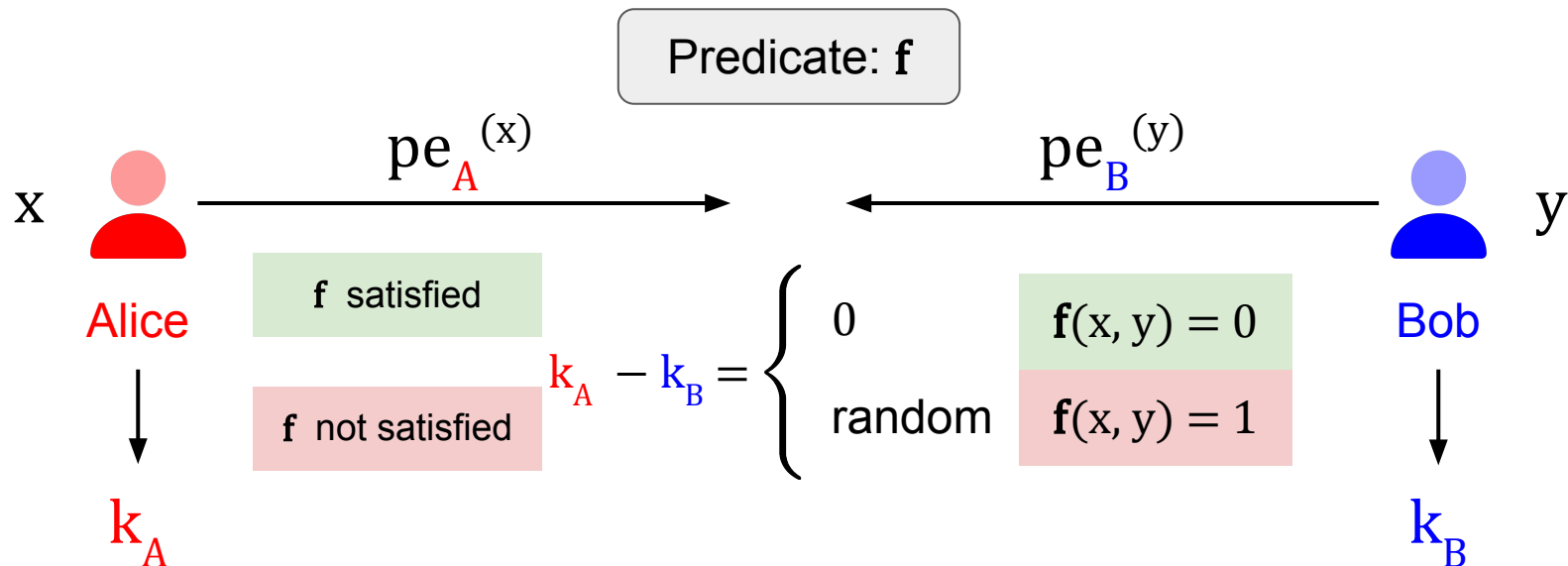
Roadmap

1. Overview of our work
2. MKHSS optimizations
3. **Non-interactive conditional key exchange optimizations**
4. Useful instantiations of key exchange
 - a. Fuzzy PAKE
 - b. Geolocation-based key exchange
5. Performance evaluation
6. Future works and conclusion

Non-Interactive Conditional Key Exchange [CDHJS'25]



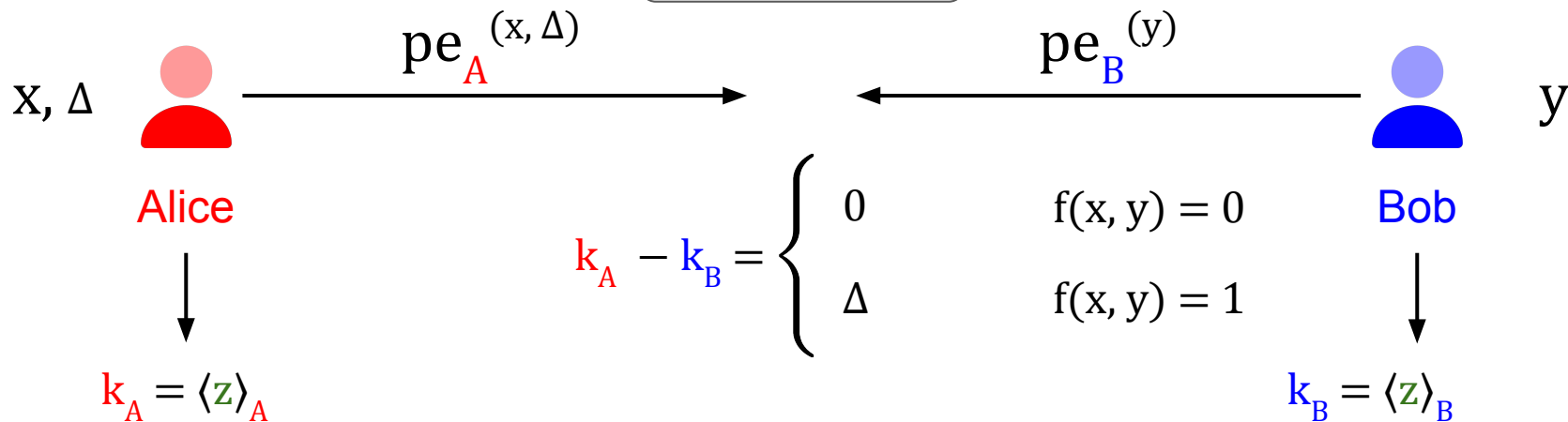
Formalizing Non-Interactive Conditional Key Exchange [CDHJS'25]



Naïve Attempt 1

$$\Delta \leftarrow \$ \{1, \dots, 2^\lambda - 1\}$$

Predicate: f



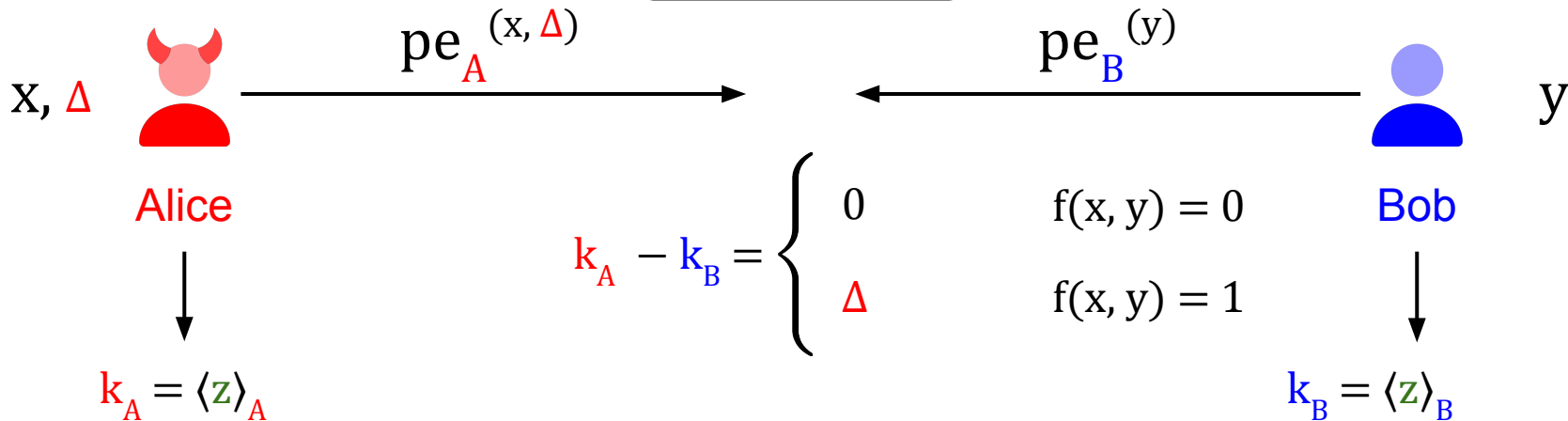
HSS Output

$$z = f(x, y) \cdot \Delta$$

Naïve Attempt 1

$$\Delta \leftarrow \$ \{1, \dots, 2^\lambda - 1\}$$

Predicate: f



Attack

Even if predicate **is not satisfied** ($f(x, y) = 1$), Alice can still compute:

$$k_B = k_A - \Delta$$

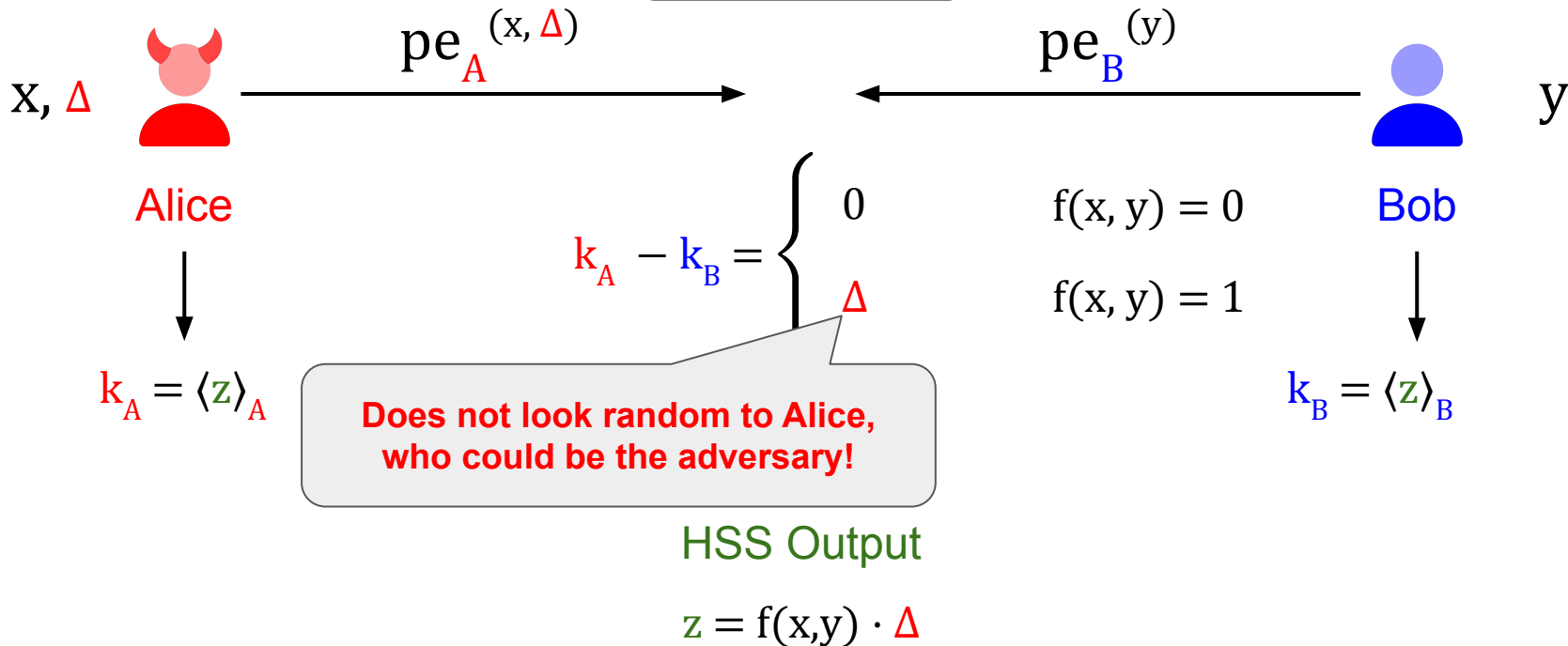
HSS Output

$$z = f(x, y) \cdot \Delta$$

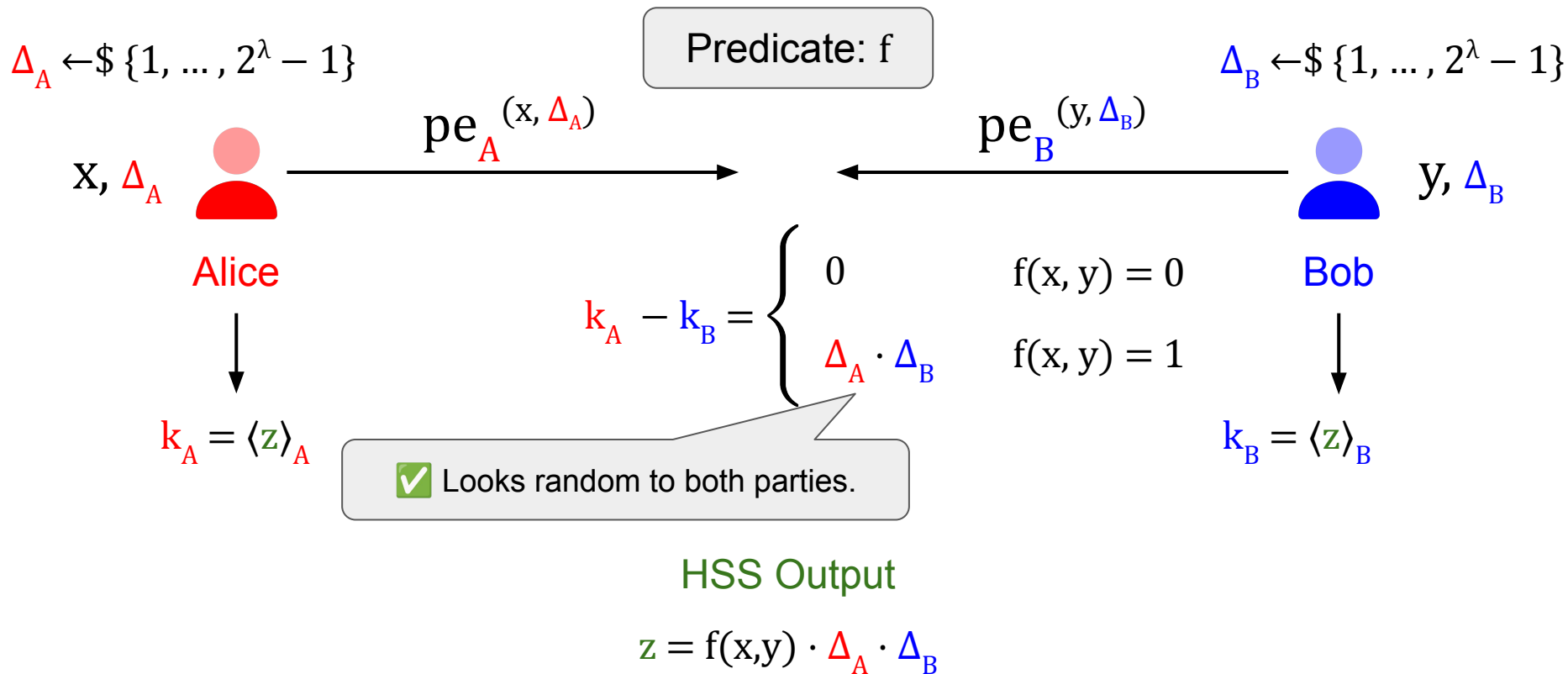
Naïve Attempt 1

$$\Delta \leftarrow \$ \{1, \dots, 2^\lambda - 1\}$$

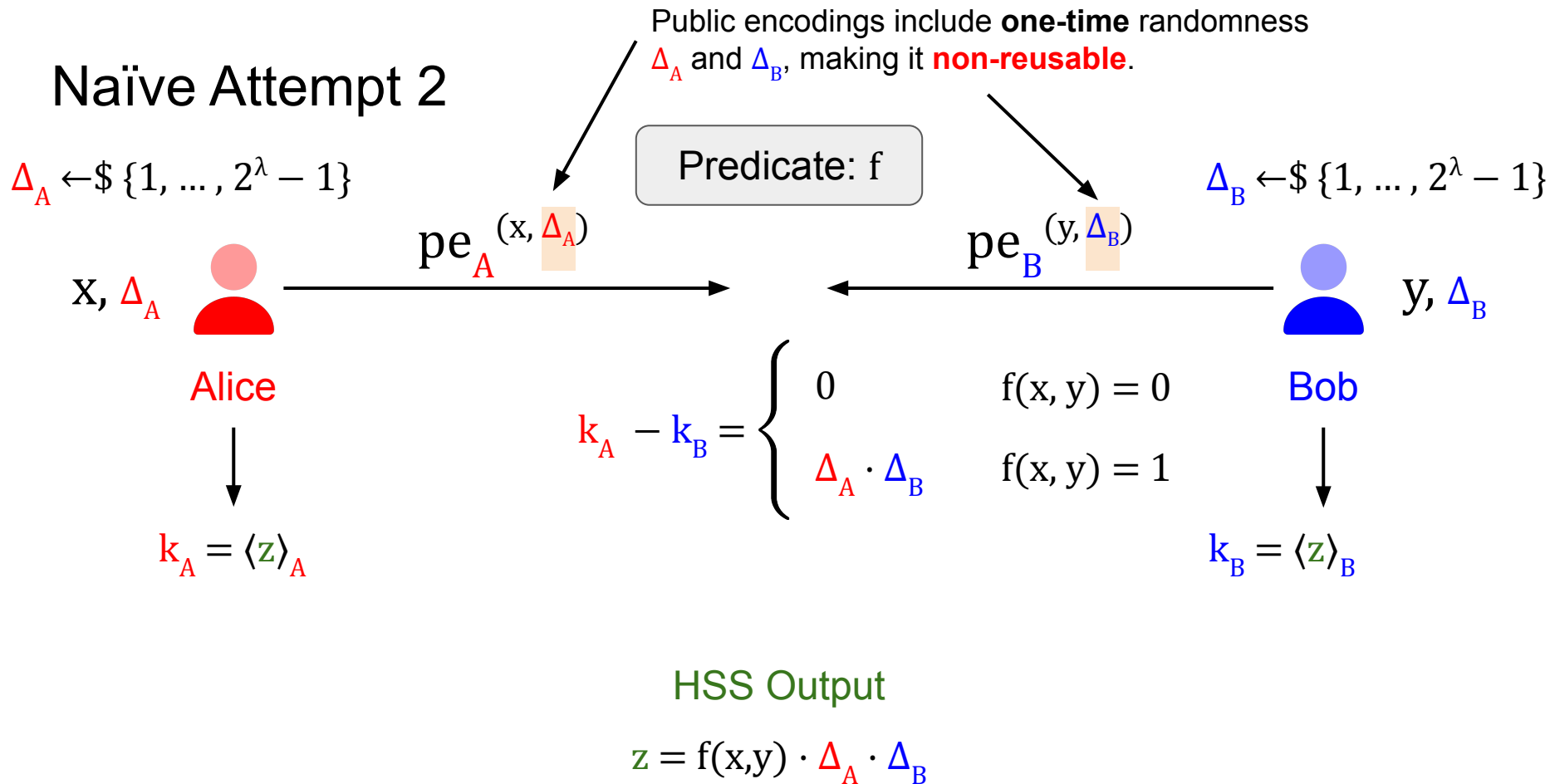
Predicate: f



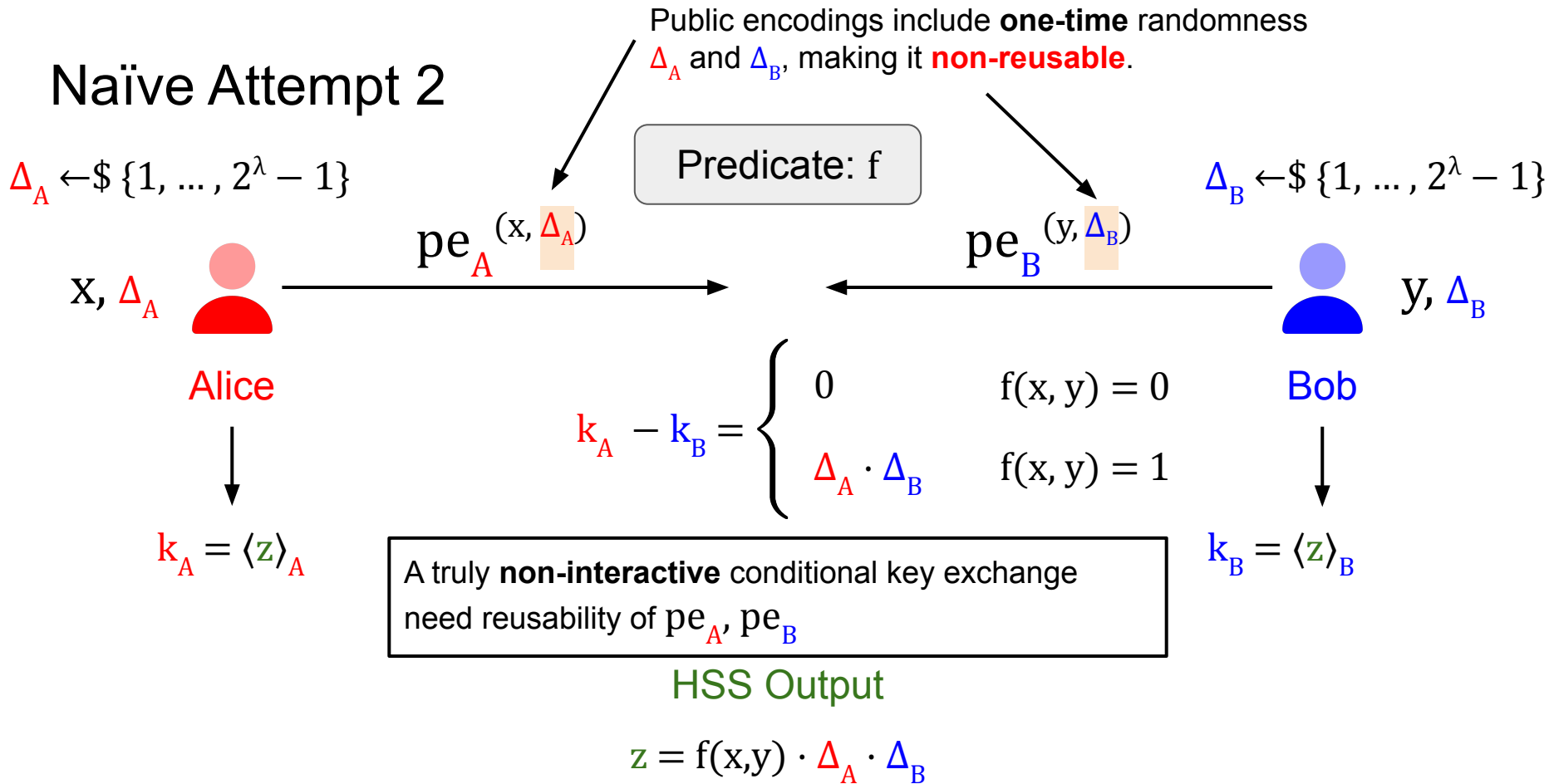
Naïve Attempt 2



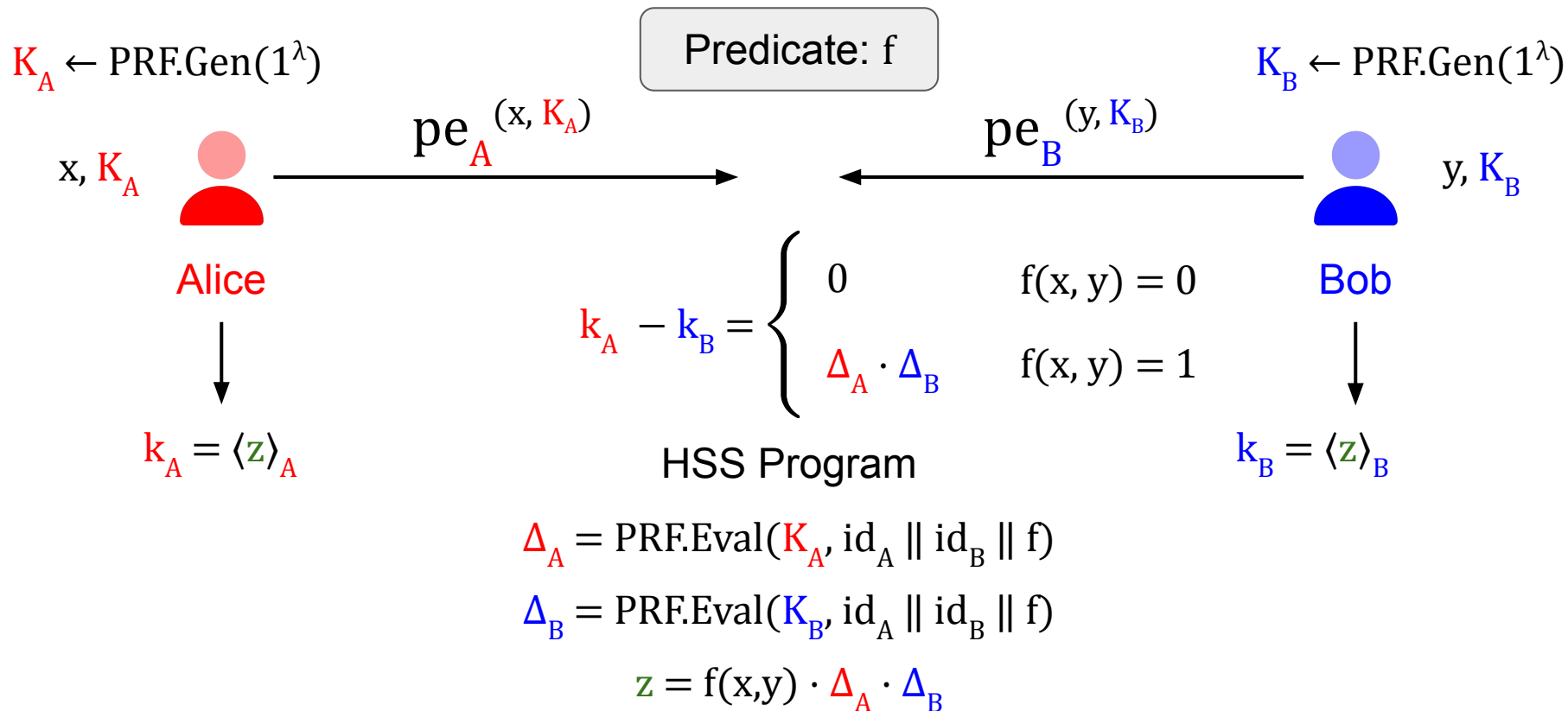
Naïve Attempt 2



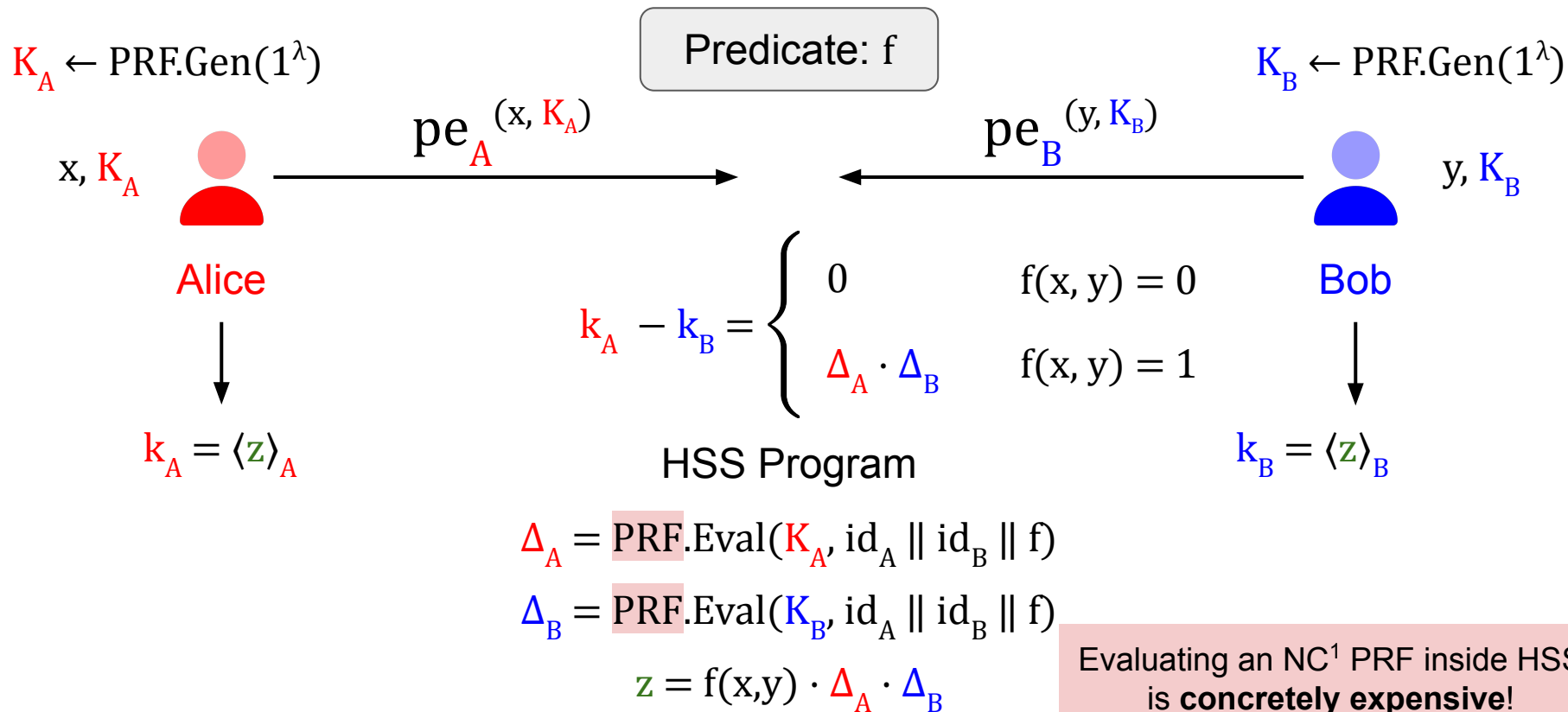
Naïve Attempt 2



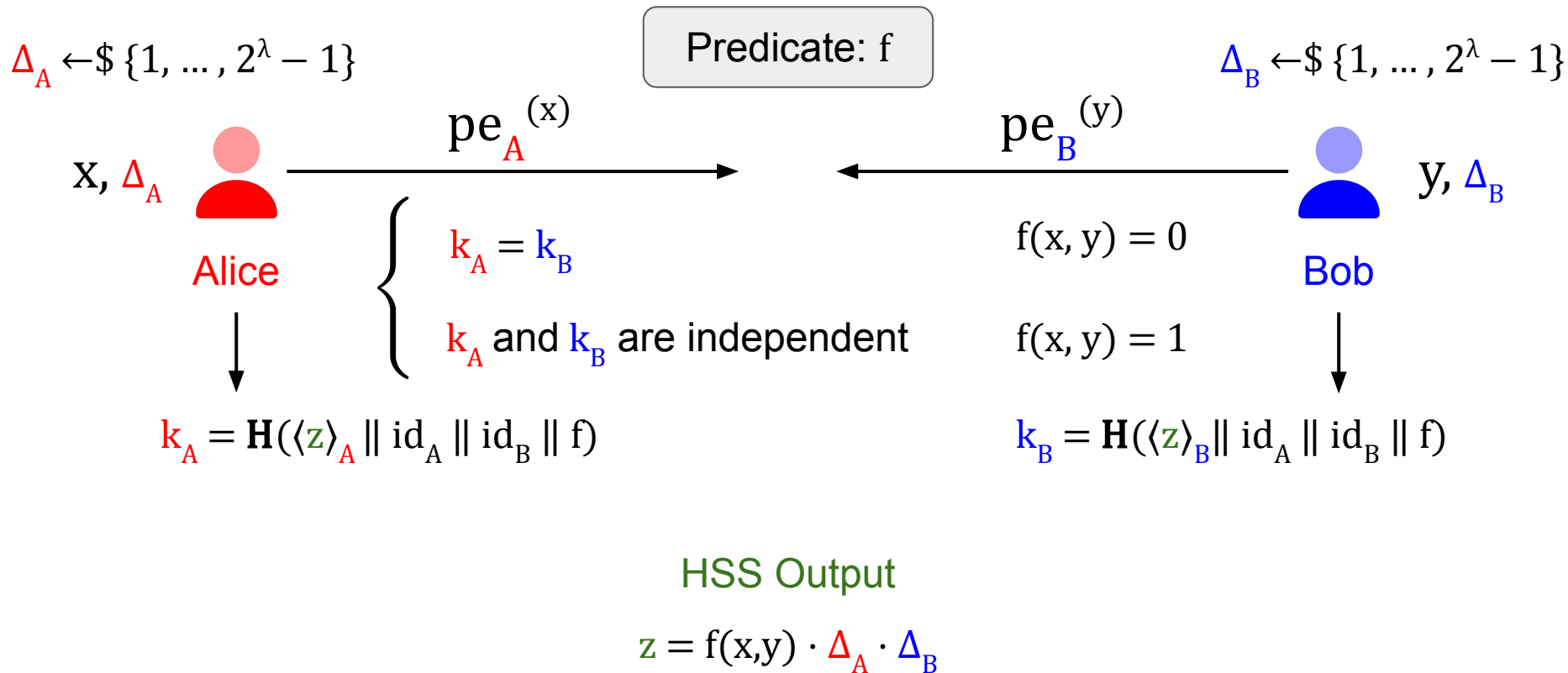
Solution by Prior Work: Use a PRF [CDHJS'25]



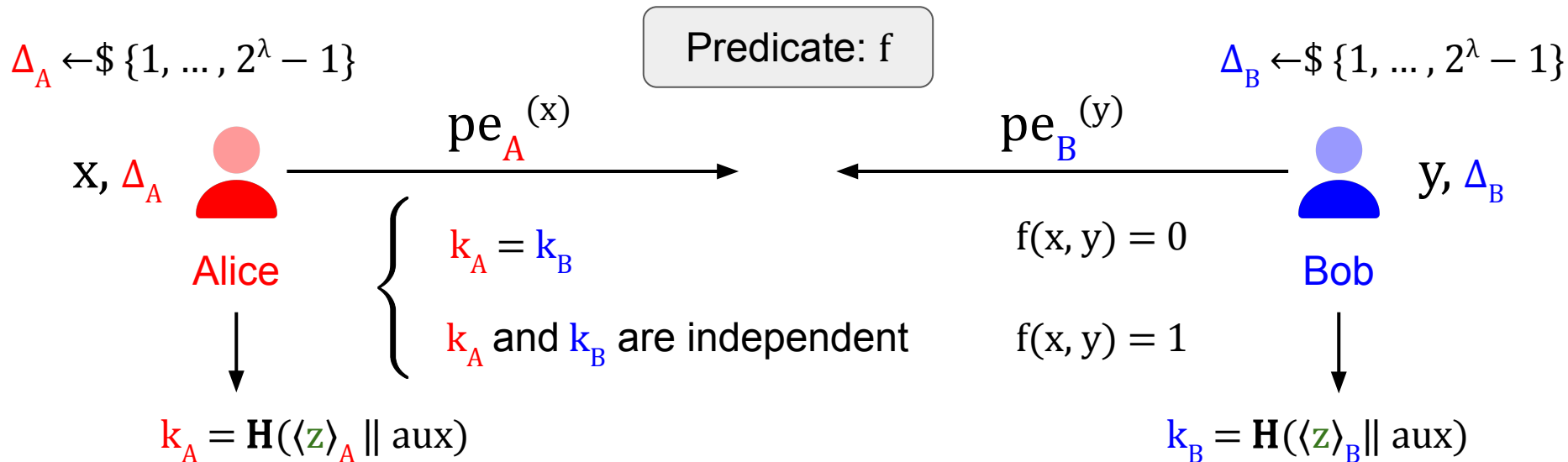
Solution by Prior Work: Use a PRF [CDHJS'25]



Our Optimization #1: Use a Hash Outside of HSS Instead



Our Optimization #1: Use a Hash Outside of HSS Instead

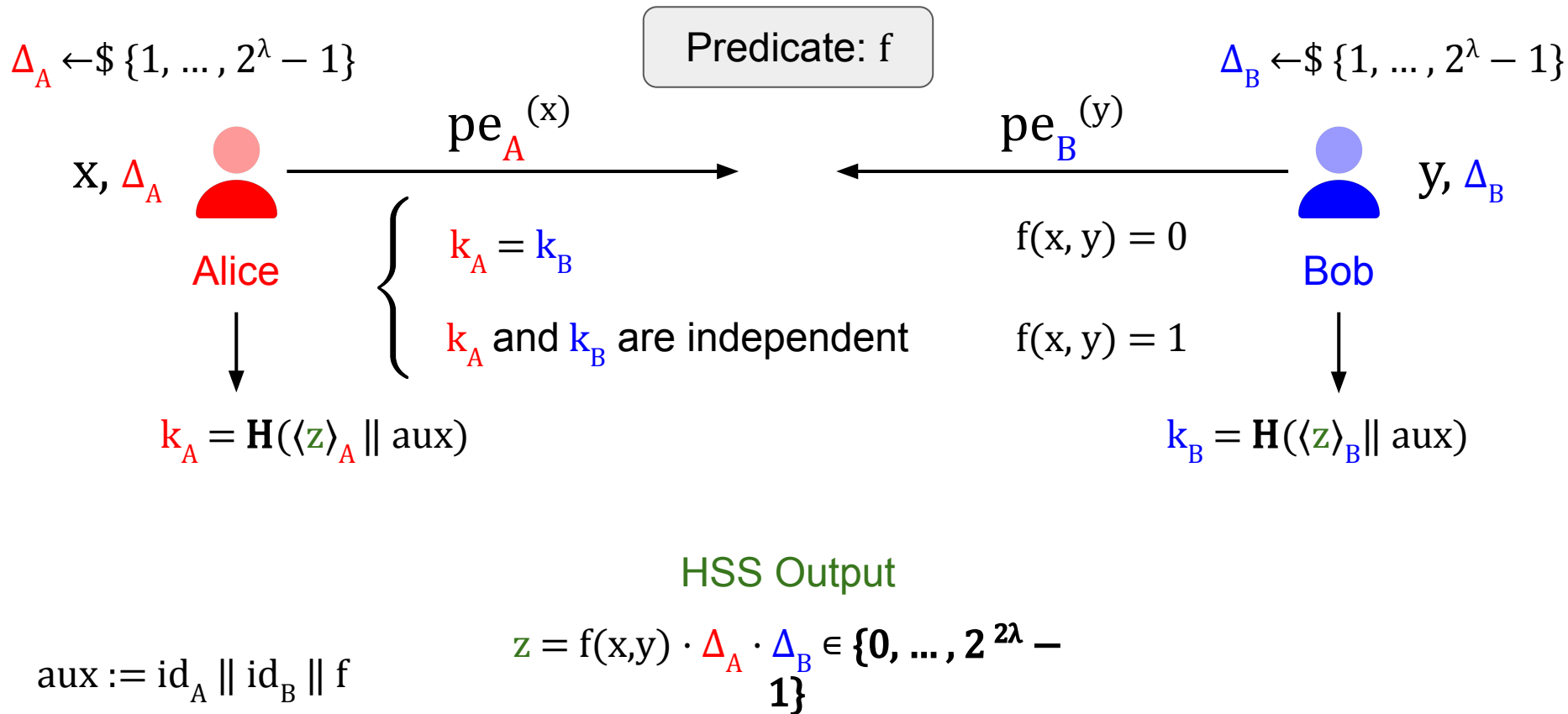


HSS Output

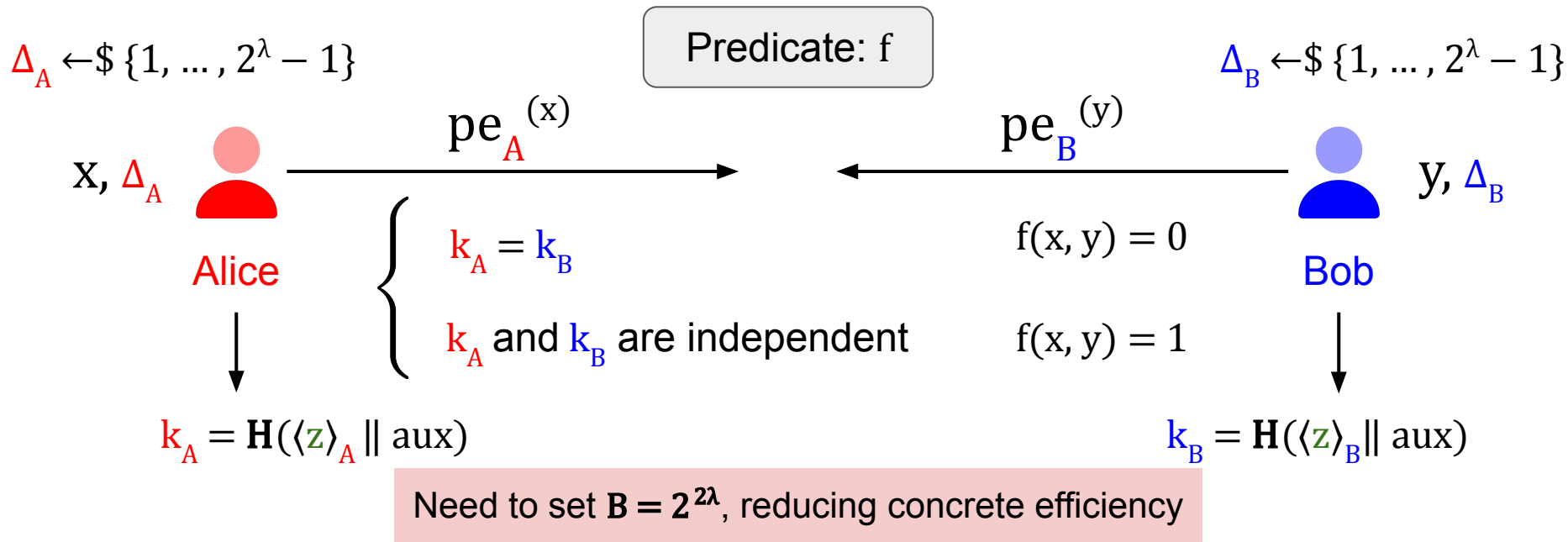
$$z = f(x, y) \cdot \Delta_A \cdot \Delta_B$$

$$\text{aux} := \text{id}_A \parallel \text{id}_B \parallel f$$

Inefficiency: Need HSS to Support Large Integer Values



Inefficiency: Need HSS to Support Large Integer Values

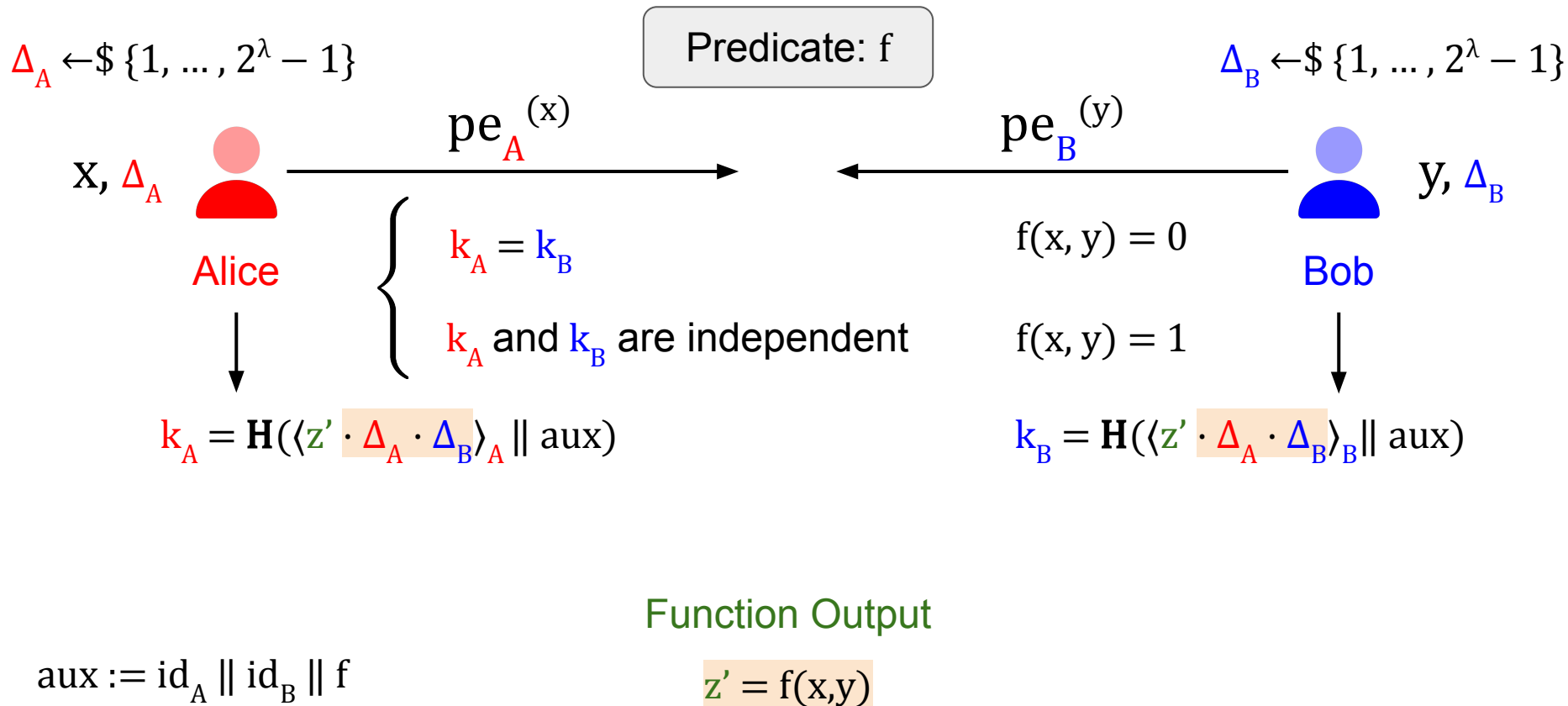


HSS Output

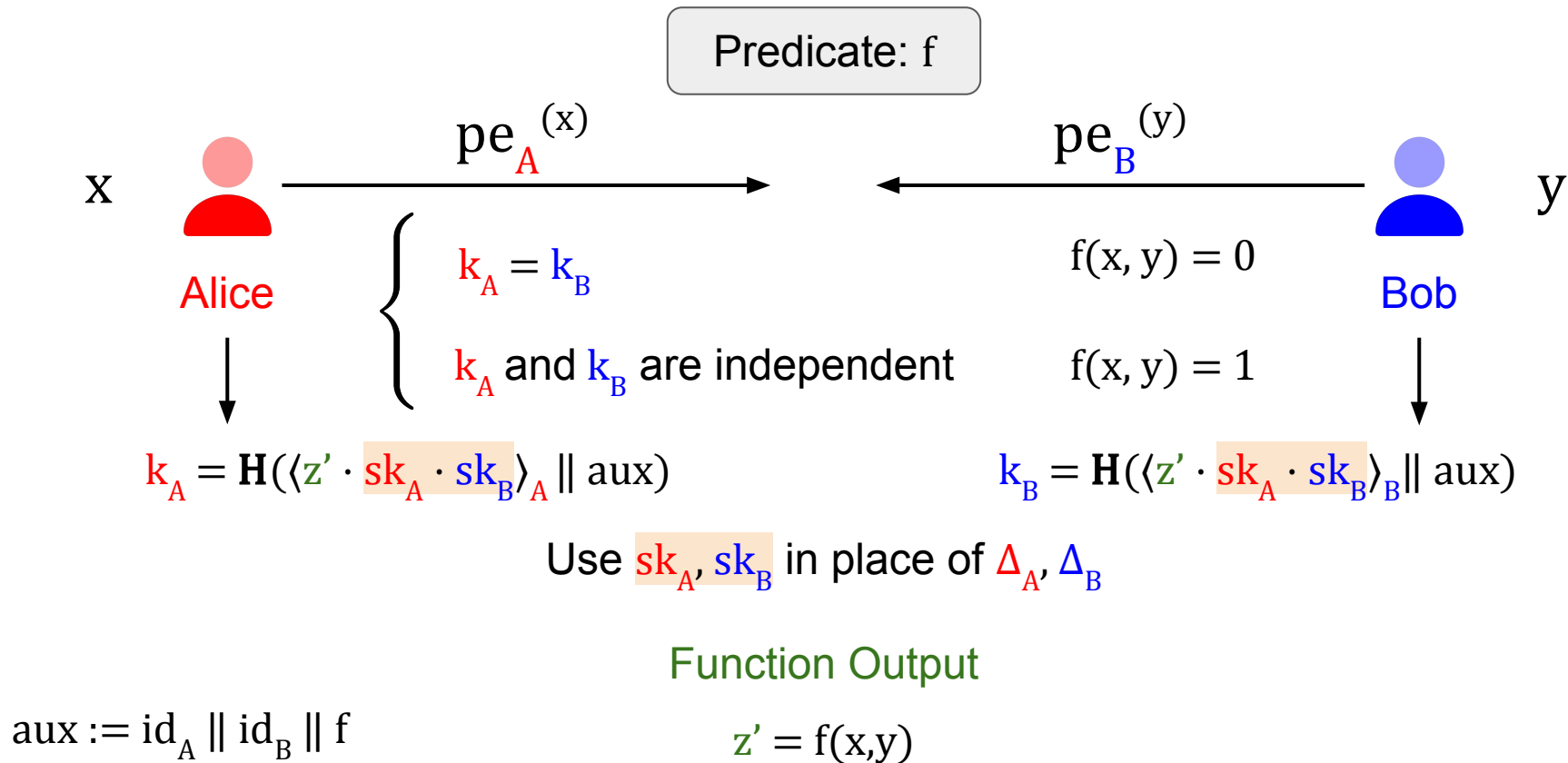
$$\text{aux} := \text{id}_A \parallel \text{id}_B \parallel f$$

$$z = f(x, y) \cdot \Delta_A \cdot \Delta_B \in \{0, \dots, 2^{2\lambda} - 1\}$$

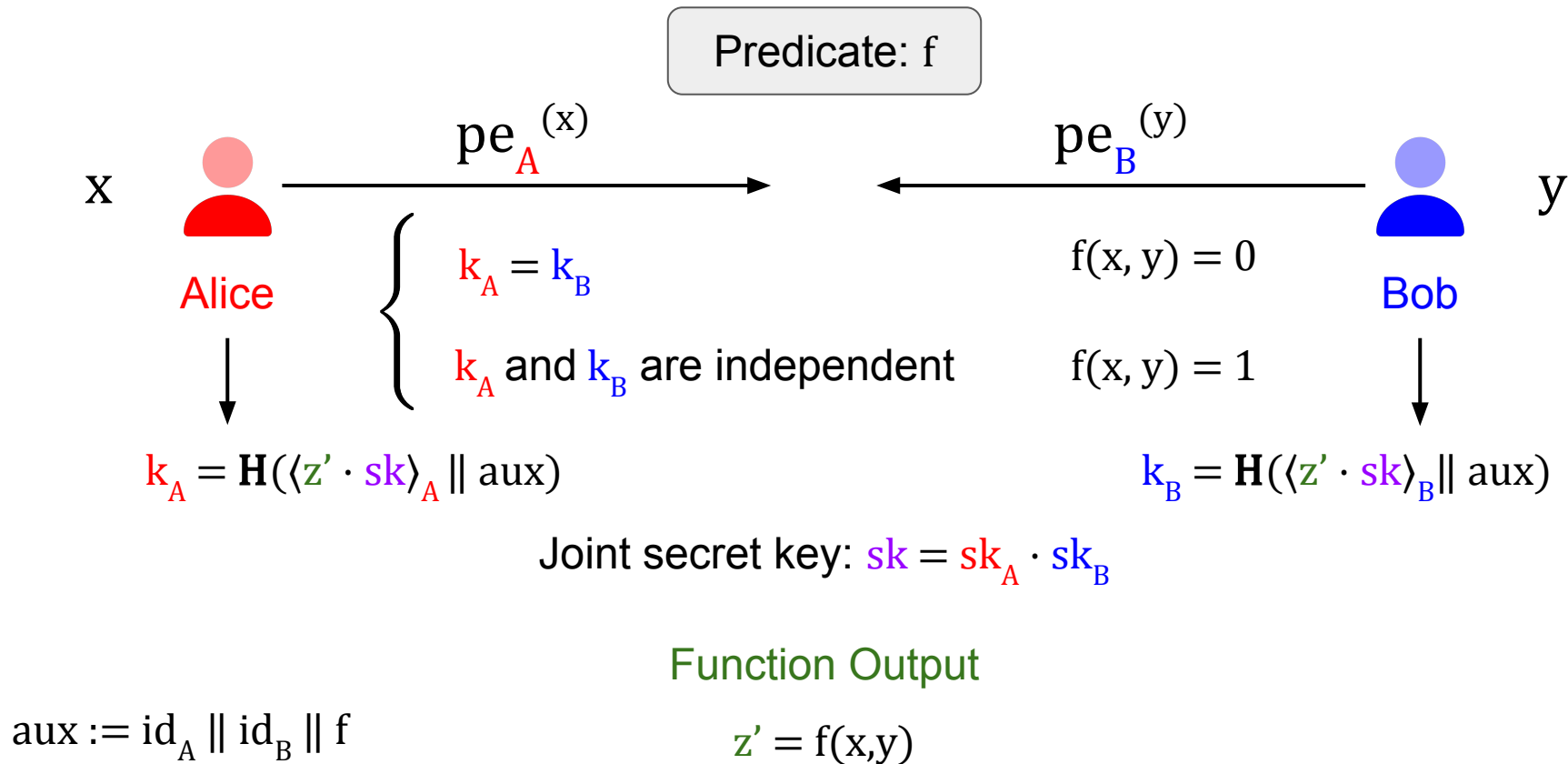
Our Optimization #2: Exploit Memory Share Structure



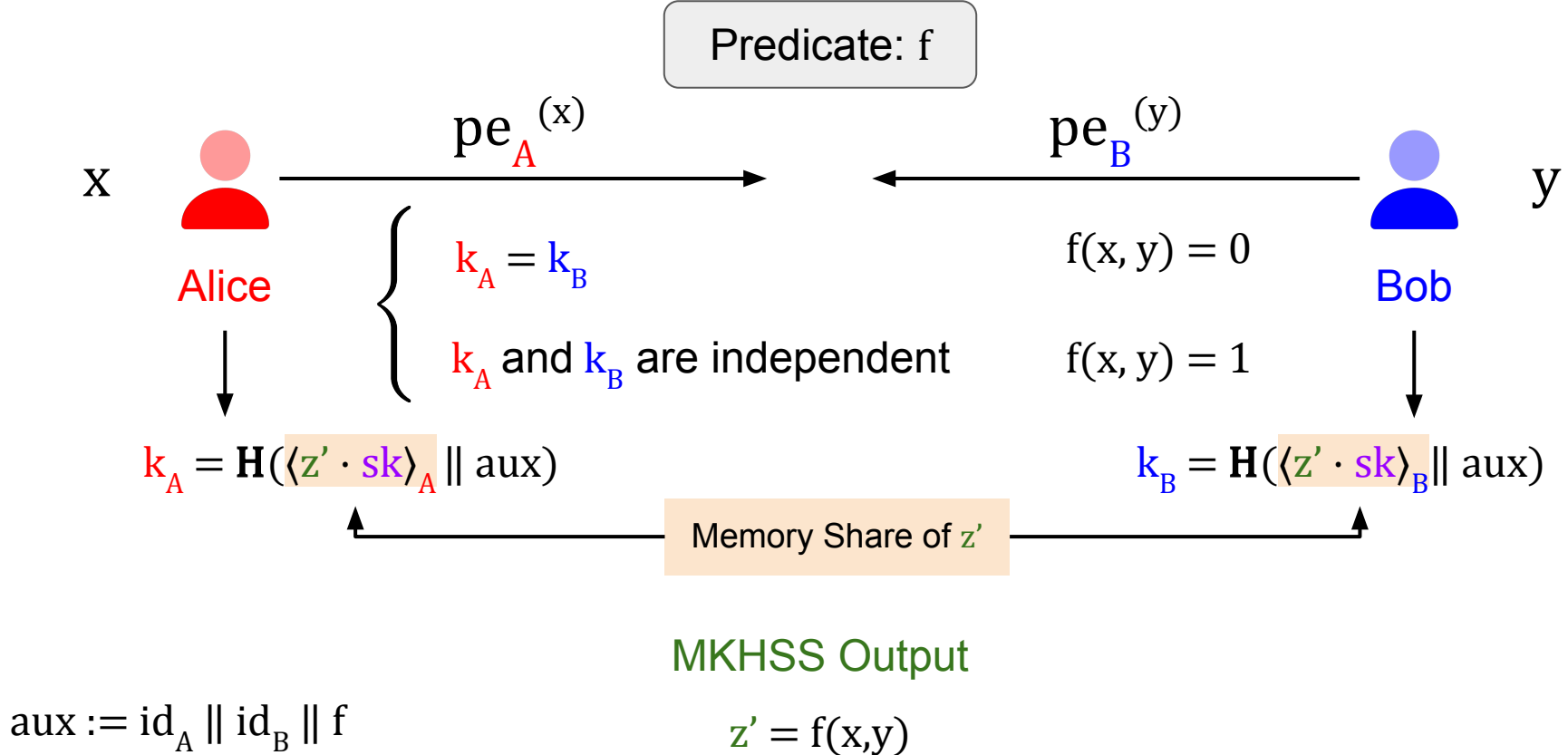
Our Optimization #2: Exploit Memory Share Structure



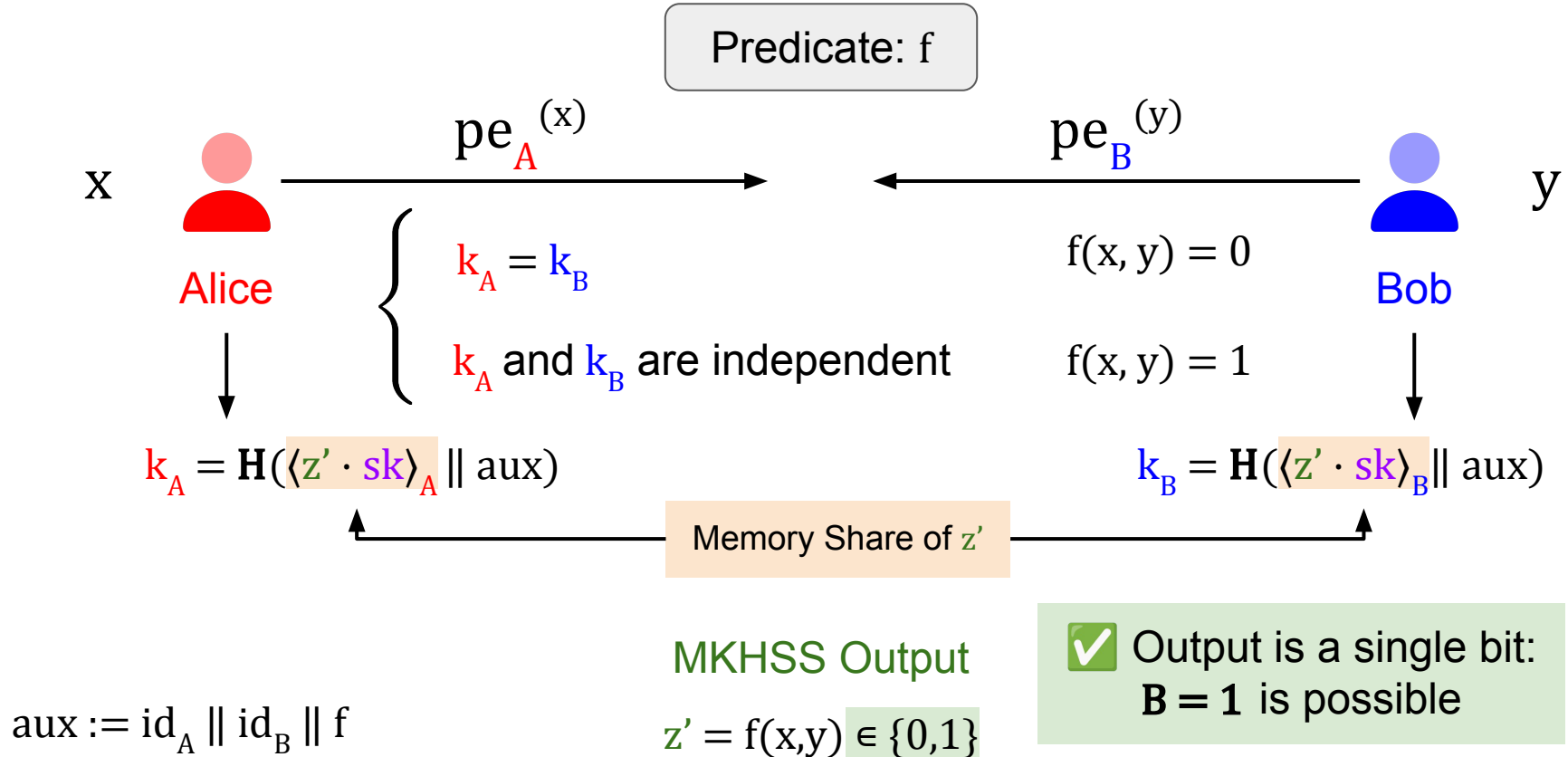
Our Optimization #2: Exploit Memory Share Structure



Our Optimization #2: Exploit Memory Share Structure



Our Optimization #2: Exploit Memory Share Structure



Roadmap

1. Overview of our work
2. MKHSS optimizations
3. Non-interactive conditional key exchange optimizations
- 4. Useful instantiations of key exchange**
 - a. Fuzzy PAKE
 - b. Geolocation-based key exchange
5. Performance evaluation
6. Future works and conclusion

Prior work requires 5+ rounds of interaction

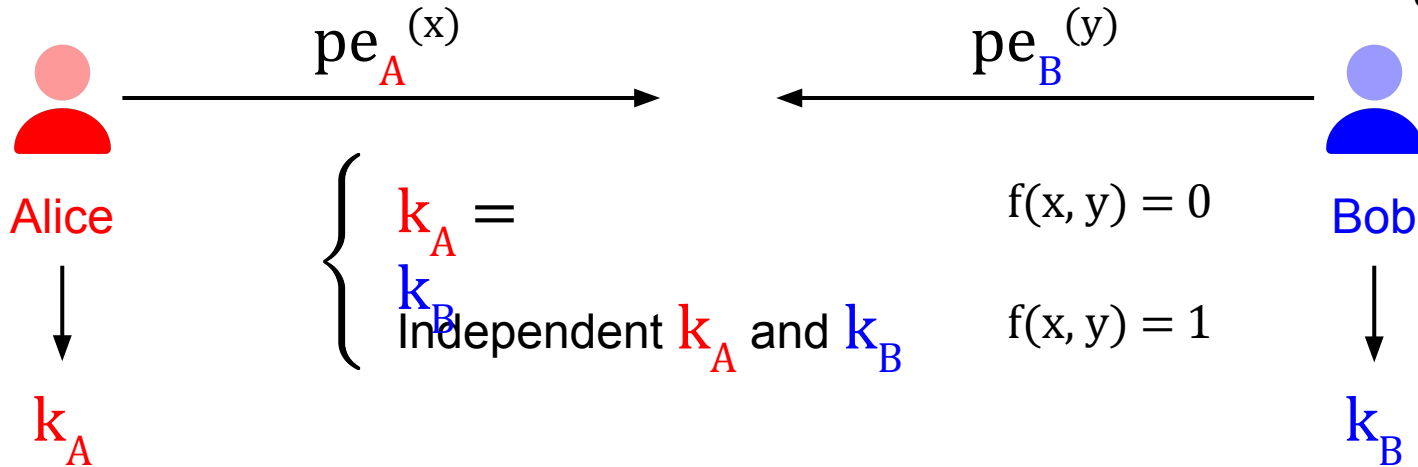
Concrete Instantiation: **f**PAKE [DHPRY'18]

fPAKE: **fuzzy** password-authenticated key exchange

Predicate: $f(x, y) = 1 \Leftrightarrow \text{EditDistance}(x, y) \leq T$

correct
horse
battery
staple = x

corrupt
hose
buttery
stable = y



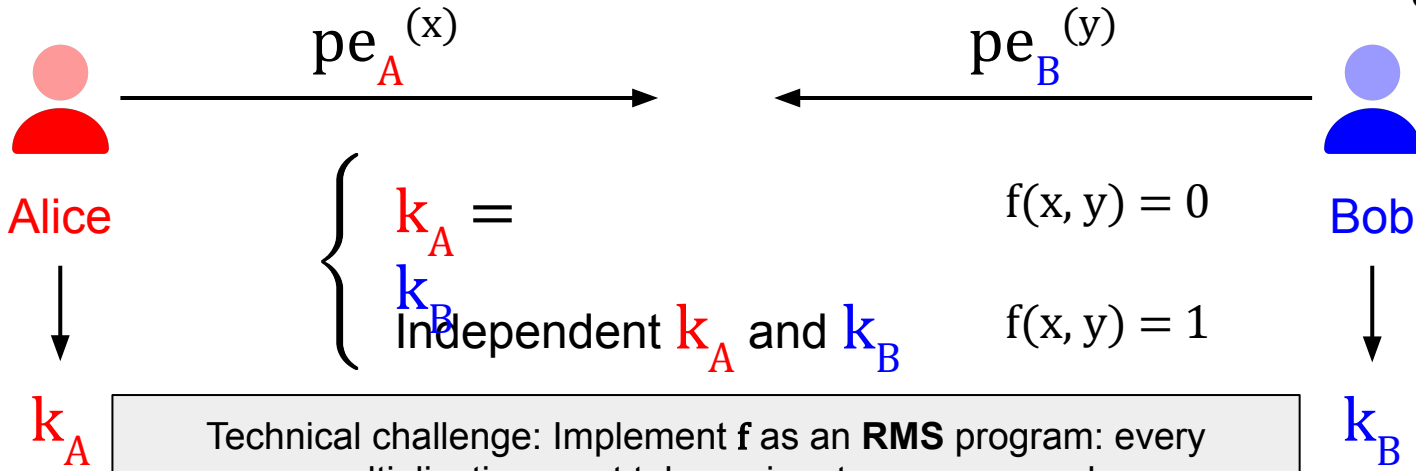
Concrete Instantiation: **f**PAKE

fPAKE: **fuzzy** password-authenticated key exchange

Predicate: $f(x,y) = 1 \Leftrightarrow \text{EditDistance}(x, y) \leq T$

correct
horse
battery
staple = x

corrupt
hose
buttery
stable = y



Technical challenge: Implement **f** as an **RMS** program: every multiplication must take an input as an operand

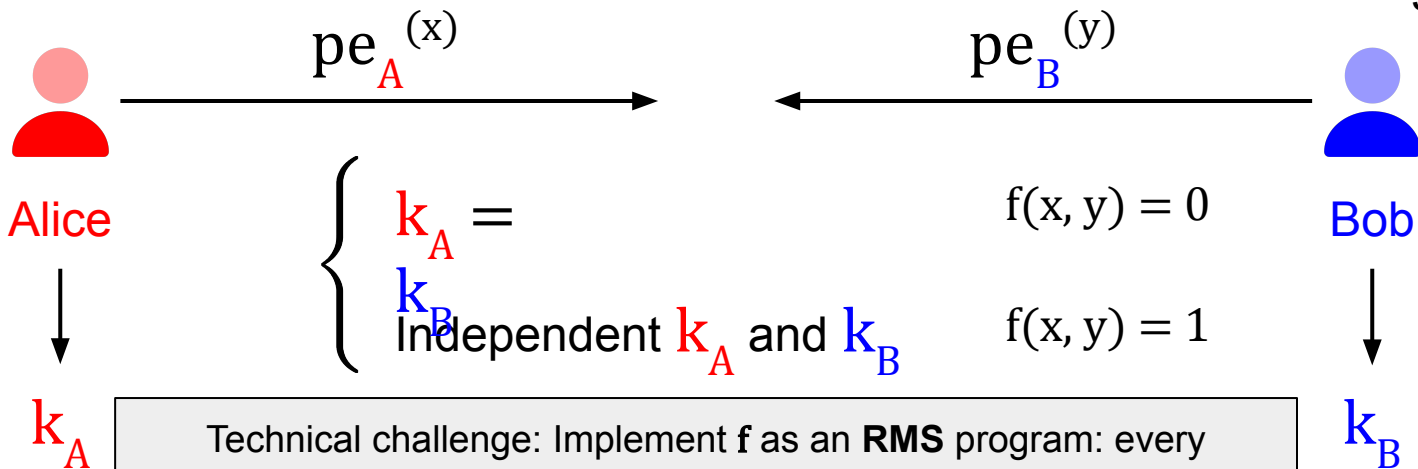
Concrete Instantiation: fPAKE

fPAKE: **fuzzy** password-authenticated key exchange

Predicate: $f(x,y) = 1 \Leftrightarrow \text{EditDistance}(x, y) \leq T$

correct
horse
battery
staple = x

corrupt
hose
buttery
stable = y



Our contribution: compute useful fuzziness metrics

Useful fuzziness metric: Hamming distance

x = correct horse battery staple

y = corrupt house buttery stable

$$(\text{HD}(x, y) = 4)$$

$$x \approx y \Leftrightarrow \text{HD}(x, y) \leq T$$

Our contribution: compute useful fuzziness metrics

Useful fuzziness metric: Hamming distance

x = correct horse battery staple

y = corrupt house buttery stable

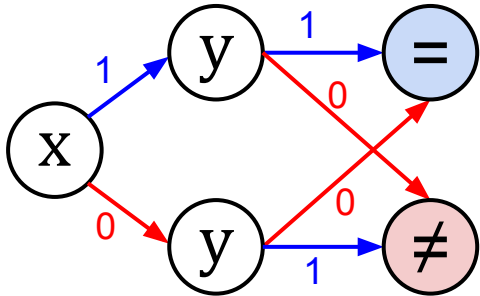
$$(\text{HD}(x, y) = 4)$$

$$x \approx y \Leftrightarrow \text{HD}(x, y) \leq T$$

As a warmup, we show how to compute this for **binary** strings.

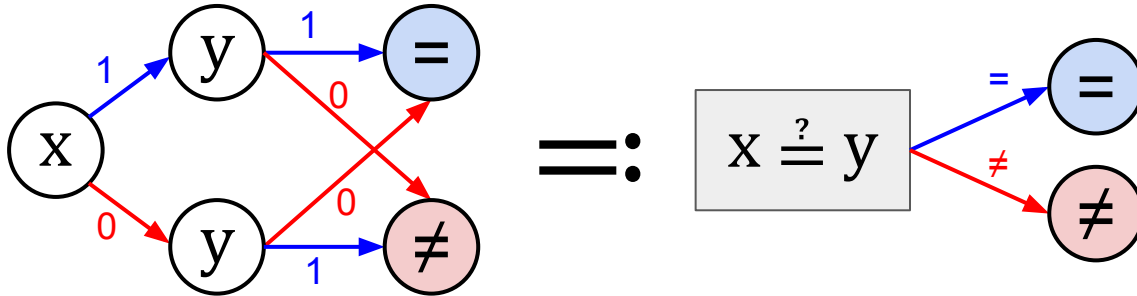
Warmup: branching program for binary Hamming distance

Bit equality: $x \stackrel{?}{=} y$



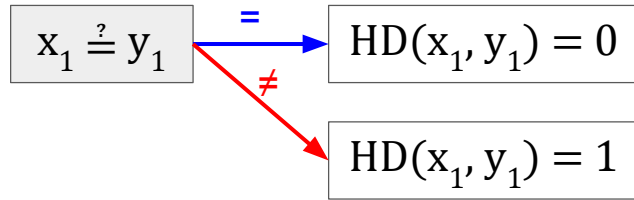
Warmup: branching program for binary Hamming distance

Bit equality: $x \stackrel{?}{=} y$



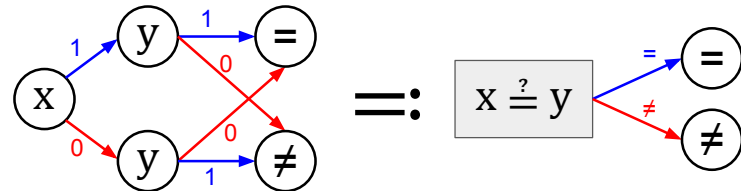
Warmup: branching program for binary Hamming distance

Threshold Hamming distance: $HD(x, y) \stackrel{?}{\leq} T$



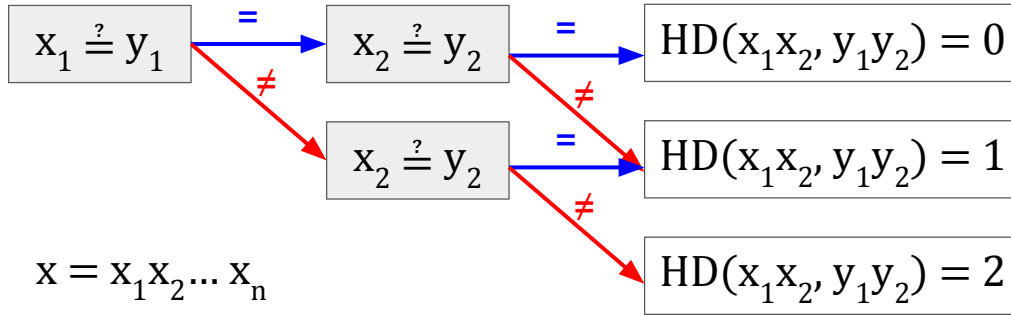
$$x = x_1 x_2 \dots x_n$$

$$y = y_1 y_2 \dots y_n$$



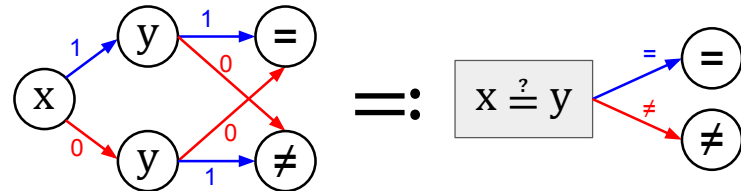
Warmup: branching program for binary Hamming distance

Threshold Hamming distance: $HD(x, y) \stackrel{?}{\leq} T$



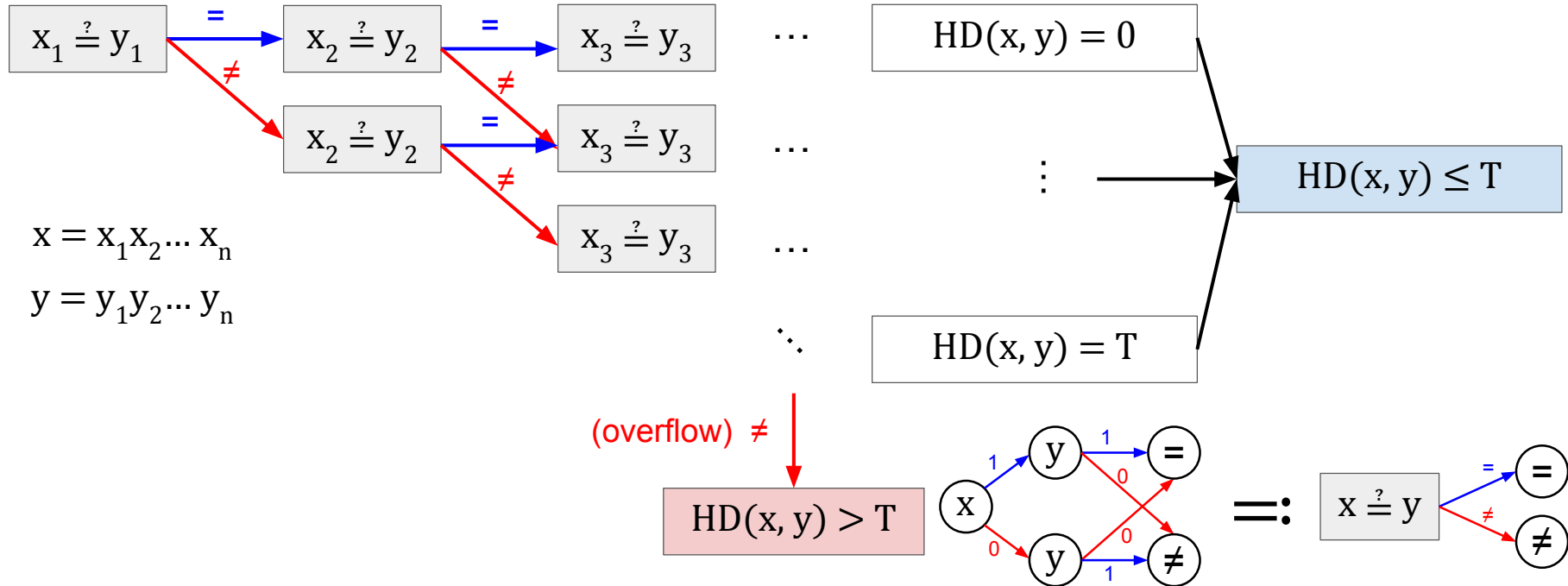
$$x = x_1x_2 \cdots x_n$$

$$y = y_1y_2 \cdots y_n$$



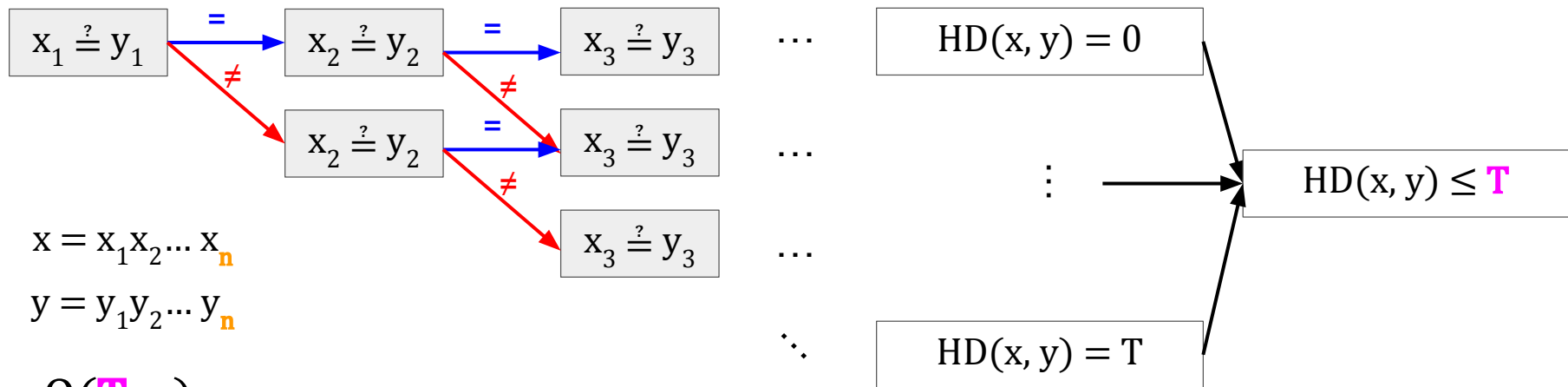
Warmup: branching program for binary Hamming distance

Threshold Hamming distance: $HD(x, y) \stackrel{?}{\leq} T$

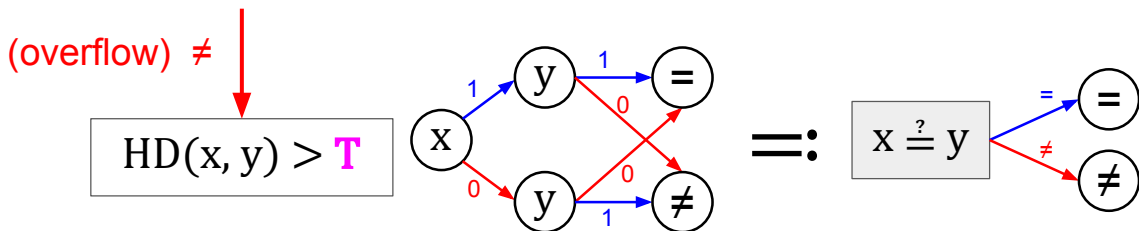


Warmup: branching program for binary Hamming distance

Threshold Hamming distance: $HD(x, y) \stackrel{?}{\leq} T$

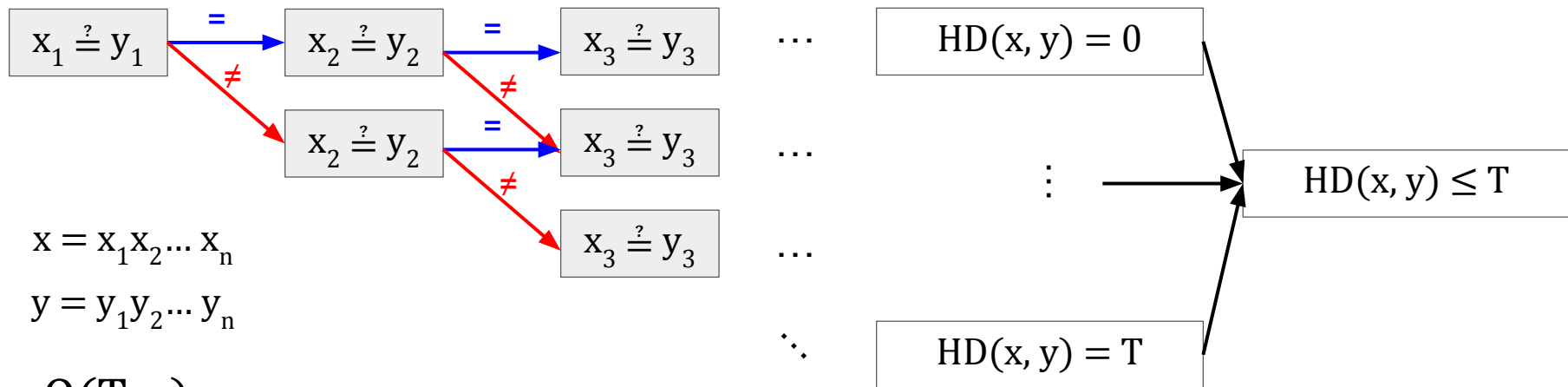


Size $O(T \cdot n)$



Warmup: branching program for binary Hamming distance

Threshold Hamming distance: $HD(x, y) \stackrel{?}{\leq} T$



$$x = x_1 x_2 \dots x_n$$

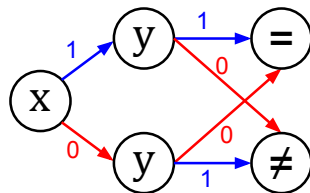
$$y = y_1 y_2 \dots y_n$$

Size $O(T \cdot n)$

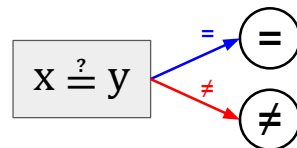
Evaluation cost is one **Mult (5 ms)** per **node**

(overflow) $\neq$

$HD(x, y) > T$

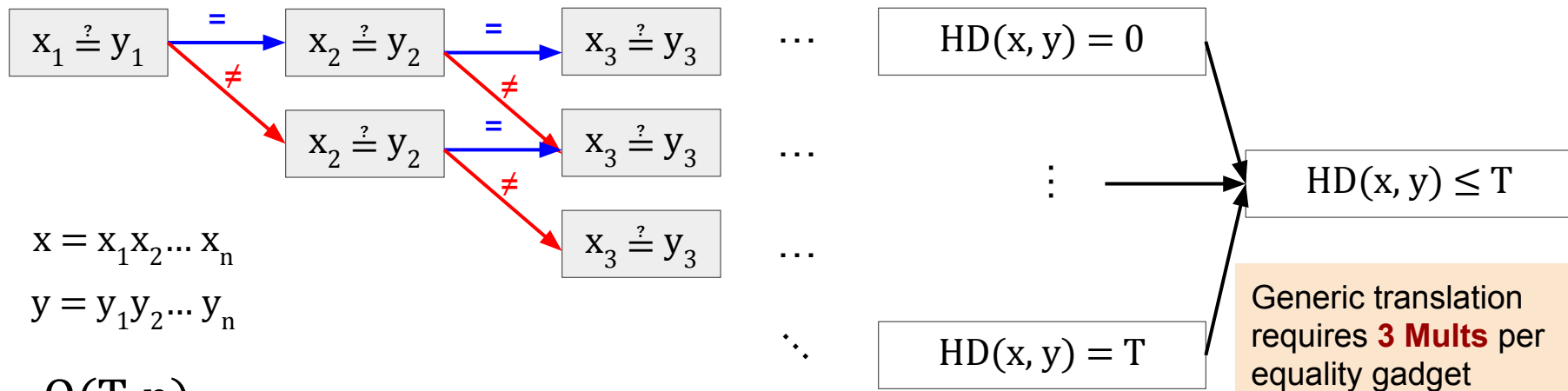


$\equiv$



Warmup: branching program for binary Hamming distance

Threshold Hamming distance: $HD(x, y) \stackrel{?}{\leq} T$

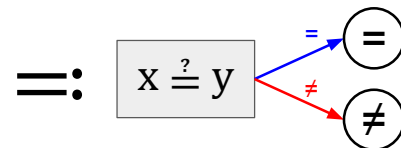
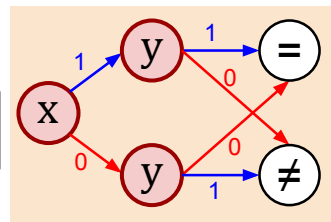
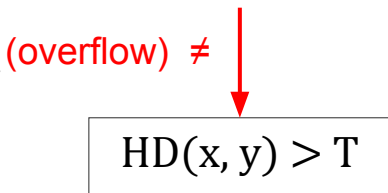


$$x = x_1 x_2 \dots x_n$$

$$y = y_1 y_2 \dots y_n$$

Size $O(T \cdot n)$

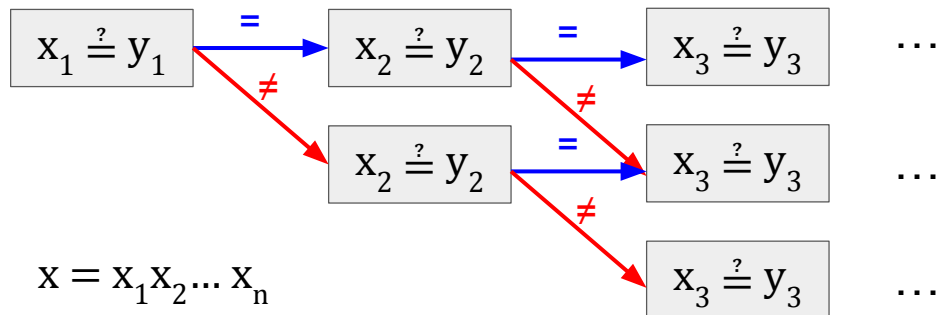
Evaluation cost is one **Mult (5 ms)** per **node** via **generic translation** from branching programs [BCGIO'17]



Warmup: branching program for binary Hamming distance

Threshold Hamming distance: $HD(x, y) \stackrel{?}{\leq} T$

$$(x - y)^2 = \begin{cases} 0 & \text{if } x = y \\ 1 & \text{if } x \neq y \end{cases}$$



$$x = x_1 x_2 \dots x_n$$

$$y = y_1 y_2 \dots y_n$$

$$HD(x, y) = 0$$

⋮

$$HD(x, y) = T$$

$$HD(x, y) \leq T$$

We compute using **2 Mults** instead of **3**:

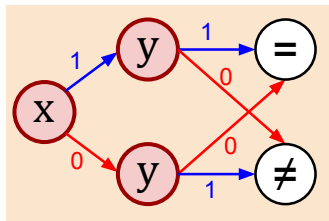
Idea: Compute $(x-y)^2$

Size $O(T \cdot n)$

Evaluation cost is one **Mult (5 ms)** per **node** via **generic translation** from branching programs [BCGIO'17]

(overflow) $\neq$

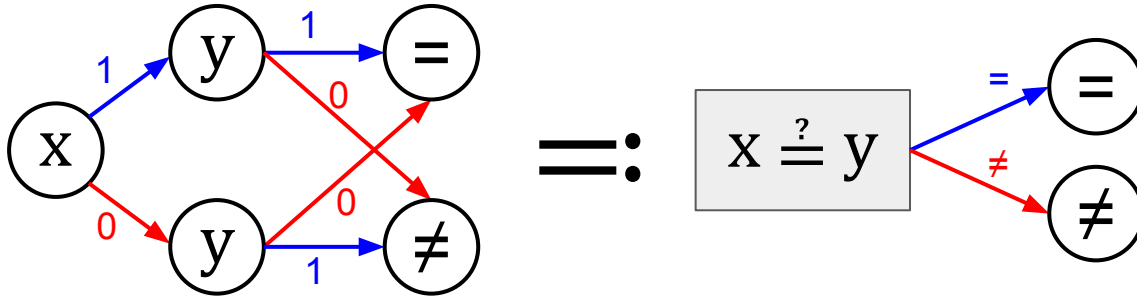
$$HD(x, y) > T$$



$$=: x \stackrel{?}{=} y \begin{cases} = \\ \neq \end{cases}$$

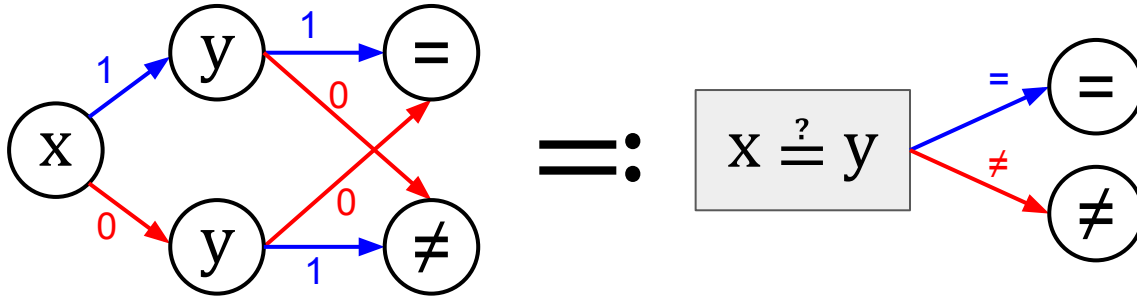
Generalization: Hamming distance over any alphabet

Recall bit equality: $x \stackrel{?}{=} y$



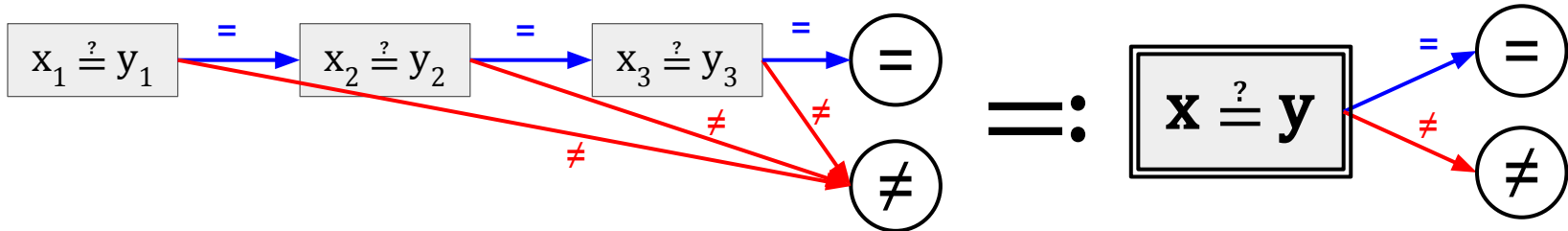
Generalization: Hamming distance over any alphabet

Recall bit equality: $x \stackrel{?}{=} y$



Character equality: $\mathbf{x} \stackrel{?}{=} \mathbf{y}$

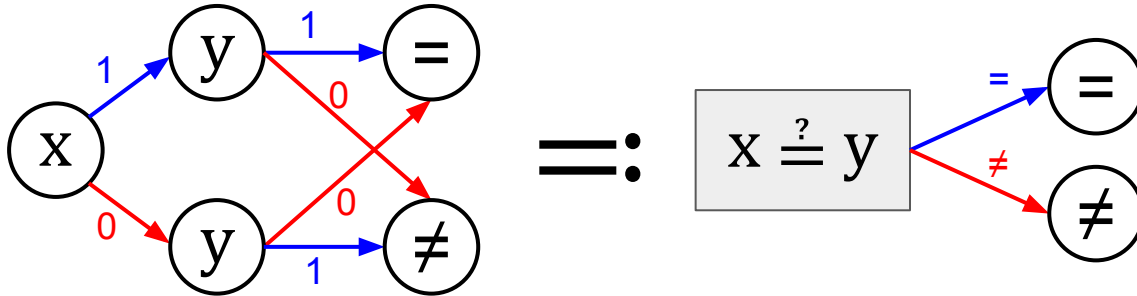
(Encode each character in binary: $\mathbf{x} = x_1x_2x_3$)



Generalization: Hamming distance over any alphabet

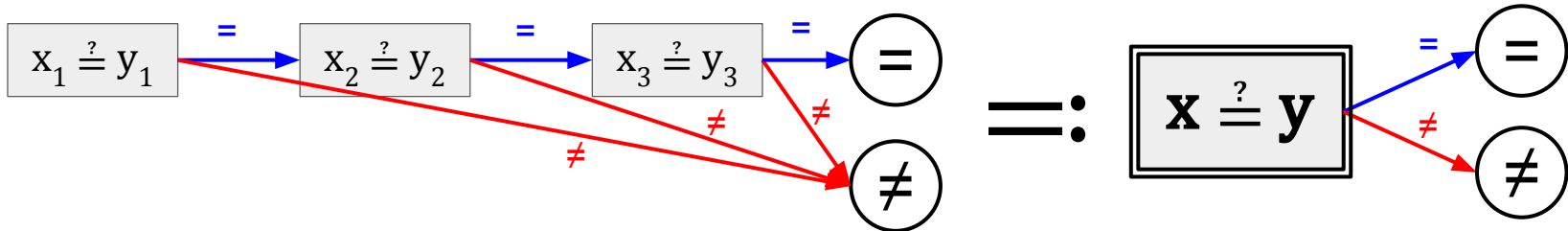
Recall bit equality: $x \stackrel{?}{=} y$

Can also compute a recursive notion of Hamming distance, which tolerates insertions and deletions better. (See Paper)



Character equality: $\mathbf{x} \stackrel{?}{=} \mathbf{y}$

(Encode each character in binary: $\mathbf{x} = x_1 x_2 x_3$)

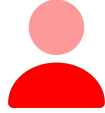


Roadmap

1. Overview of our work
2. MKHSS optimizations
3. Non-interactive conditional key exchange optimizations
- 4. Useful instantiations of key exchange**
 - a. Fuzzy PAKE
 - b. Geolocation-based key exchange
5. Performance evaluation
6. Future works and conclusion

Implementing geolocation-based key exchange

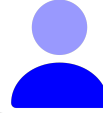
$x =$



pe_A



pe_B



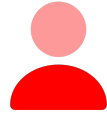
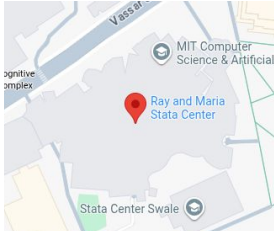
Distance Policy: P

$y =$



Implementing geolocation-based key exchange

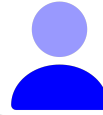
$$\vec{x} = (x_1, x_2)$$



pe_A



pe_B



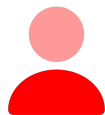
Distance Policy: P

$$\vec{y} = (y_1, y_2)$$



Implementing geolocation-based key exchange

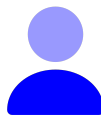
$$\vec{x} = (x_1, x_2)$$



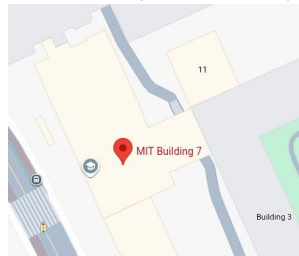
pe_A



pe_B



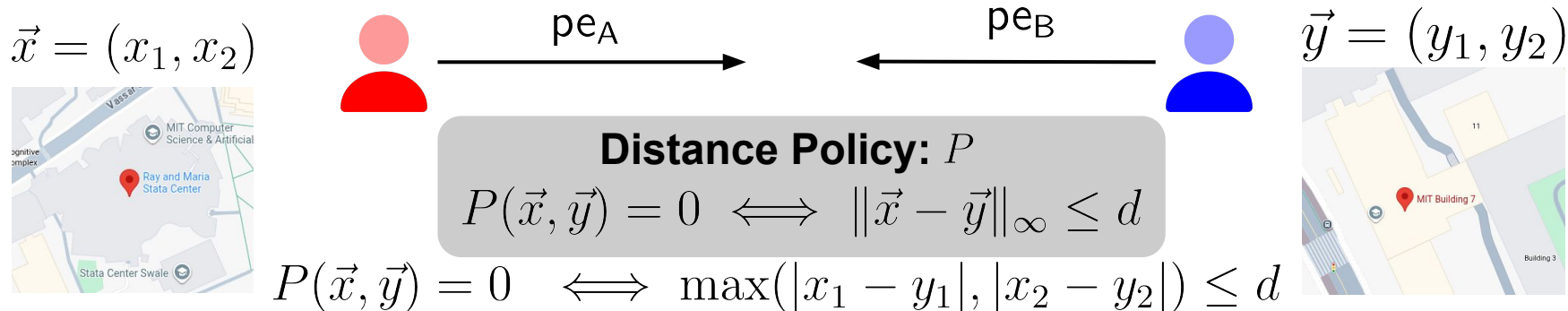
$$\vec{y} = (y_1, y_2)$$



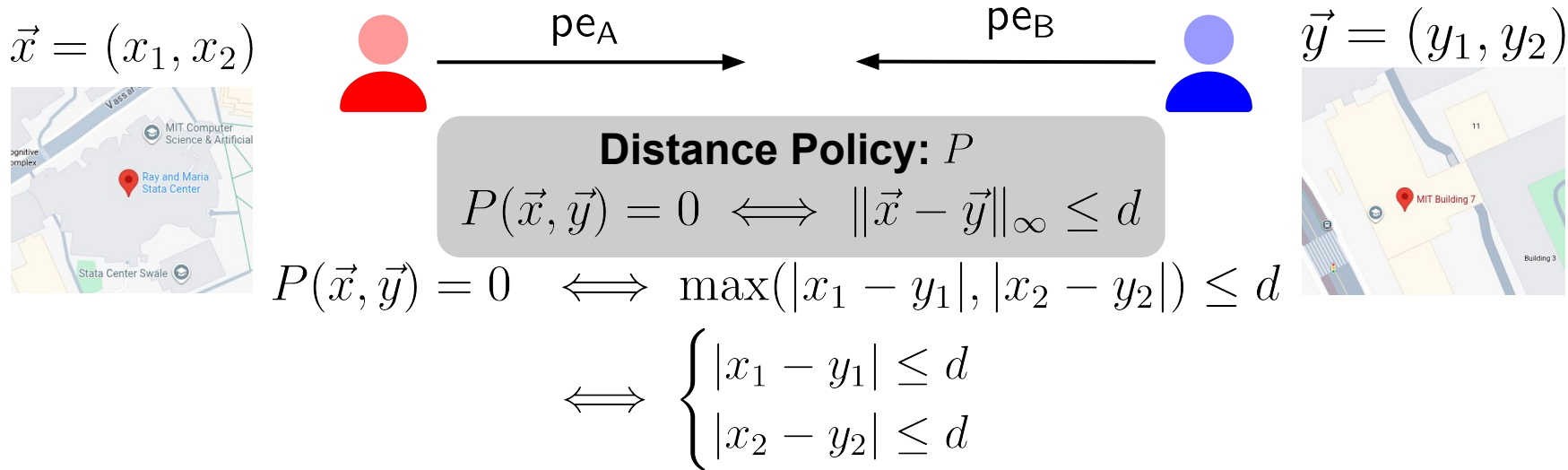
Distance Policy: P

$$P(\vec{x}, \vec{y}) = 0 \iff \|\vec{x} - \vec{y}\|_{\infty} \leq d$$

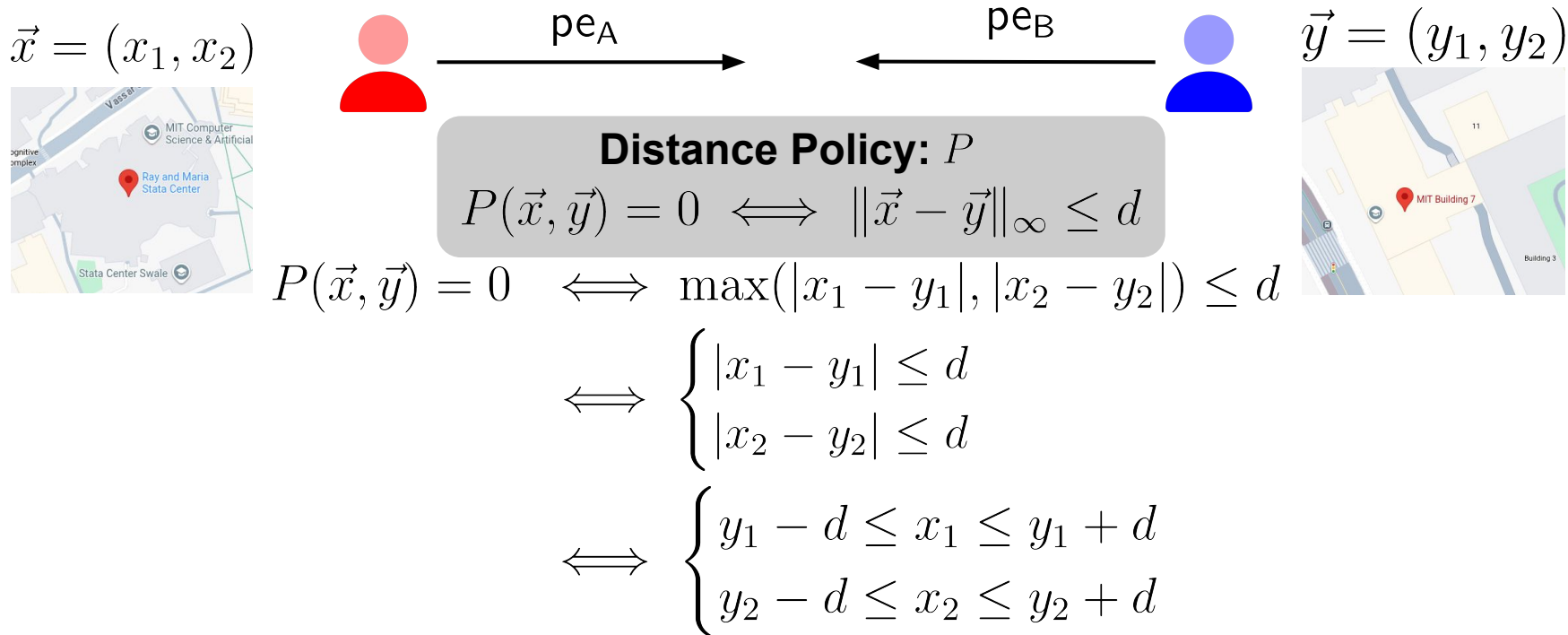
Implementing geolocation-based key exchange



Implementing geolocation-based key exchange

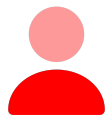
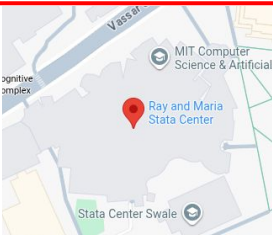


Implementing geolocation-based key exchange



Implementing geolocation-based key exchange

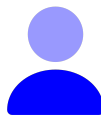
$$\vec{x} = (x_1, x_2)$$



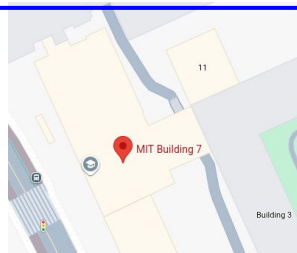
pe_A



pe_B



$$\vec{y} = (y_1, y_2)$$



Distance Policy: P

$$P(\vec{x}, \vec{y}) = 0 \iff \|\vec{x} - \vec{y}\|_{\infty} \leq d$$

$$P(\vec{x}, \vec{y}) = 0 \iff \max(|x_1 - y_1|, |x_2 - y_2|) \leq d$$

Reduces the problem to
four integer comparisons

on

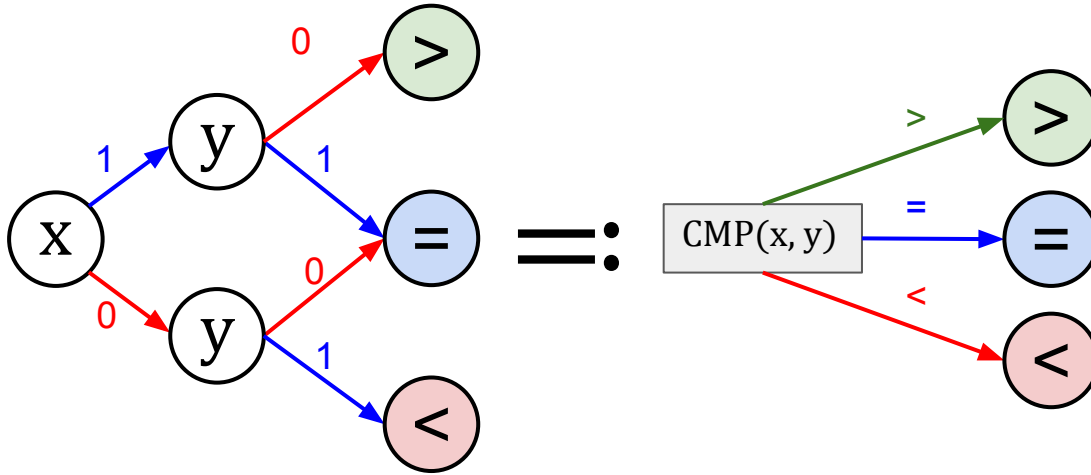
inputs known to some party

$$\iff \begin{cases} |x_1 - y_1| \leq d \\ |x_2 - y_2| \leq d \end{cases}$$

$$\iff \begin{cases} y_1 - d \leq x_1 \leq y_1 + d \\ y_2 - d \leq x_2 \leq y_2 + d \end{cases}$$

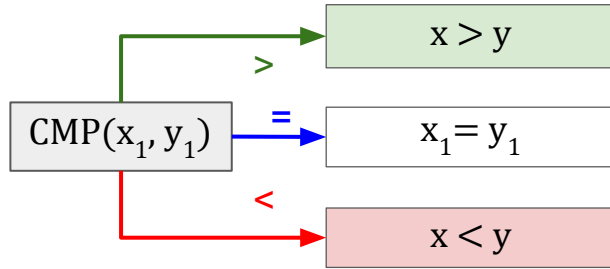
Branching Program for Integer Comparison

Bit comparison: $\text{CMP}(x, y) \in \{<, =, >\}$



Branching Program for Integer Comparison

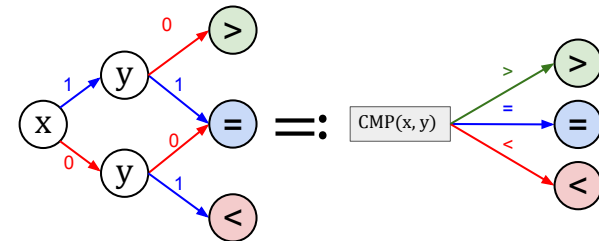
Comparing x and y .



Binary decompositions

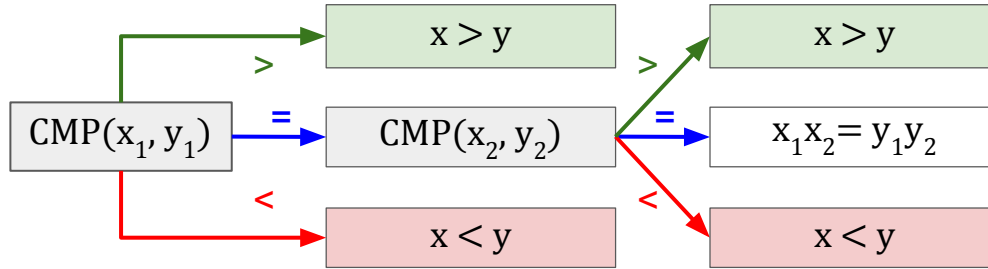
$\text{bit_decomp}(x) = x_1 x_2 \dots x_n$

$\text{bit_decomp}(y) = y_1 y_2 \dots y_n$



Branching Program for Integer Comparison

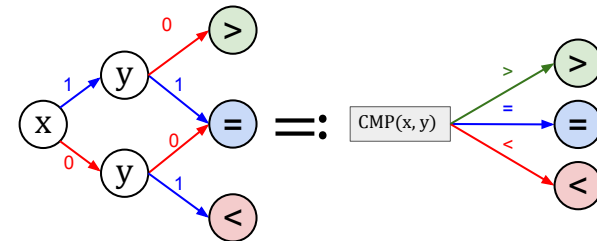
Comparing x and y .



Binary decompositions

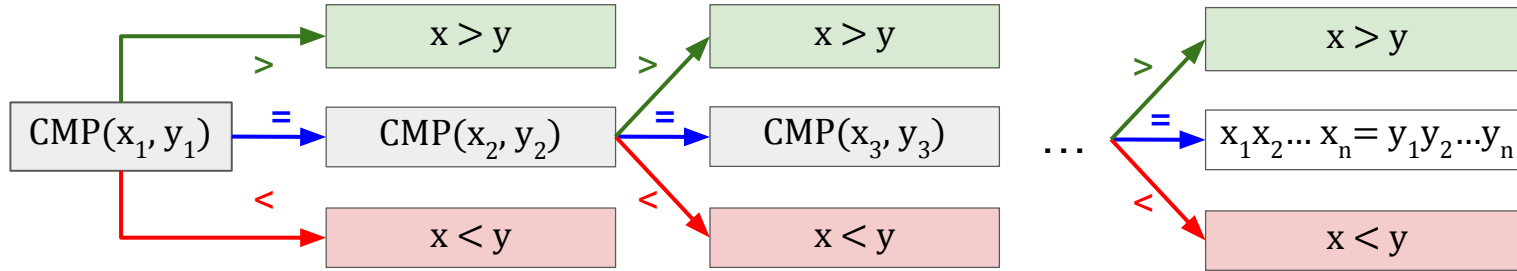
$$\text{bit_decomp}(x) = x_1x_2 \dots x_n$$

$$\text{bit_decomp}(y) = y_1y_2 \dots y_n$$



Branching Program for Integer Comparison

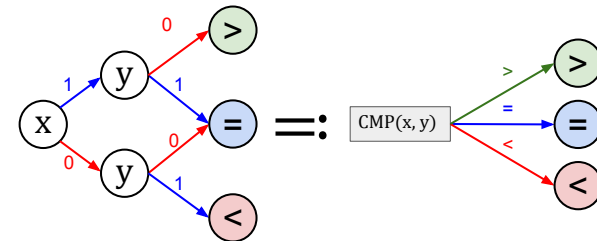
Comparing x and y .



Binary decompositions

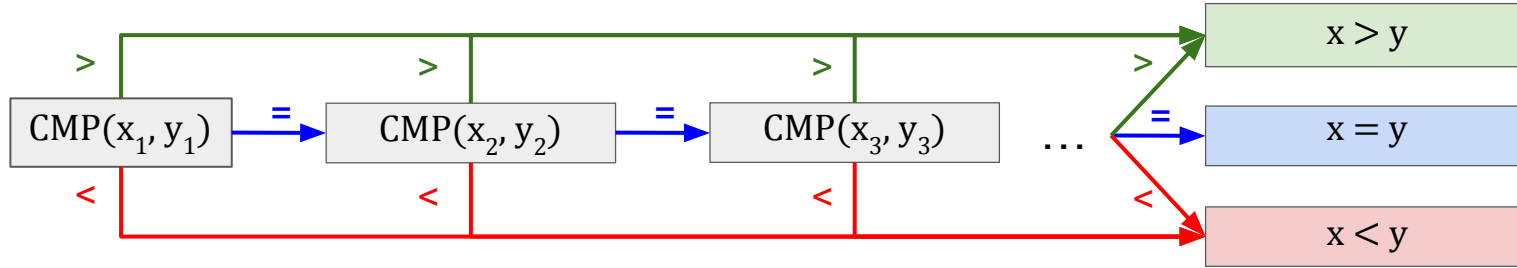
$\text{bit_decomp}(x) = x_1 x_2 \dots x_n$

$\text{bit_decomp}(y) = y_1 y_2 \dots y_n$



Branching Program for Integer Comparison

Comparing x and y .

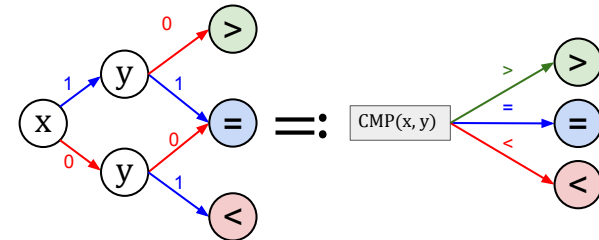


Binary decompositions

$$\text{bit_decomp}(x) = x_1 x_2 \dots x_n$$

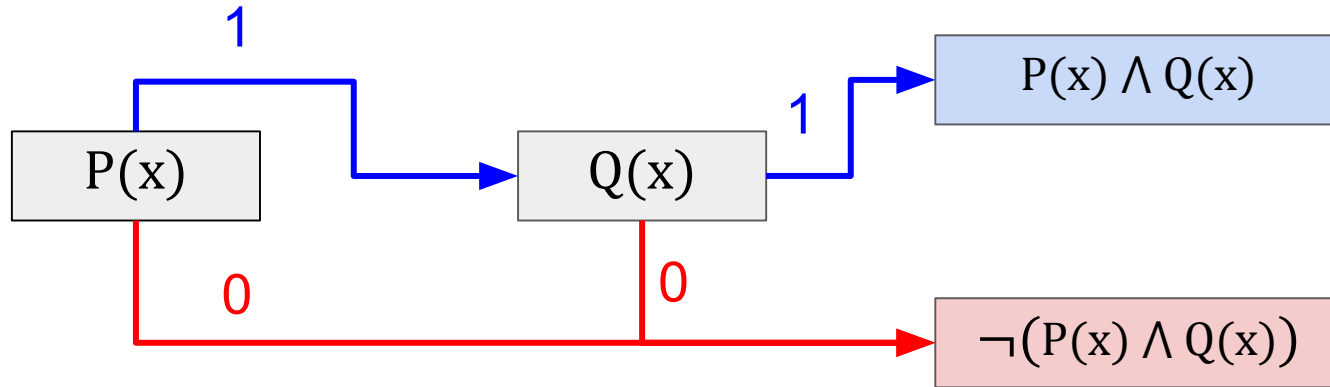
$$\text{bit_decomp}(y) = y_1 y_2 \dots y_n$$

Concrete Cost: $3n$ RMS multiplications



Logical conjunctions

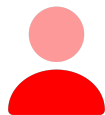
Compute $P(x) \wedge Q(x)$ given branching programs that compute P and Q :



$$\text{Size}(P(x) \wedge Q(x)) = \text{Size}(P(x)) + \text{Size}(Q(x))$$

Implementing geolocation-based key exchange

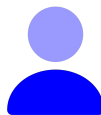
$$\vec{x} = (x_1, x_2)$$



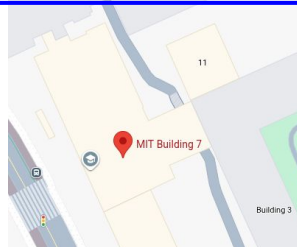
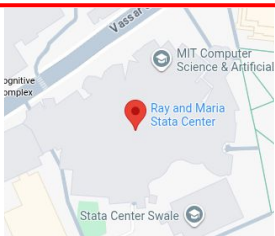
pe_A



pe_B



$$\vec{y} = (y_1, y_2)$$



Distance Policy: P

$$P(\vec{x}, \vec{y}) = 0 \iff \|\vec{x} - \vec{y}\|_{\infty} \leq d$$

$$P(\vec{x}, \vec{y}) = 0 \iff \max(|x_1 - y_1|, |x_2 - y_2|) \leq d$$

Reduces the problem to
four integer comparisons

on

inputs known to some party

$$\iff \begin{cases} |x_1 - y_1| \leq d \\ |x_2 - y_2| \leq d \end{cases}$$

$$\iff \begin{cases} y_1 - d \leq x_1 \leq y_1 + d \\ y_2 - d \leq x_2 \leq y_2 + d \end{cases}$$

Concrete Cost: $4 \times 3n = \mathbf{12\,n}$ RMS multiplications

Extension: “multi-factor” key exchange

Can use logical conjunctions to combine geolocation and passphrase

Roadmap

1. Overview of our work
2. MKHSS optimizations
3. Non-interactive conditional key exchange optimizations
4. Useful instantiations of key exchange
 - a. Fuzzy PAKE
 - b. Geolocation-based key exchange
- 5. Performance evaluation**
6. Future works and conclusion

Parameters

Magnitude bound on HSS values: $B = 1$

(sufficient for all of our key exchange applications, which only uses bits)

Security parameter: $\lambda = 128$

Size of modulus N : 3072 bits, sufficient for 128 bits of security

Runtime: Multi-Key HSS encode

Parameter: $B = 1$ (sufficient for all of our key exchange applications)

Recall this means all intermediate HSS values are in $\{-1,0,1\}$

Procedure	Our runtime (ms)	Baseline runtime (ms)	Our speedup
KeyGen	7.0	104.5	15×
EncodeInput	2.6	10.3	4.0×

Time for each party to encode input $\mathbf{x} = (x_1, \dots, x_n) \in [-B, B]^n$:

$\text{Time}(\text{KeyGen}) + n \cdot \text{Time}(\text{EncodeInput})$

Runtime: Multi-Key HSS evaluate

Parameter: $B = 1$

Procedure	Our runtime (ms)	Baseline runtime (ms)	Our speedup
Init (Alice)	4.4	96.4	22×
Init (Bob)	3.1	50.3	16×
SyncSelfShare	1.3	5.2	4×
SyncOtherShare	1.8	61.8	35×
RMS: Addition	9.5×10^{-3}	38.3×10^{-3}	4.0×
RMS: Multiplication	5.0	224.6	45×

Time to run RMS program $f(x,y)$: $\text{Time}(\text{Init}) + n \cdot \text{Time}(\text{SyncSelfShare}) + m \cdot \text{Time}(\text{SyncSelfShare}) + \text{RMS operations}$

Own input $\mathbf{x}$ $= (x_1, \dots, x_n) \in [-B, B]^n$

Other party's input $\mathbf{y}$ $= (y_1, \dots, y_m) \in [-B, B]^m$

Communication: Multi-Key HSS (continued)

Parameter: $B = 1$

Data	Size (kB)	Size (kB)	Our saving
Transmitted public key	3.1	6.2	2×
Transmitted input share	1.5	4.6	3×

Communication requirement for one party in MKHSS
for input $\mathbf{x} = (x_1, \dots, x_n) \in [-B, B]^n$:

$$\text{Size(pk)} + n \cdot \text{Size(InputShare)}$$

Communication: Multi-Key HSS

Parameter: $B = 1$

Data	Size (kB)	Size (kB)	Our saving
Transmitted public key	3.1	6.2	2×
Transmitted input share	1.5	4.6	3×

The 3× reduction comes from a simplification of input shares

Our Input Share

$$\begin{array}{c} \text{Enc}_{\text{pk}}(x \cdot s) \\ \in \mathbb{Z}_{N^2}^* \end{array}$$

Baseline Input Share

$$\begin{array}{c} \left(\begin{array}{c} \text{Enc}'_{\text{pk}}(x \cdot s) \\ \text{Enc}_{\text{pk}}(x) \end{array} \right) \\ \in \mathbb{Z}_{N^4}^* \times \mathbb{Z}_{N^2}^* \end{array}$$

Bit Length

$$2 \log_2(N)$$

$$6 \log_2(N)$$

Concrete Performance: Fuzzy PAKE Runtime

# chars in password	Bits per char	# typos permitted	Our runtime (sec)	Baseline runtime* (sec)	Our speedup
72	5	2	7.56	252 (~4 mins)	33×
80	16	1	19.7	678 (~11 mins)	34×
120	8	3	27.5	920 (~15 mins)	33×

*: We gave the baseline an advantage by allowing it to use the random-oracle-based construction of key exchange from MKHSS.

Thus the difference is solely due to MKHSS speedups.

Evaluation: Fuzzy PAKE runtime

# chars in password	Bits per char	# typos permitted	Our runtime (sec)	Baseline runtime* (sec)	Our speedup
72	5	2	7.56	252 (~4 mins)	33×
80	16	1	19.7	678 (~11 mins)	34×
120	8	3	27.5	920 (~15 mins)	33×

Further speedup possible with AVX512

[Langowski-Devadas'25]

Our runtime with AVX512 (sec)
3.17
8.47
11.7

Evaluation: Fuzzy PAKE communication

# chars in password	Bits per char	# typos permitted	Our communication cost (MB)	Baseline communication cost (MB)	Our savings
72	5	2	1.1	3.3	3×
80	16	1	3.9	11.8	3×
120	8	3	3.0	8.9	3×

Our **3×** reduction comes from a reduction in the size of input shares of MKHSS

Runtime: Geolocation-Based Key Exchange

# dimensions for coordinate	Precision of coordinate (bits)	Our runtime (sec)	Baseline* runtime (sec)	Our speedup
2	32	1.65	54.1	33×
3	48	3.63	122	33×
4	64	6.46	216	33×

Communication: Geolocation-Based Key Exchange

# dimensions for coordinate	Precision of coordinate (bits)	Our communication cost (kB)	Baseline communication cost (kB)	Our savings
2	32	301.1	897.2	3×
3	48	669.8	2003.1	3×
4	64	1185.9	3551.4	3×

Roadmap

1. Overview of our work
2. MKHSS optimizations
3. Non-interactive conditional key exchange optimizations
4. Useful instantiations of key exchange
 - a. Fuzzy PAKE
 - b. Geolocation-based key exchange
5. Performance evaluation
6. Future works and conclusion

Future directions

- Add malicious security to key exchange with minimal performance overhead?
 - This can be done generically with zero knowledge proofs
- Application of our structural simplification to other HSS protocols? Recall:
 - $(\text{Enc}(\cancel{x}), \text{Enc}(x \cdot s)) \rightarrow \text{Enc}(x \cdot s)$
 - $(\langle \cancel{x} \rangle_\sigma, \langle x \cdot s \rangle_\sigma) \rightarrow \langle x \cdot s \rangle_\sigma$
- Other useful predicates for key exchange?
- Other practical applications of MKHSS?

Thank you!

Paper (Lali's talk): <https://ia.cr/2025/094>

Paper (Kevin's talk): <https://ia.cr/2025/1803>

Code: <https://github.com/kevin-he-01/mkhss>

References

- [ADOS'22] D. Abram, I. Damgård, C. Orlandi, and P. Scholl. An algebraic framework for silent preprocessing with trustless setup and active security.
- [BCGIO'17] E. Boyle, G. Couteau, N. Gilboa, Y. Ishai, and M. Orrù. Homomorphic secret sharing: Optimizations and applications.
- [BCP'03] E. Bresson, D. Catalano, and D. Pointcheval. A simple public-key cryptosystem with a double trapdoor decryption mechanism and its applications.
- [BGI'16] E. Boyle, N. Gilboa, and Y. Ishai. Breaking the circuit size barrier for secure computation under DDH.
- [CDHJS'25] G. Couteau, L. Devadas, A. Hegde, A. Jain, and S. Servan-Schreiber. Multi-key homomorphic secret sharing.
- [DH'76] W. Diffie and M. E. Hellman. New directions in cryptography.
- [DHPRY'18] P.-A. Dupont, J. Hesse, D. Pointcheval, L. Reyzin, and S. Yakoubov. Fuzzy password-authenticated key exchange.
- [DJ'03] I. Damgård and M. Jurik. A length-flexible threshold cryptosystem with applications.
- [FHKP'13] E. S. V. Freire, D. Hofheinz, E. Kiltz, and K. G. Paterson. Non-interactive key exchange.
- [Langowski-Devadas'25] S. Langowski and S. Devadas. Efficient modular multiplication using vector instructions on commodity hardware.